

NCHU - [DRL][FR] Final Proposal Phase 7

media.mp4

檔案

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Smart_City_Drone_Law_Enforcement.pdf

PDF

Frank_oral_defense_0725_v7.pptx

簡報

我想要開發一個 Deep Reinforcement Learning 專案

如附圖 1 ilearning 平台上的描述

但我現在還沒找到要做的主題 請你建議我

請你在完理解過教授要的 proposal 架構後再推薦我最適合用教授這套 A-to-Z 的論文寫作方式做 **final project proposal**

附檔是教授提供的 reference 請幫我完整全部檔案都看完, 理解後

推薦我最適合套用進此流程的 DRL 題目

附上教授 A-to-Z 的論文寫作流程影片介紹:

<https://www.youtube.com/watch?v=MqwmP4CJnSE>

顯示更多顯示較少

我看完了你貼的 ilearning 作業說明，也把三個附檔都過了一遍。我的判斷很明確：

最適合你拿來做這門 DRL final project proposal 的題目，是：

「用 Deep Reinforcement Learning 做都市路口自適應交通號誌控制（Adaptive Traffic Signal Control）」。

你附的兩份 drone reference，其實是在示範同一種教授喜歡的敘事方式：先把問題的重要性講大，再整理 related work，接著把方法流程、資料/系統設計、實驗比較、限制與未來導入講清楚。口試簡報版明確就是 **Motivation → Related Works → Proposed Method → Experimental Results → Conclusion**，還把資料量、模型、限制都量化寫出來；Smart City PDF 則把它壓成「問題—系統流程—實驗—風險—落地路線圖」的簡報故事。至於 2330 那份雖然不是 DRL 論文，但它很像一個「固定頁面模板 + 關鍵結論 + 證據框 + 決策邏輯」的視覺報告範本，這對你做 proposal 投影片其實很有參考價值。

Frank_oral_defense_0725_v7 Smart_City_Drone_Law_Enforcement

2330_SMC_FVG_MTF_Report_A4_Port...

再加上你附的影片頁面就是 **Project Proposal A to Z (字幕外加)**；搭配你上傳的 mp4 內容，我理解教授現在要的不是「先把系統全部做完」，而是要你先把整條論文故事線講順：**這個問題為什麼值得做、現在卡在哪裡、你打算怎麼設計、未來要怎麼驗證**。影片裡對 abstract 的要求其實很清楚，就是 **Attention Getter / Motivation → But → Cure → Development → Experiments → Findings**；而實驗段落也要先想好 **baseline、metrics、setup、tuning、useful results**。  YouTube

為什麼我最推薦「DRL 自適應交通號誌控制」

第一，它**最貼近教授 reference 的味道**。你手上的 reference 是智慧城市、無人機、執法、系統導入，核心共同點不是「影像辨識」而已，而是**真實城市問題 + AI/系統化設計 + 可驗證的實驗**。交通號誌控制同樣是很典型的智慧城市議題，問題好講、價值大、實驗也能量化。

第二，它**非常適合 A-to-Z 的 proposal 寫法**。

這題天然就能寫成：

- Attention Getter / Motivation：都市壅塞、固定時制浪費通行效率
- But：固定式或傳統 rule-based 控制難以適應動態車流
- Cure：用 DRL 根據即時狀態決定號誌切換
- Development：定義 state / action / reward，建立 SUMO 模擬環境
- Experiments：和 fixed-time、actuated control、DQN/PPO 比較
- Findings：看平均等待時間、queue length、travel time 是否下降

第三，它**工具鏈成熟、可重現、風險低**。SUMO 是官方維護的開源微觀交通模擬器，能處理大型路網；SUMO-RL 已經把 Traffic Signal Control 包成可直接拿來做 RL 的環境，而且支援 Gymnasium、Stable-Baselines3、RLlib，也能改 state 和 reward；Gymnasium 本身提供標準 RL API 與自訂環境教學；Stable-Baselines3 則提供穩定的 PyTorch 演算法實作，像 PPO 與 DQN 都可直接用。🌱 [Eclipse SUMO+7](#)

第四，它**很容易控制專案範圍**。你可以先做「單一路口」拿到能跑的結果，再決定要不要擴成「多路口」或「multi-agent」。SUMO-RL 也已經明講支援 multi-agent；PettingZoo 則是 Farama 官方的多智能體 RL 介面標準。也就是說，這題從保守版到進階版都有路可以走。📄 [Lucas Alegre+1](#)

我會怎麼幫你定這題的正式題目

我建議你不要只寫成很泛的「Traffic Signal Control using DRL」，而是寫成：

中文題目

以需求感知深度強化學習進行都市路口自適應交通號誌控制

英文題目

Demand-Aware Deep Reinforcement Learning for Adaptive Traffic Signal Control at Urban Intersections

這樣的好處是你 proposal 裡的「novelty」比較容易站住腳：

你不是只說「我用 DRL」，而是說你要做 **demand-aware state design** 或 **reward shaping**，讓 agent 不只顧單一方面，而是同時兼顧等待時間、排隊長度與通行效率。

這題最適合的 proposal 骨架

1. Introduction / A-to-F

先講城市壅塞與固定號誌的侷限，再指出現有方法在動態需求、尖離峰切換、不同流向公平性上的不足，最後收斂到你的研究問題：

「能不能讓 DRL 根據即時車流自適應調整號誌，相比固定時制顯著降低等待成本？」

2. Related Work / NotebookLM

文獻建議分三條線整理：

- 傳統交通號誌控制：fixed-time、actuated、rule-based
- RL / DRL traffic signal control：Q-learning、DQN、PPO、multi-agent
- reward / state design：queue、delay、pressure、fairness

教授 reference 不是單純列 paper，而是有「從舊到新、從簡單到複雜」的演進感，這點你要學。

Frank_oral_defense_0725_v7

3. Proposed Design / 規格書

這一段你要寫得像系統設計，而不是只有概念：

- Environment：SUMO / SUMO-RL
- Agent：先用 DQN 或 PPO
- State：各車道 queue length、waiting time、current phase、phase elapsed time
- Action：切換到哪個 phase，或維持 / 切換
- Reward：負的總等待時間 + queue penalty
- Baselines：fixed-time、actuated control
- Metrics：average waiting time、queue length、throughput、travel time

我建議你的最小可行版本

不要一開始就做太大。最穩的版本是：

- 場景：單一路口
- 演算法：DQN、PPO 二選一，外加 fixed-time baseline
- 目標：讓 average waiting time 明顯下降
- 可選加分點：再加入第二個 reward term，處理 fairness 或尖峰流量

這樣 proposal 好寫，final 也比較做得完。

我不推薦你現在優先選的題目

1. 無人機巡航路徑規劃 DRL

題目很帥，也跟 reference 很像，但實作風險高。AirSim 官方雖然有 RL 範例，但文件自己也說這一塊仍在 active development；你很容易把時間花在模擬器與飛行控制，而不是論文主體。  Microsoft GitHub

2. CARLA 自動駕駛 DRL

CARLA 本身很強，是為自駕研究打造的開源模擬平台；但官方 RL repo 也明寫了只放 inference code、training code 沒有釋出。這種題目工程量通常比課程 final project 大很多。  CARLA Simulator+1

三個候選題目的結論比較

題目	和教授 reference 的契合度	Proposal 好寫程度	實作風險	我建議
DRL 自適應交通號誌控制	9.5/10	9.5/10	4/10	最推薦
DRL 無人機巡航 / 巡檢路徑規劃	8.5/10	7/10	7.5/10	第二順位
DRL 自動駕駛 (CARLA)	7/10	6/10	9/10	不建議當首選

我的最終建議

你的情況下，最穩、最像教授會買單、最能套進 A-to-Z 流程的，就是「DRL 自適應交通號誌控制」。

因為它同時滿足四件事：

- 有**真實世界動機**
- 有**明確 baseline 與 metrics**
- 有**成熟工具鏈**
- 有**好寫的論文故事**

你決定選這題的話，我下一則就直接幫你寫成**教授格式的 final project proposal 初稿**，包含：
題目、abstract、introduction、related work 架構、proposed design、experiment plan、預期貢獻。

<https://www.youtube.com/shorts/qeNim1T082Y>

這個盲人眼鏡開源專案 可以嗎
也幫我把 github 位置找出來

可以，但要分開看：

如果你是問「**這個題目能不能當 final project proposal**」：可以。

如果你是問「**可不可以直接當 DRL 專案原封不動拿去做**」：我不建議。

你貼的這支 Shorts 看起來對應的是近來流傳的那個「95 後手搓 AI 導盲眼鏡」開源專案；相關文章直接指向同一個 GitHub 倉庫，而該 repo 自己也寫明這是一個面向視障人士的智能導航與輔助系統。 [YouTube+2](#)

這個專案本身其實**很完整，也很有 proposal 故事性**：它整合了盲道導航、過馬路輔助、物品識別查找、即時語音互動；技術上用到 YOLO 分割做盲道辨識、YOLO-E + ByteTrack 做目標追蹤、MediaPipe 做手部引導，並結合 Qwen-Omni 與阿里雲 DashScope 做語音與多模態互動。repo 也標示為 MIT license。 [GitHub+1](#)

但如果你的課是 **Deep Reinforcement Learning**，那這個 repo **目前不是一個 DRL 專案**。我查它的公開 README，找不到 reinforcement learning、RL、DQN、PPO 之類的設計痕跡；從結構看起來，它更像是**電腦視覺 + 多模態語音 + 邊緣裝置整合**的 assistive AI 系統，而不是用 agent 學 policy 的 DRL 研究。 [GitHub+3](#)

所以我的判斷是：

結論一：可以當靈感來源，但不要直接把它當成「DRL 題目」交上去。

因為教授要的是可以用 A-to-Z 論文方式講清楚的 proposal，而這個專案原型雖然很吸睛，卻比較偏系統整合。若你直接照搬，老師很可能會問：「你的 reinforcement learning 在哪裡？」 [GitHub+1](#)

結論二：最好的做法，是把它改寫成一個“以這個導盲眼鏡為應用場景”的 DRL 題目。

例如你可以改成：

- **DRL-based Assistive Navigation Policy for Blind-Path Following and Obstacle Avoidance**
- **Using Deep Reinforcement Learning to Optimize Audio Guidance for Smart Glasses Navigation**
- **A DRL Decision Module for Crosswalk Assistance in Smart Glasses for the Visually Impaired**

這樣就能把原本的 CV 感知模組保留成「環境輸入」，而把真正的研究貢獻放在 **agent 如何根據狀態決定下一步提示/方向/避障策略**。這樣比較符合 DRL 課程定位，也更容易寫成教授喜歡的 A-to-Z 結構。

GitHub 位置

倉庫是：[AI-FanGe/OpenAIGlasses_for_Navigation](#)。文章與 repo 互相對得上，repo 首頁也明確寫它是「a open framework for blind navigation based on esp32」。[知乎專欄+1](#)

另外提醒你兩個現實面問題。第一，repo 自己明寫**僅供交流學習，不要直接給視障人群使用**，所以你 proposal 最好定位成研究原型，不要寫成 ready-to-deploy 產品。第二，它的 README 顯示需要 GPU、CUDA、ESP32-CAM、DashScope API key，而且 quick start 裡的 clone 指令還留著像 placeholder 的路徑，代表專案雖然開源，但工程細節還有不少要自己補。[GitHub+2](#)

我的建議會是：

這題可以做，但要改成「導盲眼鏡中的 DRL 決策/導航策略」而不是「開源導盲眼鏡系統重現」。

你若要，我下一步可以直接幫你這個題目改寫成一份**符合教授 A-to-Z 架構的 proposal 題目 + Abstract + Introduction + Related Work + Proposed Design**。

另外如果是 MemPalace 這個 AI Agent 永久記憶開源專案呢 是否適合當作題目
且因為 MemPalace 的亮點就是能把 AI 的記憶 "本地化" 如果這個本地化模型還要跑重武器 DRL 有點本末倒置 (本地化就是要越輕量越好)
所以我想從「輕量化 DRL」的切入方向
不要用深度神經網絡 (Deep Q-Network)，改用 Multi-Armed Bandit (MAB, 多臂老虎機) 或 輕量級 Q-Learning

但我連 ai agent 的架設都不會 也沒用過 這樣的程度適合做這個題目嗎
不太適合的話是否有折衷方案 連 ai agent 概念都不用摸呢
[顯示更多顯示較少](#)

可以，但我會先給你一個很務實的判斷：

MemPalace 適合當靈感來源，也適合當研究對象；但不適合你現在這個程度直接做成「完整 AI agent 記憶系統實作」的 final project。

因為 MemPalace 本身不是小玩具，它是完整的 **local-first AI memory** 系統：用語意搜尋保留逐字記憶、預設後端是 ChromaDB、知識圖譜用本機 SQLite，還有 MCP 工具、hooks、agent diary、benchmark 等完整配套。官方 repo 也主打本地化、零 API、LongMemEval raw R@5 96.6%。[GitHub+2](#)

你抓的方向其實是對的：**如果題目核心價值是「本地化、輕量化、低依賴」**，那再往上疊一個需要重訓練、重推論的 DQN，確實很可能會讓題目失焦。MemPalace 官方自己也一直強調 bounded wake-up context，大約只載入 L0+L1 的 ~600-900 tokens，且搜尋是按需觸發，不是整包上下文全塞進去；這種設計哲學本來就偏向「資源節制」而不是「深模型硬壓」。這是你可以在 proposal 裡合理主張的切入點。[mempalaceofficial.com+2](#)

但要注意一個關鍵：**MAB 或 tabular Q-learning 並不等於 Deep RL。**

Contextual bandit 比較像是「每回合看到 context，選一個 action，只看到被選 action 的 reward」，是介於 supervised learning 與 RL 之間的問題；tabular Q-learning 則是完整 RL，但它假設你有可管理的有限 state/action，透過探索與更新表格來學策略。官方與教材都把 contextual bandit 和 Q-learning 定義得很清楚，所以如果你的課名是 *Deep Reinforcement Learning*，你若只做純 MAB，教授有機會質疑「這題不夠 deep」。[Microsoft GitHub+2](#)

所以我對你這題的建議不是「放棄 MemPalace」，而是：

最適合你的版本

不要做完整 agent。改做：

「本地記憶系統中的輕量化決策層」

也就是把 MemPalace 當成既有記憶後端，你的研究重點不是重造一個 agent，而是研究：

在 token budget、延遲限制、不同 query 類型下，
要用哪一種記憶存取策略最划算？

這樣你的題目就會很乾淨。

我最推薦的題目版本

題目 A：最穩、最適合你

A Contextual Bandit for Token-Budgeted Local Memory Retrieval

白話版就是：

每來一個 query，bandit 根據 context 決定要走哪條 retrieval 路線。

例如 action 可以是：

- 只用 wake-up
- 只用 search
- 先 wake-up 再 search
- 只查某個 wing
- 查 knowledge graph
- top-k 取 3 / 5 / 10

這些 action 都很小、很離散，很符合 contextual bandit「選有限個 action、看 chosen action reward」的型態。MemPalace 官方也提供了不用 agent 的 CLI 與 Python API：你可以直接跑 `mempalace wake-up`、`mempalace search`，或用 `search_memories()` 之類的 Python API 把結果接進你自己的實驗程式，不一定要碰 MCP agent。🔗 mempalaceofficial.com+2

這題最大的優點是：

你完全可以不會架 AI agent，也能做。

第二順位：稍微更像 RL

題目 B：進階但還可控

Tabular Q-Learning for Multi-Step Local Memory Access Policies

這版把問題變成多步決策，例如：

1. 先讀 wake-up

2. 判斷要不要搜 wing
3. 搜不到再深搜
4. 最後決定回傳多少內容

這時候就不是 one-shot bandit，而是有狀態轉移的 sequential decision problem，tabular Q-learning 在概念上比較合理。Q-learning 本來就是不需要先知道完整 MDP，就能從互動資料學 action-value；而且 Gymnasium 也有官方 tabular Q-learning 教學與自訂環境入口，對學生來說比 DQN 友善很多。

 Dive into Deep

Learning+1

但這題比題目 A 稍難，因為你要自己定義 state transition。

你現在的程度，適不適合？

我的判斷是：

做「完整 MemPalace agent + 本地模型 + MCP + hooks + RL」：不適合。

MemPalace 官方文件已經很明白，它平常的使用方式會牽涉到 MCP integration、Claude Code plugin、Gemini CLI、hooks；而「specialist agents」目前官方 docs 主要描述的是透過 MCP tools 寫 diary 的 workflow，不是那種你開箱即用就有完整 agent orchestration 的教學專案。對一個還沒碰過 AI agent 架設的人，這個入口太陡。  mempalaceofficial.com+2

做「MemPalace 當後端 + 你自己寫一個輕量 decision layer」：適合。

因為你可以把所有 agent 概念先拿掉，只保留：

- query
- retrieval action
- reward
- evaluation

這樣本質上就是一個 machine learning / RL 實驗題，不是 agent engineering 題。

如果你想連 agent 概念都不要碰

那我建議你直接再退一步，做成：

題目 C：最保守、最容易成功

Lightweight RL for Local Memory Retrieval Strategy Selection

這題甚至可以不真的安裝 MemPalace。

你只要自己準備一個小型資料集，例如：

- 一批對話紀錄 / 筆記 / 文件片段
- 一組 query

- 每個 query 對應的 relevant memory 片段標註

然後模擬幾種 retrieval strategy：

- 全域搜尋
- 分 room 搜尋
- 限 top-k
- 先短上下文再深搜
- 查知識圖譜/metadata 或不查

最後用 contextual bandit 或 tabular Q-learning 去學「哪種 query 應該走哪種策略」，reward 可以是：

$$\text{reward} = \text{recall@k} - \alpha \cdot \text{token_cost} - \beta \cdot \text{latency}$$

這樣你：

- 不用會 agent
- 不用會 MCP
- 不用接本地 LLM
- 甚至不用碰 MemPalace 真正的工程堆疊

但論文故事仍然成立，而且非常貼近你在意的「本地化」「輕量化」「不要重武器」。這個方向也跟 contextual bandit 在互動式推薦、客服 agent 裡的實際應用邏輯很接近；Microsoft 就有把 contextual bandits / MAB 用在 virtual agent 的推薦與意圖消歧上，重點正是 partial feedback 與資料效率。  [arXiv+1](#)

我給你的最終建議

若以你現在的程度，我會這樣排序：

最推薦：

題目 A：Contextual Bandit for Token-Budgeted Local Memory Retrieval

因為最輕、最不需要 agent、最符合你要的本地化邏輯。

次推薦：

題目 C：完全不碰 agent 的 retrieval strategy selection

最保守，成功率最高。

不推薦現在做：

完整 MemPalace agent 系統 + RL 強化

因為你會先死在系統整合，不是死在研究本身。

一句話結論

MemPalace 可以做，但不要把題目定成「做一個 AI agent 永久記憶系統」。

對你最好的版本是把它改成：

「本地記憶系統中，如何用輕量化 RL / contextual bandit，在 token 與延遲限制下選擇最佳 retrieval 策略？」

這樣你幾乎可以完全不碰 agent 概念，卻還是保留 MemPalace 的亮點與研究價值。官方 GitHub 就是 [MemPalace/mempalace](#)。 

下一步最有幫助的做法，是我直接幫你把這題寫成教授 A-to-Z 架構的 final project proposal 題目 + abstract + 方法設計。

看完你的說明 我決定不要 MemPalace, 盲人眼鏡 這兩個題目了
所以我要請你再推薦我其他三個題目

好，我這次不把先前提過的交通號誌控制算進來，直接給你 3 個新的 DRL final project 題目。我挑題的標準是：

要能套進你教授那種 Motivation → Related Work → Proposed Method → Experiments → Findings 的 A-to-Z 結構、不用碰 AI agent 架設、有現成環境或官方工具可用，而且對你現在的程度不會太失控。

Gymnasium 提供標準 RL 環境介面與自訂環境教學，Stable-Baselines3 則有可靠的 DQN / PPO 實作，這代表你可以把重心放在「題目與實驗設計」，而不是先卡死在基礎框架。  [健身房文檔+5](#)

1. 用 DRL 做庫存補貨策略最佳化

我最推薦你把這題當成**第一順位**。原因很簡單：它最不需要碰 agent 概念，問題定義也最乾淨。你可以把它寫成「在需求波動下，agent 如何決定每期補貨量，以最小化缺貨成本、持有成本與訂購成本」。這種題目非常適合教授的 A-to-Z：動機很好講，related work 可以寫傳統 EOQ / heuristic / OR 方法，method 可以定義 state、action、reward，experiments 則直接和 base-stock policy、heuristic policy 或 OR baseline 比。OR-Gym 這條線本來就是把 knapsack、bin packing、multi-echelon inventory management 等經典 OR 問題重寫成 RL 任務，並且和 MILP、heuristics 做 benchmark；現在也有更新到 Gymnasium 相容的 inventory repo。  [arXiv+3](#)

如果你想控制難度，這題還能自然分層：

最簡版做**單倉、單品項、離散補貨量**；中階版做**多倉或 lead time**；進階版才碰 multi-echelon。你甚至可以先用 tabular Q-learning 做很小的 state space，再升級成 DQN 或 PPO，這樣既照顧你目前程度，也比較符合課程的 DRL 味道。對你來說，它最大的優點是：**實驗好跑、baseline 好找、失敗也容易 debug**。  [arXiv+2](#)

2. 用 DRL 做建築能源 / 電池排程控制

這題是我給你的**第二順位**，也是三題裡「最像當代應用論文」的一題。CityLearn 本來就是為建築能源管理、需求響應與多智能體 RL 研究建立的標準化環境，官方文件明講它的目的是讓不同 RL 演算法在同一套城市建築能源協調問題上可以被公平比較；相關論文也指出，城市建築與需求響應是 RL 很有代表性的真實應用場景。CityLearn v2 甚至擴充到更多 building control 問題與 benchmark。  [citylearn.net+3](#)

這題很適合教授的 proposal 架構，因為故事線很完整：

Motivation 可以講節能、尖峰削減、電網壓力；But 可以講 rule-based control 或 MPC 的侷限；Cure 就是 DRL；Experiments 可以比較 rule-based control、MPC、PPO/DQN 或 simple heuristic。CityLearn Challenge 也把常見評估目標定得很清楚，例如 peak demand、load shifting、整體成本等，這會讓你的

proposal 顯得很「像真的研究」。不過老實說，這題比庫存控制難一些，因為它的 state / reward / 控制目標通常更複雜，有些設定還帶 multi-agent 味道。 [citylearn.net+2](#)

3. 用 DRL 學習 Job-Shop Scheduling 的派工策略

這題是我給你的**第三順位**，也是三題裡研究味最重的一題。NeurIPS 2020 的 *Learning to Dispatch for Job Shop Scheduling via Deep Reinforcement Learning* 就是非常典型的範例：它不是手工設計 dispatching rule，而是讓 DRL 自動學習高品質的 priority dispatching rule，並且有官方實作 repo。後續也有多個 job-shop RL framework 與環境可參考。這表示它非常適合寫成「傳統 rule 很依賴專家知識，但 DRL 可以直接從狀態學 policy」的論文故事。 [NeurIPS 會議錄+5](#)

但我要先提醒你，這題雖然很漂亮，**也是三題裡最難的一題**。難點不只在演算法，而是在問題表示、排程狀態建模、reward 設計、以及和 heuristic / OR baseline 的比較。如果你現在想要的是「能穩穩做完 proposal，後續也有機會做出 prototype」，那它不如庫存控制穩；但如果你想讓題目看起來更學術、更像 conference paper 路線，這題很有競爭力。 [arXiv+2](#)

直接幫你做結論

如果你要的是**最穩、最容易成功、最不需要碰 agent 概念**，選：

庫存補貨策略最佳化

如果你要的是**最有真實世界應用感、最好講 motivation、最像智慧系統論文**，選：

建築能源 / 電池排程控制

如果你要的是**最有研究味、最像正式 DRL 論文題目**，選：

Job-Shop Scheduling 派工策略學習。 [arXiv+2](#)

以你現在的程度，我的單一建議是：

先選「庫存補貨策略最佳化」。它最符合「你還不想碰 agent、但又要做出像樣 DRL proposal」這個條件。你如果點頭，我下一則就直接幫你這題寫成教授格式的 **題目 + Abstract + Introduction + Related Work + Proposed Design + Experiment Plan**。

釐清一點：我當然要碰 agent, drl 就是 s(state), a(action), r(reward) 組件組成, agent 概念在這之中非常重要, 我前面說不要碰 agent 是指 "ai agent" 如 openclaw, antigravity, claude code 這些, 不是 drl 世界中的 agent, 所以不要再說 **不需要碰 agent 概念**，請知悉

1. 以需求感知深度強化學習進行都市路口自適應交通號誌控制
2. DRL 做建築能源 / 電池排程控制

這兩個題目你覺得哪個適合套進教授 A to Z 的論文敘述流程 以及 兩者的難度差異
顯示更多顯示較少

收到。

我之後會明確區分：

- **DRL 裡的 agent**：你說的 *s / a / r* 框架核心
- **AI agent 工具鏈**：像 OpenClaw、Antigravity、Claude Code 這類系統

你這次問的兩個題目，我直接給結論：

若目標是更順地套進教授 **A-to-Z 論文敘事流程**，且更容易做出一份漂亮的 **proposal**，我會選 **1. 都市路口自適應交通號誌控制**。

若目標是題目更有能源／永續研究感、較像近年 **benchmark-driven DRL** 研究，**2. 建築能源／電池排程控制** 也很好，但整體敘事與實作都會更難。

先講總結

哪個更適合教授 **A-to-Z 流程**？

交通號誌控制比較適合。

原因不是它比較高級，而是它的故事線更直觀：

- **Attention Getter / Motivation** 很強：塞車、等待、效率、碳排，人人都懂
- **But** 很自然：固定時制、actuated control 仍然不夠 adaptive
- **Cure** 很清楚：讓 agent 根據車流狀態動態選 phase
- **Development** 很好寫：state、action、reward 都很具體
- **Experiments** 很好做：直接和 fixed-time、actuated controller 比
- **Findings** 很好呈現：平均等待時間、queue length、travel time 一看就懂

SUMO-RL 官方文件本來就是為 **Traffic Signal Control** 做的 RL 環境，支援 Gymnasium，相容 SB3 / RLlib，且可以從 **single-agent** 開始，也能自訂 state 和 reward。SUMO 本身也原生支援 **actuated traffic lights**，所以 baseline 非常自然。RESCO benchmark 進一步把單路口、多路口、真實城市啟發的場景和標準 RL 介面整理好，還明確列出常見 state 特徵如 queue length、waiting time，以及 action 是選擇不衝突的綠燈 phase。  Lucas Alegre+3

哪個比較難？

建築能源／電池排程控制比較難。

不是因為它沒有工具，而是因為它的問題結構天生更複雜。CityLearn 雖然也很成熟，甚至就是為 building energy coordination / demand response 建的 Gymnasium benchmark，但它的目標通常是**多目標**的：peak demand、ramping、load factor、net electricity consumption 等，而且常常牽涉 battery、PV、heat pump、occupant feedback、district-level coordination。這代表你不只是在設計一個 agent，而是在設計一個**多指標、多時間尺度、強 domain knowledge** 的控制問題。  CityLearn+3

兩題放進同一張比較表

面向	交通號誌控制	建築能源／電池排程控制
A-to-Z 故事好講程度	很強 ，問題直觀、畫面感強	強，但需要先交代能源系統背景
Motivation	塞車、等候、號誌效率，容易讓老師秒懂	電網負載、需求響應、削峰填谷，較專業
State 設計	車流量、queue length、waiting time、current phase，直觀	SOC、負載、PV、價格、氣候、舒適度等，較抽象
Action 設計	切換 phase / 保持 phase，很直觀	充放電、功率調節、設備控制，較多 domain 限制
Reward 設計	delay、queue、pressure 等都常見且好解釋	常是多目標 cost，權重設計較難
Baseline 好不好找	很好找 ：fixed-time、actuated	也有，但通常要講 RBC、MPC、multi-objective cost
圖表呈現	很好看，路口圖、車流圖、學習曲線都容易理解	可做，但圖表較偏 KPI / 能源曲線
單智能體起步	很自然 ：單一路口 single-agent	可以，但很多文獻與環境都偏 district / multi-agent
實作風險	中等	中高
Proposal 成功率	高	中高，但更吃表達與建模能力

為什麼交通號誌更適合教授的 A-to-Z

你前面給的教授流程，本質上是在要求一篇 proposal 要有一條非常順的「問題—缺口—方法—驗證」敘事線。交通號誌這題在這方面幾乎是範本級：

1) Motivation 特別強

城市壅塞與等待成本是人人有感的問題。你不需要花很多篇幅教育聽眾背景，老師一看就知道這題的重要性。RL traffic signal benchmark 論文也直接把這類問題放在真實交通場景與 calibrated demand 上比較，說明這

 NeurIPS Datasets and

不是玩具問題。 [Benchmarks](#)

2) Related Work 很好鋪陳

你可以非常自然地寫成：

- fixed-time
- actuated control
- adaptive control
- RL / DRL traffic signal control

SUMO 官方文件明確說明 actuated traffic lights 的運作邏輯：車流連續就延長 phase，出現足夠 gap 才切換。這讓你在 Related Work 裡能把傳統控制講得很具體，而不是空泛地說「舊方法不夠好」。 Eclipse SUMO

3) Proposed Design 最容易具體化

SUMO-RL 本來就提供 single-agent 和 multi-agent 入口，state / reward 可改，動作就是控制號誌。這讓你的 Proposed Design 可以很快落成：

- state：queue、waiting time、approaching vehicles
- action：選 phase
- reward：-delay / -queue / -pressure

這種方法章節很容易寫得像研究，不會只有概念。  Lucas Alegre+2

4) Experiments 最容易設計

你幾乎不用苦思 baseline：

- fixed-time
- actuated
- 你的 DRL agent

而且指標也成熟：delay、trip time、waiting time、queue length、pressure。這些在 benchmark 文獻裡本來就是常見項目。  Eclipse SUMO+1

為什麼建築能源／電池排程比較難

這題其實很優秀，只是比較「研究型」。

1) Motivation 沒有交通那麼秒懂

CityLearn 的出發點非常強，和 demand response、distributed energy resources、電網韌性、去碳化直接相關；官方與論文都明講它是用來標準化 building energy coordination / demand response 的 RL 研究。

 CityLearn+1

但問題是，這些詞對聽眾不是直覺的。

你在 proposal 一開始就得先解釋：

- 為什麼 peak demand 重要
- 為什麼 ramping 是問題
- 為什麼 load shifting 有價值
- battery / PV / heat pump 怎麼跟控制目標連在一起

這會吃掉你的敘事篇幅。

2) Reward 比交通複雜很多

CityLearn Challenge 2020 就是很典型的例子：它直接把目標定成 5 個等權重指標的 multi-objective cost，還用 hand-tuned RBC 當 baseline 正規化。這很適合研究，但對課堂 proposal 來說，會讓你更難講清楚「我的 agent 到底在學什麼」。  CityLearn+1

3) Domain knowledge 門檻比較高

CityLearn v2 明講它可以拿來研究 battery charge/discharge、V2G、heat pump modulation，還有 occupant feedback；同時也指出它更適合做**快速比較控制演算法**，若要嚴格審查絕對能耗，還是需要更高保真度的模型。這代表你除了 DRL，本身還得理解不少能源系統語境。  [arXiv](#)

4) 容易不小心把題目做太大

CityLearn 支援 central single-agent、independent multi-agent、coordinated multi-agent，還能自訂 district、observations、reward。這是優點，也是陷阱：題目很容易膨脹。  [arXiv+1](#)

如果以「proposal 漂亮程度」來選

我會這樣排：

第一名：交通號誌控制

因為它最符合教授那種：

- 一句話就能講清楚問題
- 一頁就能講清楚 baseline
- 一張圖就能講清楚 agent 在幹嘛

第二名：建築能源／電池排程

它不是不好，而是它更適合你已經很確定自己要往能源控制研究寫，並且願意花時間處理多目標 reward 與能源背景。

如果以「實作難度」分級

我會給你這個判斷：

- **交通號誌控制：6.5/10**
 - 單一路口 + single-agent + PPO/DQN + fixed-time/actuated baseline
 - 可控、好 debug、好做 ablation
- **建築能源／電池排程：8/10**
 - 就算只做 battery control，reward 與 KPI 還是比交通更抽象
 - 如果碰 district coordination 或 multi-agent，難度再上升

這個差距的核心不在演算法，而在 **問題建模成本**。

我的最終建議

若你現在要的是：最適合教授 A-to-Z、最容易把 proposal 說得漂亮、而且後續真的比較有機會做完，選 1。

也就是：

以需求感知深度強化學習進行都市路口自適應交通號誌控制

因為它在三件事上都比較優：

- 論文故事最好講
- baseline 最自然
- state / action / reward 最直觀

而 **建築能源／電池排程控制** 我會留給「你很確定想往能源控制寫」的情況。那題研究味更濃，但也更容易在 proposal 階段就把自己拖進多目標、長時間尺度、能源背景解釋的泥沼。 

你若要，我下一步可以直接幫你做一版**兩題並排的教授 A-to-Z proposal 骨架比較表**，再把交通號誌那題收斂成你可以直接交的題目名稱、abstract 和 experiment plan。

忽略 "實作難度" 這一點 我後續 final project 都會交給 ai agent antigravity 執行 我只負責規劃及下指令 部參與實際程式開發 我開發的是整個專案 這是我的角色

在知道以上背景下再幫我分析一次這兩個題目

了解，我重新用你的**真實角色**來分析：

你不是在比「哪個比較好自己寫 code」，而是在比

哪個題目更適合你做專案總設計、論文敘事、規格拆解、baseline 定義、驗證指標規劃，然後再交給 Antigravity 落地。

在這個前提下，判斷標準就變成 5 件事：

1. 題目能不能被你**清楚寫成一份可執行 spec**
2. baseline、KPI、ablation 能不能**一開始就定義乾淨**
3. 文獻與 benchmark 生態夠不夠成熟，方便你做 proposal 的 **Related Work 與 Experimental Plan**
4. 結果能不能**視覺化、講故事、說服老師**
5. 題目有沒有足夠空間讓你作為「專案規劃者」提出自己的研究主張，而不只是套環境跑模型

在這個標準下，兩題都成立，但**優勢方向不一樣**。 

先給結論

如果你要的是：最適合教授 A-to-Z 敘事、最好寫成一份漂亮 proposal、最好規劃成可交付專案 brief，選：

1. 以需求感知深度強化學習進行都市路口自適應交通號誌控制

如果你要的是：研究味更濃、benchmark 更像正式學術社群題目、final report 更容易帶出永續/能源/多目標控制價值，選：

2. DRL 做建築能源 / 電池排程控制

也就是說，忽略實作難度之後，交通號誌仍然比較適合 A-to-Z proposal；但建築能源的競爭力會明顯上升，不再只是「比較難所以不選」。  OpenReview+3

為什麼在你的角色下，交通號誌仍然是 A-to-Z 較強題

你現在最重要的工作，不是寫演算法，而是把題目定成一個可以被 **agent** 精準執行的研究計畫。交通號誌控制在這件事上特別強，因為它的任務邊界天然清楚：

- 環境：SUMO / SUMO-RL
- 控制對象：traffic light
- state：queue、waiting time、current phase 等
- action：phase selection / keep-switch
- reward：delay、queue、pressure
- baseline：fixed-time、actuated
- KPI：waiting time、queue length、travel time、throughput

這些元件在 SUMO-RL 文件和 SUMO 官方交通號誌機制裡都非常成熟，而且 SUMO 官方本來就支援 fixed、actuated、delay_based 等交通燈邏輯，SUMO-RL 也明寫 state / reward 易於修改、可 single-agent 也可 multi-agent。這代表你在 proposal 裡能把「研究問題」直接翻成「系統規格」。  Lucas Alegre+1

更重要的是，這題的 **research gap** 很好寫。

RESCO benchmark 那篇 NeurIPS Datasets and Benchmarks 論文直接指出，過去不少 traffic signal RL 方法在更真實的 sensing assumptions 和非 stylized intersection layouts 下不夠 robust；它提供 benchmark 的目的，就是要讓這個領域能被標準化比較。這對你來說很好用，因為你可以自然地把 proposal 的缺口寫成：

不是「我也來做一個 DRL 號誌控制」，
而是「在更 realistic demand / sensing 設定下，設計 demand-aware state/reward 是否能提升穩健性」。

這個敘事非常符合 A-to-Z 的 But → Cure → Development → Experiments。  OpenReview

再來，交通號誌的結果呈現特別適合你這種規劃角色。

因為你最後交的不只是 code，而是整個 project。交通題目很容易做出讓老師一眼看懂的成果頁：

- 路口示意圖
- 每 phase 的控制邏輯
- 學習曲線
- 平均等待時間前後比較
- queue length 熱圖
- fixed-time / actuated / DRL 的對照表

這種題目很吃「你怎麼包裝成一個完整專案」，而這正好是你的強項。  Lucas Alegre+1

為什麼在你的角色下，建築能源 / 電池排程會變得更有吸引力

如果今天你要自己寫 code，我仍會比較保守；但你現在明說後續 implementation 交給 agent，那建築能源這題的分數會上升，原因是它的**研究框架非常成熟**。

CityLearn 官方把自己定義成一個 open-source、Farama Gymnasium 的 building energy coordination / demand response RL 環境，目標就是**標準化 RL agent 的比較**。它支援 single-agent 與 multi-agent，並且明確說可用於 active energy storage 的 load shifting，以及 heat pump / electric heater 的控制。這代表它不是零散 demo，而是已經有社群共識的 benchmark 場域。[CityLearn](#)

另外，CityLearn v2 的定位其實非常漂亮。官方與論文都強調它不只是在做 battery scheduling，而是在做 **energy-flexible、resilient、occupant-centric、carbon-aware** 的 grid-interactive communities；並且提供 RL 應用範例來管理 battery charging/discharging、vehicle-to-grid control，以及 heat pump power modulation。這種題目一旦被你包成 proposal，會比交通號誌更有「近年研究題目感」。[arXiv+1](#)

再者，CityLearn Challenge 也幫你把**實驗正當性**準備好了。官方 challenge 明確定義 multi-objective cost，至少包含 peak demand 等多個等權重指標；這意味著你在 proposal 中不必自己發明一整套 KPI 體系，而是可以沿用既有 benchmark 精神，再加入你自己的研究主張。對一個負責規劃整體專案的人來說，這很有價值。

[CityLearn+1](#)

兩題真正的差別，不在「誰比較難」，而在「誰比較好被你定義成一個研究故事」

交通號誌控制

它的優勢是 **單一核心問題非常聚焦**。

你幾乎可以用一句話講完：

讓 agent 根據即時車流狀態決定號誌 phase，以降低等待與壅塞。

這意味著你在當 PM / architect 的時候，spec 很容易寫得又短又準。

你交給 Antigraity 的 prompt 也會很清楚：環境、state、action、reward、baseline、metric 一條線到底。這種題目特別適合做成教授喜歡的「從動機一路推到實驗」的 proposal。

建築能源 / 電池排程

它的優勢是 **研究外延更大、價值敘事更高階**。

但代價是你必須先替題目「縮焦」。因為 CityLearn 同時碰 demand response、battery、heat pump、occupant comfort、carbon-aware control，若你不先把題目收斂，proposal 很容易變成：

什麼都能做，所以不知道這次到底要做哪一個。

也就是說，這題不怕 agent 寫不出來，而是怕你在上游 spec 階段**沒有把研究邊界切乾淨**。[CityLearn+1](#)

如果站在「你要寫給 Antigraity 的任務書」角度看

交通號誌比較像

一份可以直接拆成工作包的專案：

- Task 1: 建立 SUMO/SUMO-RL 單路口環境
- Task 2: 實作 fixed-time / actuated baseline
- Task 3: 定義 demand-aware state
- Task 4: 訓練 PPO / DQN
- Task 5: 比較 delay、queue、travel time
- Task 6: 畫圖與整理 ablation

建築能源比較像

一份需要你先做「研究範圍治理」的專案：

- 先決定只做 battery，還是 battery + heat pump
- 先決定 single-building 還是 district
- 先決定 single-agent 還是 multi-agent
- 先決定主目標是 cost、peak、carbon、comfort 哪一個
- 再決定 reward 是否多目標加權
- 最後才進入訓練與比較

所以，對一個專案規劃者而言，交通題比較像可立即發包的題目；能源題比較像需要先做策略收斂的題目。

 Lucas Alegre+2

在你的背景下，我會這樣下最終判斷

若你的第一優先是：

讓 proposal 最像教授 A-to-Z 流程、最順、最有說服力、最容易被老師快速理解與買單

我仍然選：

1. 以需求感知深度強化學習進行都市路口自適應交通號誌控制

因為它在以下四點最強：

- 問題敘事最直觀
- 缺口最好寫
- baseline 最自然
- 專案規格最好拆解給 coding agent 執行  OpenReview+2

若你的第一優先是：

讓 final project 看起來更像近年 benchmark-driven 的正式研究題，並帶有能源、永續、城市系統控制的高階價值

那麼：

2. DRL 做建築能源 / 電池排程控制

其實會是更有學術質感的選擇。尤其在你不用自己寫 code 的前提下，它的主要門檻不再是開發，而是你能不能把題目收斂成一個乾淨的研究版本。CityLearn 的 benchmark 與 challenge 生態，確實給了它很強的學術支撐。 CityLearn+2

一句話結論

只看 A-to-Z proposal 適配度：交通號誌勝。

只看研究題目質感與 benchmark 生態：建築能源幾乎追平，甚至在某些老師眼中會更“像研究”。

所以最後不是「哪個比較難」，而是：

- 你要一個**更好講、好包裝、好拆規格**的題目 → 選交通號誌
- 你要一個**更有研究味、更像正式 benchmark 論文**的題目 → 選建築能源 / 電池排程

以你現在說的角色設定，我會給你一個更精準的建議：

如果你這次最重視 proposal 贏面，選交通號誌。

如果你最重視 final report 的研究感與題目格調，選建築能源，但一定要先幫它縮焦。

下一步最有效的是，我直接幫你做：

「這兩題在教授 A-to-Z 流程下的並排 proposal 骨架」，讓你一眼看出哪個比較適合你出手。

CityLearn 有跟 "自適應交通號誌" 相關的題目嗎 主題擦到邊都可以

貼上的 Markdown (1).md

檔案

好，以下進入 **Step 2：題目定位與研究問題收斂**。

這一步的目標，不是先把方法細節寫滿，而是先把你的題目從「好聽的方向」收斂成「可被寫成 proposal、可被驗證、可被後續實作」的研究版本。

Step 2：題目定位與研究問題收斂

一、我先替這個題目下工作定義

你的題目目前最適合被收斂成：

在單一都市化路口的模擬環境中，設計一個具需求感知能力的深度強化學習交通號誌控制器，並與 **fixed-time** 與 **actuated control** 比較其在等待時間、排隊長度與旅行時間上的表現。

這樣收斂的原因很明確：

- **交通號誌控制本來就是 RL 很自然的決策問題**：交通狀態是輸入，號誌 phase 是動作，延遲與壅塞可作為回饋。RESCO 論文也直接把這件事寫成「根據當前交通狀態輸出下一個 signal phases」的控制問題。

 [datasets-benchmarks-](#)

[proceedings.neurips.cc](#)

- **題目要先聚焦在單一路口、單智能體**，proposal 才會乾淨；SUMO-RL 官方也明確支援 single-agent 與 multiagent，且 state 與 reward 易於客製。 [lucasalegre.github.io+1](#)

二、合理假設

為了讓 proposal 可寫、可做、可驗證，我先明確列出這一版的合理假設：

1. **研究場景先以單一都市路口為主**，不直接擴成全區域多路口協同。
2. **環境以 SUMO / SUMO-RL 模擬為主**，不主張這次 proposal 直接做真實道路部署。SUMO-RL 官方就是為 SUMO 上的 traffic signal control RL 環境設計。 [lucasalegre.github.io+1](#)
3. **baseline 至少包含 fixed-time 與 actuated control**。SUMO 官方文件明確支援固定時制與 actuated traffic lights，且 actuated 是根據 gap 或 delay 動態調整。 [Eclipse SUMO](#)
4. **demand-aware 的主張，不是另做交通流預測大模型**，而是讓 agent 的 state / reward / control logic 更明確感知「當前與短時需求壓力」。這樣研究邊界會比較清楚。
5. **proposal 階段以研究問題、系統規格、實驗設計為主**，不承諾完整 deployment。

三、研究背景

在都市交通中，號誌化路口會顯著影響通勤延遲。RESCO benchmark 論文指出，都市通勤時間中約有 **12% 到 55%** 來自號誌化路口造成的停止或接近延遲，因此更好的號誌控制有潛力降低通勤時間、擁塞、排放與燃油消

 [datasets-benchmarks-](#)

耗。[proceedings.neurips.cc](#)

傳統交通號誌控制長期依賴固定時制或感應式控制。SUMO 官方文件說明，預設交通燈可用固定週期運作；而 actuated 版本則讓綠燈時長在一定範圍內變動，依據 induction loop 偵測到的 gap，或依 vehicle delay 來決定。這代表傳統方法確實已具備某種「動態性」，但其調整邏輯仍屬預先定義規則，而不是從資料互動中學得策略。 [Eclipse SUMO](#)

近年的 DRL 交通號誌研究快速成長。2024 年的 survey 指出，固定時制在面對日益複雜的都市交通需求時已顯侷限，而 DRL 已成為重要研究方向；但同時，該 survey 也強調許多 TSC 研究仍主要在理想化模擬環境中驗證，距離真實部署還有差距。RESCO benchmark 也進一步指出，過去不少方法在不同感測假設與較真實的非 stylized intersection layouts 下並不夠 robust。 [科學直接+2](#)

四、核心問題

把問題濃縮成一句話：

如何讓交通號誌控制器不只是「被動依規則延長或縮短綠燈」，而是能夠根據路口當下的需求壓力主動學出更好的 phase 決策策略？

更具體地說，這個題目的核心不是單純「用 DRL 控號誌」，而是：

在固定時制與感應式控制已存在的前提下，
需求感知（demand-aware）DRL 是否能更有效率地利用即時交通狀態，降低延遲與壅塞？

五、研究動機

這個題目的動機可以分成三層。

1. 實務動機

都市路口的交通需求高度波動，尖峰、離峰、方向性流量不均、短時突增等情況都很常見。固定時制不會即時回應這種波動；即便是 actuated control，通常也只是對「目前是否還有車」做局部延長，未必能對整體需求壓力做更全面的平衡。SUMO 官方文件中，actuated 的本質仍是基於 gap 或 delay 的規則式調整。[Eclipse SUMO](#)

2. 研究動機

RL/DRL 的優勢在於，它能把號誌控制視為序列決策問題：

智能體根據當前狀態選動作，並從回饋中逐步調整策略。RESCO 論文和 2024 survey 都把 TSC 視為典型的 DRL

[datasets-benchmarks-](#)

應用場景，並且指出 state、action、reward 的設計是這類研究的核心。[proceedings.neurips.cc+1](#)

3. 你的 proposal 動機

你的 proposal 最值得強調的，不是「我也做一個 DRL 控號誌」，而是：

我想把 DRL 的感知重心，從一般流量表徵，進一步收斂到“需求壓力”本身。

也就是說，讓 agent 更直接感知：

- 哪個方向現在排隊更嚴重
- 哪個相位的等待正在累積
- 哪些 incoming lanes 的需求壓力即將變成 bottleneck

這樣的 demand-aware framing，比單純說「用 PPO / DQN 控紅綠燈」更有研究主張。

六、為何值得做

這題值得做，不只是因為它是熱門 DRL 題，而是因為它同時滿足三個層面。

1. 問題重要

號誌化路口直接影響通勤延遲、擁塞與排放。這是典型高社會價值議題。 [datasets-benchmarks-proceedings.neurips.cc](#)

2. 研究上有缺口

RESCO benchmark 的重要訊息是：先前很多 RL 交通號誌方法，在較真實的 layout 與感測假設下不夠穩健。2024 survey 也指出，很多研究仍停留在理想化模擬。這使得你的 proposal 可以合理主張：
不是只追求更高分數，而是要讓需求表徵與控制策略更貼近實際路口的決策需要。 [OpenReview+2](#)

3. 方法與驗證環境成熟

SUMO、SUMO-RL、Gym-compatible tooling 與 RESCO benchmark 已形成相對成熟的研究生態。這代表這題不只是概念有趣，而是有標準化工具鏈支撐 proposal 後續落地。SUMO-RL 官方與 Stable-Baselines3 專案頁都明確指出其與 Gym / RL library 的相容性。 [lucasalegre.github.io+1](#)

七、與傳統 fixed-time / actuated traffic signal control 的差異

這一段是 proposal 裡一定要講清楚的。

1. Fixed-time control

固定時制的本質是：

相位順序固定、週期固定、綠燈分配預先設定。

優點是簡單、穩定、易部署；缺點是對即時需求變動不敏感。SUMO 官方文件預設就是 fixed cycle，這也正好能當成最基本 baseline。 [Eclipse SUMO](#)

2. Actuated control

感應式控制的本质是：

根據偵測到的車流 gap 或 delay，延長或縮短特定相位的綠燈時間。

它已經比 fixed-time 更動態，但仍主要依靠人工設計的規則與觸發條件。SUMO 官方文件也說明，actuated 與 delay_based 仍是在既有 phase sequence 下變動時長，而不是由學習器主動學得整體控制政策。 [Eclipse SUMO](#)

3. Demand-aware DRL

你要做的不是單純的 adaptive，而是：

讓 agent 在每個決策點，根據需求壓力表徵主動決定下一步號誌控制行為，以最大化長期回報。

白話說：

- fixed-time：先排好時刻表
- actuated：看到車就多給幾秒
- demand-aware DRL：看整體需求壓力，再學「現在應該偏向服務哪個方向、何時切換、怎樣長期更划算」

這就是你題目的本質差異。

八、研究主張 (novel angle)

我建議你的 **novel angle** 不要寫得太大，而要寫得剛好能在 proposal 站得住。

建議主張

本研究主張：在單一路口自適應交通號誌控制中，若將交通需求壓力明確納入狀態表徵與回饋設計，可望使 **DRL controller** 相較於 **fixed-time** 與 **actuated control**，更有效地平衡不同來向車流需求，進而降低整體等待時間與排隊長度。

這個主張有三個好處：

1. 不誇大

你不是說自己要發明全新 RL 演算法，而是聚焦在「demand-aware 設計」。

2. 和現有文獻對得上

2024 survey 特別指出 TSC 研究的主流差異常落在 **state representation**、**action selection**、**reward design**。你把 novel angle 放在這裡，最合理。 [科學直接](#)

3. 容易轉成規格

下一步到 Step 3 時，你就能自然展開：

- demand-aware state
- demand-aware reward
- 可能的壓力平衡控制邏輯

九、主研究問題 (Main Research Question)

Main Research Question

在單一路口都市交通場景中，需求感知深度強化學習是否能比 **fixed-time** 與 **actuated traffic signal control** 更有效地降低整體延遲與壅塞？

白話版：

如果讓號誌控制器更清楚感知需求壓力，它能不能真的比傳統方法更會控紅綠燈？

十、子研究問題 (Sub-questions)

Sub-question 1

哪些交通需求資訊最能有效表徵路口控制決策所需的 **demand pressure**？

白話版：

是 queue length 比較重要？還是 waiting time？還是每個 incoming lane 的需求分佈？

這一題對應的是 **state representation**。

Sub-question 2

需求感知應該主要體現在 **state** 設計、**reward** 設計，還是兩者同時設計，才能帶來較穩定的控制效果？

白話版：

只把需求資訊塞進觀察值就夠了，還是 **reward** 也要明確懲罰需求失衡與排隊累積？

這一題對應的是 **state-reward coupling**。

Sub-question 3

在不同交通需求強度或不均衡流量條件下，**demand-aware DRL** 相對於 **fixed-time** 與 **actuated control** 的優勢是否一致？

白話版：

需求感知是不是只在尖峰才有用？還是離峰、不均衡流量、突發壓力情境下也有價值？

這一題對應的是 **robustness across demand scenarios**。

十一、我建議你最終採用的研究版本

如果要把題目正式收斂成 proposal 可直接使用的版本，我建議你用下面這個表述：

中文版

本研究聚焦於單一路口都市交通場景，提出一種需求感知深度強化學習交通號誌控制方法。相較於固定時制與感應式號誌控制，本研究將交通需求壓力納入狀態表徵與回饋設計，探討其是否能在不同交通需求條件下更有效降低平均等待時間、排隊長度與旅行時間。

英文版

This study focuses on adaptive traffic signal control at a single urban intersection and proposes a demand-aware deep reinforcement learning approach. Compared with fixed-time and actuated control, the proposed method explicitly incorporates traffic demand pressure into state representation and reward design, and evaluates whether it can more effectively reduce average waiting time, queue length, and travel time under varying traffic demand conditions.

十二、Step 2 的最終收斂結論

你這個題目，最好的研究定位不是：

「我想用 DRL 來控交通號誌」

而是：

「我想驗證：若把需求壓力更明確地放進 DRL 的感知與優化目標中，是否能讓單一路口自適應交通號誌控制比 **fixed-time** 與 **actuated control** 更有效。」

這樣收斂後，題目就同時具備：

- 明確背景
- 清楚基準
- 合理缺口
- 可操作的研究問題
- 可延伸到 Step 3 的方法論框架

下一步就可以很自然進到 **Step 3：建立方法論框架**。

好 幫我詳細推薦跟 "網路吞吐量設計應用 DRL" 有關的 能獲得 100 分滿分的期末報告題目
請推薦我 10 個題目並作比較分析
最後推薦你心目中的第一名給我

搜尋前請先看我先前給你的 Final Project Proposal 要求 如附圖所示 我要能適合先提案再收斂成期末專題
且能更做出來的題目

media.mp4

檔案

Smart_City_Drone_Law_Enforcement.pdf

PDF

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Frank_oral_defense_0725_v7.pdf

PDF

From 0 to 1 Deep RL with SMC MDP.pdf

PDF

<https://www.youtube.com/watch?v=MqwmP4CJnSE>

附上三種參考資料(請自行閱讀附檔後自動對應以下範圍) 幫我做成同樣的研究流程

1. A to Z 論文撰寫框架: 有 2 份
2. 專題提案成果: 有 3 份
3. 教授的 chatgpt 收斂出專題提案的對話過程: 有 1 份

我的要求目前只有以下三點 其餘需求請你幫我補充成可以獲得 100 分滿分期末專題提案的程度交付物 並幫我規劃推進步驟:

1. 讓 "以深度強化學習進行單一瓶颈鏈路壅塞控制與吞吐量最佳化" 這題目能產出如 "專題提案成果" 的最終交付物給我
2. 我的所有程式碼工作都會交給 antigravity 執行工作且要用到工具鏈 **"openspec"**
3. final project 都會交給 ai agent antigravity 執行 我只負責規劃及下指令 部參與實際程式開發 我開發的是整個專案 這是我的角色

顯示更多顯示較少

收到。之後只要你跟我要「prompt」，但沒有特別指定品質，我都會預設用這個等級：**最高品質、可直接餵回執行、任務邊界完整、驗收條件明確、能接到後續階段。**

這份是 **Phase 2 / OpenSpec change 01 : project-charter** 的最佳 prompt。它的設計依據是：OpenSpec 的核心是先把規格變成 single source of truth，再進入實作；官方與相關文件也強調以 proposal / spec / design / tasks 這類 artifacts 讓人與 AI coding assistant 在寫 code 前先對齊。[GitHub+2](#)

同時，這份 prompt 會把本專題的技術邊界鎖定在 ns-3 / ns3-gym / DRL congestion control 的 Phase 0、Phase 1 定案上；ns-3 官方支援多種 TCP congestion control baseline，ns3-gym 則是為 ns-3 與 RL 整合而設計，這些都會被要求寫進 charter 作為後續 source of truth。[NSNAM+2](#)

請下一回合直接貼上以下整段：

你現在要執行的是我 DRL Final Project 的 ****Phase 2：用 OpenSpec 建立專案真相來源**** 的第一階段：

OpenSpec change 01:project-charter

最高任務目標

你要基於前面已完成的 Phase 0 與 Phase 1 定案內容，替我產出一份可以作為整個專案 ****source of truth**** 的 OpenSpec-style project charter。

這不是 proposal 草稿，也不是一般摘要，而是後續所有 change 的上游規格母文件。

它必須能讓我、Antigravity、OpenSpec、後續 PPT / GitHub / 10 分鐘影片全部對齊同一個方向。

本階段只產出文件，不寫任何程式碼，不做實作，不啟動 ns-3，不啟動 ns3-gym，不訓練模型。

一、Phase 0 / Phase 1 已凍結內容

請你完整承接以下已定案內容，不得自行改題、擴題或改回其他題目。

1. 正式題目

中文題目

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

英文題目

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

簡報首頁版題目

DRL-Based Congestion Control over a Bottleneck Link

建議 GitHub repo name

drl-bottleneck-congestion-control

2. 研究定位

本專題是：

- networking systems 題
- DRL sequential decision-making 題
- congestion control / throughput optimization 題

- ns-3 / ns3-gym / Stable-Baselines3 專題
- baseline comparison 專題
- proposal-first / spec-first 專題

本專題不是：

- IPFS 實作題
- QUIC 實作題
- Linux kernel TCP 修改題
- 真實 Internet deployment 題
- multi-agent 題
- multi-path routing 題
- 大型網路拓模題
- Pantheon full integration 題
- 自適應交通號誌控制題

3. 核心研究主張

本研究將單一瓶頸鏈路的壅塞控制建模為深度強化學習問題，讓 agent 根據網路狀態觀測值調整傳輸控制行為，並透過 throughput、RTT、packet loss 與 utility 指標，和 NewReno、CUBIC、BBR 等 TCP baseline 進行比較。

本研究不承諾 DRL 必然全面超越所有 TCP baseline。

成功標準是建立清楚的 DRL formulation、可重現的 baseline / experiment plan、以及能支撐後續實作的 OpenSpec source of truth。

4. 已定案工具鏈

後續 implementation 預設工具鏈為：

- ns-3: network simulator
- ns3-gym: ns-3 與 RL agent 的介面
- Stable-Baselines3: DRL 訓練框架
- DQN: MVP 第一版演算法，因為 action 初始設計為 discrete
- PP0: future extension, 不是 MVP 必做
- OpenSpec: 規格中樞
- Antigravity: 後續實作者

5. 已定案 baseline

必做 baseline

- NewReno
- CUBIC

強烈建議 baseline

- BBR

加分 baseline

- Vegas
- Westwood
- DCTCP
- LEDBAT

6. 已定案 metrics

必做 metrics：

- average throughput
- average RTT

- packet loss rate
- utility score
- training reward curve
- convergence behavior

加分 metrics：

- queue occupancy
- Jain's fairness index
- generalization across different bandwidth / delay settings

7. 已定案 MVP

MVP 的最低完成標準是：

1. 明確定義 single bottleneck link 場景
2. 明確定義 NewReno / CUBIC / BBR baseline plan
3. 明確定義 ns3-gym environment 的 MDP 需求
4. 明確定義 DQN MVP 的 state / action / reward
5. 明確定義 throughput / RTT / loss / utility 評估方式
6. 後續能產出 baseline vs DRL 的比較圖表
7. GitHub README / PPT / 10 分鐘影片能清楚說明整體研究設計

二、你的任務角色

你現在不是一般助理，而是：

- OpenSpec 架構師
- 研究總設計師
- 技術 PM
- 論文 proposal 規劃顧問
- AI coding agent 工作規格設計者

你的產出必須足以讓後續 Antigravity 在不偏航的情況下，依照 OpenSpec change 逐步執行。

三、OpenSpec change 01 的核心目的

change-01-project-charter 的任務是建立整個專案的 ****上游共識文件****。

它要回答：

1. 這個專題到底要做什麼？
2. 這個專題明確不做什麼？
3. 為什麼這題值得做？
4. 為什麼這題適合 DRL？
5. 為什麼要先從 single bottleneck link 開始？
6. 為什麼 baseline 要選 NewReno / CUBIC / BBR？
7. 為什麼 MVP 先選 DQN 而不是直接 PP0？
8. 成功標準是什麼？
9. 失敗或卡關時如何降階？
10. 後續 change-02、03、04 應如何銜接？

四、你必須先搜尋並引用的來源

請你在正式輸出前先做最新網路搜尋。

不可只靠記憶。

優先使用來源

請優先搜尋並引用：

1. OpenSpec 官方網站 / GitHub README / docs
2. ns-3 官方 TCP model 文件
3. ns3-gym 官方 GitHub / ns-3 app 頁面
4. Stable-Baselines3 DQN 文件
5. Stable-Baselines3 PPO 文件
6. Aurora 原始論文
7. Pantheon paper / 官方 benchmark 說明
8. 必要時補充 RFC 或正式論文

你必須用來源確認的判斷

請確認並支撐以下判斷：

- OpenSpec 適合用來建立 spec-driven source of truth
- ns-3 適合做 TCP / congestion control baseline
- ns-3 支援 NewReno、CUBIC、BBR 等 TCP congestion control
- ns3-gym 適合將 ns-3 包成 RL environment
- DQN 適合 discrete action 的第一版 MVP
- PPO 適合留作 future extension, 不是本 change 必做
- Aurora 可支撐 DRL for congestion control 的研究正當性
- Pantheon 適合作為 benchmark / reproducibility 思想來源, 但不是 MVP 必裝工具
- throughput 不能是唯一成功指標, 必須同時納入 RTT / loss / utility

五、OpenSpec change 01 必須產出的內容

請嚴格按照以下結構輸出。

OpenSpec change 01: project-charter 總結

請用 150–250 字說明：

- change 01 的目的
- 為什麼它是整個專案的 source of truth
- 它如何約束後續 change 02 / 03 / 04
- 它如何避免 Antigravity 偏航

1. Change Metadata

請輸出：

- Change ID
- Change title
- Change type
- Status
- Owner
- Implementer
- Related phase
- Related downstream changes
- Required before coding: Yes / No
- Code implementation allowed in this change: Yes / No

請明確設定：

- Change ID: 01-project-charter
- Change type: documentation / governance / scope definition

- Status: draft for review
- Owner: user / spec owner
- Implementer: Antigravity, after approval
- Code implementation allowed: No

2. Project Charter

請產出正式 project charter, 包含:

2.1 Project Name

請包含:

- 中文名稱
- 英文名稱
- Repo name

2.2 Project Mission

用 1-2 段正式寫出這個專題的使命。

2.3 Project Background

請寫出:

- 網路吞吐量為什麼重要
- congestion control 為什麼是核心
- 為什麼這跟分散式系統 / P2P / IPFS 背景有知識連續性
- 為什麼本專題不直接做 IPFS

2.4 Problem Statement

請正式定義本專題要解的問題:

- 傳統 TCP congestion control 是重要 baseline
- 但 throughput / RTT / loss trade-off 不容易只靠單一指標處理
- 本研究將問題轉成 DRL sequential decision-making formulation

2.5 Proposed Direction

請說明:

- 使用 ns-3 建立 single bottleneck link
- 使用 ns3-gym 作為 RL interface
- 使用 SB3 DQN 作為 MVP agent
- 與 NewReno / CUBIC / BBR 比較
- 使用 throughput / RTT / loss / utility 評估

2.6 Success Definition

請明確定義什麼叫成功。

成功不等於:

- DRL 全面贏過所有 baseline
- 完成真實網路部署
- 完成 IPFS 實作

成功等於:

- 研究範圍明確
- baseline plan 明確
- MDP 明確
- metrics 明確

- experiment matrix 明確
- 後續 OpenSpec change 可執行
- final project 可形成 GitHub / PPT / video 交付物

3. Scope / Non-Scope

請以非常明確的方式列出。

3.1 In Scope

至少包含：

- single bottleneck link
- ns-3 simulation
- ns3-gym RL interface
- sender-side congestion-control abstraction
- DQN MVP
- NewReno / CUBIC / BBR baseline comparison
- throughput / RTT / loss / utility evaluation
- GitHub / PPT / 10-min video support
- OpenSpec-driven workflow

3.2 Out of Scope

至少包含：

- IPFS implementation
- QUIC implementation
- kernel-level TCP
- real Internet deployment
- multi-agent RL
- multi-path routing
- large-scale network topology
- Pantheon full integration as required dependency
- PPO as MVP requirement
- production-level congestion control protocol

3.3 Strict Non-Goals

請列出不能被 Antigravity 自行擴張的事項。

語氣要強硬、明確，例如：

- Do not implement IPFS.
- Do not replace the project topic.
- Do not jump to DRL before baseline benchmark is specified.
- Do not optimize for throughput alone.

4. Research Questions and Hypotheses

請把 Phase 1 的研究問題整理成 project-charter 版。

4.1 Main Research Question

請給一個正式主研究問題。

4.2 Sub-questions

至少包含：

- state representation
- action design
- reward design
- baseline comparison
- robustness / generalization

4.3 Hypotheses

請給 2-3 個 hypothesis。
語氣必須保守，不可寫成已證明結果。

5. Technical Direction Freeze

請正式凍結技術方向。

5.1 Simulator

ns-3

說明為什麼。

5.2 RL Interface

ns3-gym

說明為什麼。

5.3 Training Framework

Stable-Baselines3

說明為什麼。

5.4 MVP Algorithm

DQN

說明為什麼。

5.5 Future Algorithm

PP0

說明何時才導入。

5.6 Baselines

NewReno / CUBIC / BBR

說明每個 baseline 的角色。

5.7 Metrics

throughput / RTT / loss / utility / reward curve / convergence behavior

說明每個 metric 的用途。

6. MVP Definition

請明確定義 MVP。

6.1 MVP Must Have

列出必備項目。

6.2 MVP Should Have

列出強烈建議但非絕對必備項目。

6.3 MVP Could Have

列出加分項目。

6.4 MVP Must Not Have

列出禁止項目。

6.5 MVP Acceptance Criteria

請列出明確驗收條件。

注意：

這裡是 charter 層級，不要寫 code，也不要寫實際命令。

7. Downstream Change Map

請定義 change-01 後面三個 change 如何接續。

7.1 change-02-baseline-benchmark

請說明：

- 目的
- 輸入
- 輸出
- 成功標準
- 依賴 change-01 的哪些內容

7.2 change-03-opengym-env

請說明：

- 目的
- 輸入
- 輸出
- 成功標準
- 依賴 change-01 的哪些內容

7.3 change-04-drl-mvp

請說明：

- 目的
- 輸入
- 輸出
- 成功標準
- 依賴 change-01 的哪些內容

8. Risk Register

請建立 charter 層級 risk register。

至少包含以下風險：

1. ns3-gym 安裝 / 版本問題
2. ns-3 baseline logging 不完整
3. BBR baseline 版本相依性
4. DQN 不收敛
5. Reward 設計導致高 throughput 但高 RTT / loss
6. Antigravity 自行擴題
7. 題目被誤解為 IPFS 實作
8. 結果沒有超越 baseline
9. 實驗圖表不足以支撐簡報
10. OpenSpec 文件與實作脫節

表格欄位請包含：

- Risk ID
- Description
- Probability
- Impact
- Prevention
- Fallback
- Owner
- Trigger condition

9. OpenSpec Artifact Blueprint

請設計 `change-01-project-charter` 應該產出的 artifact。

請不要真的建立檔案，但要提供建議結構與內容摘要。

至少包含：

```
```text
openspec/
 changes/
 01-project-charter/
 proposal.md
 design.md
 tasks.md
 specs/
 project-charter.md
 scope.md
 risk-register.md
```

對每個檔案說明：

- purpose
- required content
- reviewer should check
- acceptance criteria

## # 10. proposal.md 草案內容

請直接產出一份可以放入：

```
openspec/changes/01-project-charter/proposal.md
```

的 markdown 草案。

內容至少包含：

- Why
- What Changes
- What Does Not Change
- Impact
- Dependencies
- Acceptance Criteria

注意：  
這是 proposal.md 草案，不是正式論文 proposal。

---

## # 11. design.md 草案內容

請直接產出一份可以放入：

```
openspec/changes/01-project-charter/design.md
```

的 markdown 草案。

內容至少包含：

- Design Goal
- Project Scope Boundary
- Technical Direction
- Governance Rules
- Downstream Change Dependency
- Decision Records

注意：  
design.md 只處理設計與治理，不處理程式碼架構。

---

## # 12. tasks.md 草案內容

請直接產出一份可以放入：

```
openspec/changes/01-project-charter/tasks.md
```

的 markdown 草案。

tasks 必須是文件任務，不是實作任務。

例如：

- Confirm project title
- Confirm scope / non-scope
- Confirm baseline list
- Confirm MVP success criteria
- Confirm risk register
- Review downstream changes

每個 task 請用 checkbox 格式。

---

## # 13. specs 草案摘要

請產出 specs 下三個檔案的摘要內容，不必寫成完整檔案，但要夠後續轉成完整 spec。

### 13.1 specs/project-charter.md

請提供：

- Purpose
- Project mission
- Research target
- Success definition
- OpenSpec governance role

### 13.2 specs/scope.md

請提供：

- In scope
- Out of scope
- Strict non-goals
- Expansion rules

### 13.3 specs/risk-register.md

請提供：

- Risk categories
- Risk table summary
- Fallback principles

---

## # 14. Review Checklist

請建立 change-01 驗收檢查表。

檢查表必須讓我能回答：

- 題目是否凍結？
- scope 是否清楚？
- non-goals 是否清楚？
- baseline 是否凍結？
- metrics 是否凍結？
- MVP 是否凍結？

- 後續 change 是否能接上？
- 是否有任何內容偷偷進入實作？
- 是否有任何內容偏離 Phase 0 / Phase 1 定案？

請用 checklist 格式輸出。

---

## # 15. 給 Antigravity 的 change-01 指令草案

請幫我產出一段可以交給 Antigravity 的 change-01 指令。

這段指令必須說清楚：

- 目前只做 project charter
  - 不准寫 code
  - 不准建立 ns-3 實驗
  - 不准開始 DQN
  - 必須只產出 OpenSpec artifacts
  - 必須遵守 scope / non-goals
  - 完成後等待 spec owner 驗收
- 

## 六、輸出品質要求

### 1. 語言

- 全文使用繁體中文
- 必要英文專有名詞保留英文
- 檔名、change ID、artifact 名稱保持英文

### 2. 深度

不要只列大綱。

要寫到我可以直接複製部分內容去建立 OpenSpec 文件。

### 3. 嚴謹度

所有重要判斷要有可信來源支撐。

若是推論，請明確寫「這是基於來源的推論」。

### 4. 禁止事項



本階段禁止：

- 寫任何程式碼
- 寫 shell command
- 產生 ns-3 實作
- 產生 Python 訓練腳本
- 進入 baseline benchmark
- 進入 opengym-env
- 進入 drl-mvp
- 更改題目
- 擴張到 IPFS / QUIC / multi-agent / multi-path
- 宣稱已有實驗結果

## 七、現在開始執行

請先搜尋官方文件與正式來源，再輸出。

請直接從：

## OpenSpec change 01：project-charter 總結

開始。

這份是 **Phase 2 / OpenSpec change 01：project-charter** 的最佳 prompt。它的設計依據是：OpenSpec 的核心是先把規格變成 single source of truth，再進入實作；官方與相關文件也強調以 proposal / spec / design / tasks 這類 artifacts 讓人與 AI coding assistant 在寫 code 前先對齊。([GitHub][1])

同時，這份 prompt 會把本專題的技術邊界鎖定在 ns-3 / ns3-gym / DRL congestion control 的 Phase 0、Phase 1 定案上；ns-3 官方支援多種 TCP congestion control baseline，ns3-gym 則是為 ns-3 與 RL 整合而設計，這些都會被要求寫進 charter 作為後續 source of truth。([NSNAM][2])

請下一回合直接貼上以下整段：

```
`text
```

你現在要執行的是我 DRL Final Project 的 **Phase 2：用 OpenSpec 建立專案真相來源** 的第一階段：

```
OpenSpec change 01：project-charter
```

```
最高任務目標
```

你要基於前面已完成的 Phase 0 與 Phase 1 定案內容，替我產出一份可以作為整個專案 **source of truth** 的 OpenSpec-style project charter。

這不是 proposal 草稿，也不是一般摘要，而是後續所有 change 的上游規格母文件。

它必須能讓我、Antigravity、OpenSpec、後續 PPT / GitHub / 10 分鐘影片全部對齊同一個方向。

本階段只產出文件，不寫任何程式碼，不做實作，不啟動 ns-3，不啟動 ns3-gym，不訓練模型。

---

## # 一、Phase 0 / Phase 1 已凍結內容

請你完整承接以下已定案內容，不得自行改題、擴題或改回其他題目。

### ## 1. 正式題目

#### ### 中文題目

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

#### ### 英文題目

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

#### ### 簡報首頁版題目

DRL-Based Congestion Control over a Bottleneck Link

#### ### 建議 GitHub repo name

drl-bottleneck-congestion-control

---

### ## 2. 研究定位

本專題是：

- networking systems 題
- DRL sequential decision-making 題
- congestion control / throughput optimization 題
- ns-3 / ns3-gym / Stable-Baselines3 專題
- baseline comparison 專題
- proposal-first / spec-first 專題

本專題不是：

- IPFS 實作題
- QUIC 實作題
- Linux kernel TCP 修改題
- 真實 Internet deployment 題
- multi-agent 題
- multi-path routing 題
- 大型網路拓模題
- Pantheon full integration 題
- 自適應交通號誌控制題

---

### ## 3. 核心研究主張

本研究將單一瓶頸鏈路的壅塞控制建模為深度強化學習問題，讓 agent 根據網路狀態觀測值調整傳輸控制行為，並透過 throughput、RTT、packet loss 與 utility 指標，和 NewReno、CUBIC、BBR 等 TCP baseline 進行比較。

本研究不承諾 DRL 必然全面超越所有 TCP baseline。

成功標準是建立清楚的 DRL formulation、可重現的 baseline / experiment plan、以及能支撐後續實作的 OpenSpec source of truth。

---

### ## 4. 已定案工具鏈

後續 implementation 預設工具鏈為：

- ns-3: network simulator
- ns3-gym: ns-3 與 RL agent 的介面
- Stable-Baselines3: DRL 訓練框架
- DQN: MVP 第一版演算法, 因為 action 初始設計為 discrete
- PPO: future extension, 不是 MVP 必做
- OpenSpec: 規格中樞
- Antigravity: 後續實作者

---

## ## 5. 已定案 baseline

### ### 必做 baseline

- NewReno
- CUBIC

### ### 強烈建議 baseline

- BBR

### ### 加分 baseline

- Vegas
- Westwood
- DCTCP
- LEDBAT

---

## ## 6. 已定案 metrics

必做 metrics:

- average throughput
- average RTT
- packet loss rate
- utility score
- training reward curve
- convergence behavior

加分 metrics:

- queue occupancy
- Jain's fairness index
- generalization across different bandwidth / delay settings

---

## ## 7. 已定案 MVP

MVP 的最低完成標準是：

1. 明確定義 single bottleneck link 場景
2. 明確定義 NewReno / CUBIC / BBR baseline plan
3. 明確定義 ns3-gym environment 的 MDP 需求
4. 明確定義 DQN MVP 的 state / action / reward
5. 明確定義 throughput / RTT / loss / utility 評估方式
6. 後續能產出 baseline vs DRL 的比較圖表
7. GitHub README / PPT / 10 分鐘影片能清楚說明整體研究設計

---

## # 二、你的任務角色

你現在不是一般助理, 而是：

- OpenSpec 架構師

- 研究總設計師
- 技術 PM
- 論文 proposal 規劃顧問
- AI coding agent 工作規格設計者

你的產出必須足以讓後續 Antigravity 在不偏航的情況下，依照 OpenSpec change 逐步執行。

---

### # 三、OpenSpec change 01 的核心目的

change-01-project-charter 的任務是建立整個專案的 **\*\*上游共識文件\*\***。

它要回答：

1. 這個專題到底要做什麼？
2. 這個專題明確不做什麼？
3. 為什麼這題值得做？
4. 為什麼這題適合 DRL？
5. 為什麼要先從 single bottleneck link 開始？
6. 為什麼 baseline 要選 NewReno / CUBIC / BBR？
7. 為什麼 MVP 先選 DQN 而不是直接 PPO？
8. 成功標準是什麼？
9. 失敗或卡關時如何降階？
10. 後續 change-02、03、04 應如何銜接？

---

### # 四、你必須先搜尋並引用的來源

請你在正式輸出前先用最新網路搜尋。  
不可只靠記憶。

## 優先使用來源

請優先搜尋並引用：

1. OpenSpec 官方網站 / GitHub README / docs
2. ns-3 官方 TCP model 文件
3. ns3-gym 官方 GitHub / ns-3 app 頁面
4. Stable-Baselines3 DQN 文件
5. Stable-Baselines3 PPO 文件
6. Aurora 原始論文
7. Pantheon paper / 官方 benchmark 說明
8. 必要時補充 RFC 或正式論文

## 你必須用來源確認的判斷

請確認並支撐以下判斷：

- OpenSpec 適合用來建立 spec-driven source of truth
- ns-3 適合做 TCP / congestion control baseline
- ns-3 支援 NewReno、CUBIC、BBR 等 TCP congestion control
- ns3-gym 適合將 ns-3 包成 RL environment
- DQN 適合 discrete action 的第一版 MVP
- PPO 適合留作 future extension，不是本 change 必做
- Aurora 可支撐 DRL for congestion control 的研究正當性
- Pantheon 適合作為 benchmark / reproducibility 思想來源，但不是 MVP 必裝工具
- throughput 不能是唯一成功指標，必須同時納入 RTT / loss / utility

---

### # 五、OpenSpec change 01 必須產出的內容

請嚴格按照以下結構輸出。

---

## ## # OpenSpec change 01:project-charter 總結

請用 150–250 字說明：

- change 01 的目的
- 為什麼它是整個專案的 source of truth
- 它如何約束後續 change 02 / 03 / 04
- 它如何避免 Antigravity 偏航

---

## ## # 1. Change Metadata

請輸出：

- Change ID
- Change title
- Change type
- Status
- Owner
- Implementer
- Related phase
- Related downstream changes
- Required before coding:Yes / No
- Code implementation allowed in this change:Yes / No

請明確設定：

- Change ID:01-project-charter
- Change type:documentation / governance / scope definition
- Status:draft for review
- Owner:user / spec owner
- Implementer:Antigravity, after approval
- Code implementation allowed:No

---

## ## # 2. Project Charter

請產出正式 project charter, 包含：

### ### 2.1 Project Name

請包含：

- 中文名稱
- 英文名稱
- Repo name

### ### 2.2 Project Mission

用 1–2 段正式寫出這個專題的使命。

### ### 2.3 Project Background

請寫出：

- 網路吞吐量為什麼重要
- congestion control 為什麼是核心
- 為什麼這跟分散式系統 / P2P / IPFS 背景有知識連續性
- 為什麼本專題不直接做 IPFS

### ### 2.4 Problem Statement

請正式定義本專題要解的問題：

- 傳統 TCP congestion control 是重要 baseline
- 但 throughput / RTT / loss trade-off 不容易只靠單一指標處理
- 本研究將問題轉成 DRL sequential decision-making formulation

### ### 2.5 Proposed Direction

請說明：

- 使用 ns-3 建立 single bottleneck link
- 使用 ns3-gym 作為 RL interface
- 使用 SB3 DQN 作為 MVP agent
- 與 NewReno / CUBIC / BBR 比較
- 使用 throughput / RTT / loss / utility 評估

### ### 2.6 Success Definition

請明確定義什麼叫成功。

成功不等於：

- DRL 全面贏過所有 baseline
- 完成真實網路部署
- 完成 IPFS 實作

成功等於：

- 研究範圍明確
- baseline plan 明確
- MDP 明確
- metrics 明確
- experiment matrix 明確
- 後續 OpenSpec change 可執行
- final project 可形成 GitHub / PPT / video 交付物

---

## ## # 3. Scope / Non-Scope

請以非常明確的方式列出。

### ### 3.1 In Scope

至少包含：

- single bottleneck link
- ns-3 simulation
- ns3-gym RL interface
- sender-side congestion-control abstraction
- DQN MVP
- NewReno / CUBIC / BBR baseline comparison
- throughput / RTT / loss / utility evaluation
- GitHub / PPT / 10-min video support
- OpenSpec-driven workflow

### ### 3.2 Out of Scope

至少包含：

- IPFS implementation
- QUIC implementation
- kernel-level TCP
- real Internet deployment
- multi-agent RL
- multi-path routing

- large-scale network topology
- Pantheon full integration as required dependency
- PPO as MVP requirement
- production-level congestion control protocol

### ### 3.3 Strict Non-Goals

請列出不能被 Antigravity 自行擴張的事項。

語氣要強硬、明確，例如：

- Do not implement IPFS.
- Do not replace the project topic.
- Do not jump to DRL before baseline benchmark is specified.
- Do not optimize for throughput alone.

---

## ## # 4. Research Questions and Hypotheses

請把 Phase 1 的研究問題整理成 project-charter 版。

### ### 4.1 Main Research Question

請給一個正式主研究問題。

### ### 4.2 Sub-questions

至少包含：

- state representation
- action design
- reward design
- baseline comparison
- robustness / generalization

### ### 4.3 Hypotheses

請給 2-3 個 hypothesis。

語氣必須保守，不可寫成已證明結果。

---

## ## # 5. Technical Direction Freeze

請正式凍結技術方向。

### ### 5.1 Simulator

ns-3

說明為什麼。

### ### 5.2 RL Interface

ns3-gym

說明為什麼。

### ### 5.3 Training Framework

Stable-Baselines3

說明為什麼。

### ### 5.4 MVP Algorithm

DQN

說明為什麼。

### ### 5.5 Future Algorithm

## PPO

說明何時才導入。

### ### 5.6 Baselines

NewReno / CUBIC / BBR

說明每個 baseline 的角色。

### ### 5.7 Metrics

throughput / RTT / loss / utility / reward curve / convergence behavior

說明每個 metric 的用途。

---

## ## # 6. MVP Definition

請明確定義 MVP。

### ### 6.1 MVP Must Have

列出必備項目。

### ### 6.2 MVP Should Have

列出強烈建議但非絕對必備項目。

### ### 6.3 MVP Could Have

列出加分項目。

### ### 6.4 MVP Must Not Have

列出禁止項目。

### ### 6.5 MVP Acceptance Criteria

請列出明確驗收條件。

注意：

這裡是 charter 層級，不要寫 code，也不要寫實際命令。

---

## ## # 7. Downstream Change Map

請定義 change-01 後面三個 change 如何接續。

### ### 7.1 change-02-baseline-benchmark

請說明：

- 目的
- 輸入
- 輸出
- 成功標準
- 依賴 change-01 的哪些內容

### ### 7.2 change-03-opengym-env

請說明：

- 目的
- 輸入
- 輸出



- 成功標準
- 依賴 change-01 的哪些內容

### ### 7.3 change-04-drl-mvp

請說明：

- 目的
- 輸入
- 輸出
- 成功標準
- 依賴 change-01 的哪些內容

---

### ## # 8. Risk Register

請建立 charter 層級 risk register。

至少包含以下風險：

1. ns3-gym 安裝 / 版本問題
2. ns-3 baseline logging 不完整
3. BBR baseline 版本相依性
4. DQN 不收斂
5. Reward 設計導致高 throughput 但高 RTT / loss
6. Antigravity 自行擴題
7. 題目被誤解為 IPFS 實作
8. 結果沒有超越 baseline
9. 實驗圖表不足以支撐簡報
10. OpenSpec 文件與實作脫節

表格欄位請包含：

- Risk ID
- Description
- Probability
- Impact
- Prevention
- Fallback
- Owner
- Trigger condition

---

### ## # 9. OpenSpec Artifact Blueprint

請設計 change-01-project-charter 應該產出的 artifact。

請不要真的建立檔案，但要提供建議結構與內容摘要。

至少包含：

text

openspec/

changes/

01-project-charter/

proposal.md

design.md

tasks.md

specs/

project-charter.md

scope.md  
risk-register.md

對每個檔案說明：

- \* purpose
- \* required content
- \* reviewer should check
- \* acceptance criteria

---

## # 10. proposal.md 草案內容

請直接產出一份可以放入：

text  
openspec/changes/01-project-charter/proposal.md

的 markdown 草案。

內容至少包含：

- \* Why
- \* What Changes
- \* What Does Not Change
- \* Impact
- \* Dependencies
- \* Acceptance Criteria

注意：

這是 proposal.md 草案，不是正式論文 proposal。

---

## # 11. design.md 草案內容

請直接產出一份可以放入：

text  
openspec/changes/01-project-charter/design.md

的 markdown 草案。

內容至少包含：

- \* Design Goal
- \* Project Scope Boundary
- \* Technical Direction
- \* Governance Rules
- \* Downstream Change Dependency
- \* Decision Records

注意：

design.md 只處理設計與治理，不處理程式碼架構。

---

## # 12. tasks.md 草案內容

請直接產出一份可以放入：

text  
openspec/changes/01-project-charter/tasks.md

的 markdown 草案。

tasks 必須是文件任務，不是實作任務。

例如：

- \* Confirm project title
- \* Confirm scope / non-scope
- \* Confirm baseline list
- \* Confirm MVP success criteria
- \* Confirm risk register
- \* Review downstream changes

每個 task 請用 checkbox 格式。

---

## ## # 13. specs 草案摘要

請產出 specs 下三個檔案的摘要內容，不必寫成完整檔案，但要夠後續轉成完整 spec。

### ### 13.1 specs/project-charter.md

請提供：

- \* Purpose
- \* Project mission
- \* Research target
- \* Success definition
- \* OpenSpec governance role

### ### 13.2 specs/scope.md

請提供：

- \* In scope
- \* Out of scope
- \* Strict non-goals
- \* Expansion rules

### ### 13.3 specs/risk-register.md

請提供：

- \* Risk categories
- \* Risk table summary
- \* Fallback principles

---

## ## # 14. Review Checklist

請建立 change-01 驗收檢查表。

檢查表必須讓我能回答：

- \* 題目是否凍結？
- \* scope 是否清楚？
- \* non-goals 是否清楚？
- \* baseline 是否凍結？
- \* metrics 是否凍結？
- \* MVP 是否凍結？
- \* 後續 change 是否能接上？
- \* 是否有任何內容偷偷進入實作？
- \* 是否有任何內容偏離 Phase 0 / Phase 1 定案？

請用 checklist 格式輸出。

---

## # 15. 給 Antigravity 的 change-01 指令草案

請幫我產出一段可以交給 Antigravity 的 change-01 指令。

這段指令必須說清楚：

- \* 目前只做 project charter
- \* 不准寫 code
- \* 不准建立 ns-3 實驗
- \* 不准開始 DQN
- \* 必須只產出 OpenSpec artifacts
- \* 必須遵守 scope / non-goals
- \* 完成後等待 spec owner 驗收

---

# 六、輸出品質要求

## 1. 語言

- \* 全文使用繁體中文
- \* 必要英文專有名詞保留英文
- \* 檔名、change ID、artifact 名稱保持英文

## 2. 深度

不要只列大綱。

要寫到我可以直接複製部分內容去建立 OpenSpec 文件。

## 3. 嚴謹度

所有重要判斷要有可信來源支撐。

若是推論，請明確寫「這是基於來源的推論」。

## 4. 禁止事項

本階段禁止：

- \* 寫任何程式碼
- \* 寫 shell command
- \* 產生 ns-3 實作
- \* 產生 Python 訓練腳本
- \* 進入 baseline benchmark
- \* 進入 opengym-env
- \* 進入 drl-mvp
- \* 更改題目
- \* 擴張到 IPFS / QUIC / multi-agent / multi-path
- \* 宣稱已有實驗結果

---

# 七、現在開始執行

請先搜尋官方文件與正式來源，再輸出。

請直接從：

# OpenSpec change 01: project-charter 總結

開始。

::contentReference[oaicite:2]{index=2}

[1]: [https://github.com/Fission-AI/OpenSpec?utm\\_source=chatgpt.com](https://github.com/Fission-AI/OpenSpec?utm_source=chatgpt.com) "Fission-AI/OpenSpec: Spec-driven development (SDD) ..."  
 [2]: [https://www.nsnam.org/docs/models/html/tcp.html?utm\\_source=chatgpt.com](https://www.nsnam.org/docs/models/html/tcp.html?utm_source=chatgpt.com) "16.6. TCP models in ns-3"

顯示更多顯示較少

## OpenSpec change 01 : project-charter 總結

**change-01-project-charter** 的目的，是把本專題的題目、研究邊界、成功標準、工具鏈、baseline、風險與後續 change 串接關係，正式寫成整個專案的 **source of truth**。OpenSpec 官方強調在寫 code 前，讓人類與 AI 先對齊 specs；每個 change 也可用資料夾集中放置 proposal、design、tasks、specs，避免規格散落在聊天紀錄或臨時筆記中。 [GitHub+1](#)

因此，change 01 不允許寫 code，也不允許啟動 ns-3 / ns3-gym / DQN。它只負責建立治理文件，約束後續 **change-02-baseline-benchmark**、**change-03-opengym-env**、**change-04-drl-mvp** 的方向，避免 Antigravity 自行擴題、跳過 baseline、或把專題誤改成 IPFS / QUIC / multi-agent 題。

## 1. Change Metadata

欄位	內容
Change ID	<b>01-project-charter</b>
Change title	Project Charter for DRL-Based Congestion Control over a Bottleneck Link
Change type	documentation / governance / scope definition
Status	draft for review
Owner	user / spec owner
Implementer	Antigravity, after approval
Related phase	Phase 2：用 OpenSpec 建立專案真相來源
Related downstream changes	<b>02-ns3-baseline-benchmark, 03-opengym-env, 04-drl-mvp</b>
Required before coding	Yes
Code implementation allowed in this change	No

### 治理判斷：

此 change 是所有後續 change 的前置條件。只要 change 01 尚未被 spec owner 驗收，Antigravity 不得建立 ns-3 實驗、不准串 ns3-gym、不准寫 DQN、不准建立任何實作檔案。

## 2. Project Charter

### 2.1 Project Name

類型	名稱
中文名稱	以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化
英文名稱	Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link
簡報首頁版	DRL-Based Congestion Control over a Bottleneck Link
Repo name	<b>drl-bottleneck-congestion-control</b>

## 2.2 Project Mission

本專題的使命，是將單一瓶頸鏈路上的壅塞控制問題，形式化為一個可被深度強化學習處理的序列決策問題。專題將以 **throughput**、**RTT**、**packet loss**、**utility** 作為核心評估指標，規劃一個可重現、可比較、可交付的 DRL congestion-control prototype 路線。

本專題的重點不是宣稱 DRL 必然全面優於傳統 TCP，而是建立一套清楚的研究設計：先用 ns-3 規劃 single bottleneck link 與 TCP baseline，再透過 ns3-gym 將網路環境包成 RL environment，最後以 DQN MVP 作為第一階段 DRL agent。ns-3 官方 TCP model 支援多種 TCP congestion control 演算法，包含 CUBIC、NewReno、BBR 等，適合作為 baseline planning 的基礎。  NSNAM

## 2.3 Project Background

網路吞吐量是分散式系統、內容傳輸、P2P 網路與區塊鏈網路協定中的基礎效能問題。對 IPFS 這類內容定址與 P2P 傳輸系統而言，真正的資料可用性與使用體驗，不只取決於內容是否可能被找到，也取決於底層網路傳輸是否能在瓶頸、延遲與丟包環境下維持合理效能。

壅塞控制是影響 throughput、RTT 與 packet loss 的核心機制。傳統 TCP congestion control 已非常成熟，但它們通常以固定規則或啟發式控制邏輯回應網路狀態。本專題的研究動機，是將壅塞控制改寫為 DRL agent 的 sequential decision-making task，讓 agent 依據觀測到的網路狀態做控制決策。Aurora 論文已指出，將 congestion control cast 為 RL 可以訓練 deep network policy 以捕捉資料流與網路條件中的複雜模式，同時也

 Proceedings of  
Machine Learning

提醒真實部署仍面臨 fairness、safety、generalization 等挑戰。  Research

本專題與使用者的 IPFS / 區塊鏈網路協定畢論方向具有知識連續性，但本專題**不直接實作 IPFS**。這是刻意的 scope control：先把底層壅塞控制與吞吐量最佳化問題做小、做清楚、做成可交付成果，再把 IPFS 作為 future extension 或研究動機，不讓 final project 被 P2P overlay、DHT、Bitswap、實網節點整合拖垮。

## 2.4 Problem Statement

本專題要解的問題是：

如何在單一瓶頸鏈路中，將壅塞控制建模為深度強化學習問題，讓 agent 根據網路狀態調整傳輸控制行為，並與 NewReno、CUBIC、BBR 等 TCP baseline 比較 throughput、RTT、loss 與 utility 的 trade-off？

傳統 TCP congestion control 是本題的重要 baseline，而不是被否定的對象。NewReno、CUBIC、BBR 代表不同設計邏輯的壅塞控制方法；其中 ns-3 官方文件列出 native ns-3 TCP model 支援 CUBIC、NewReno、BBR 等演算法，因此它們適合作為後續 baseline benchmark 的規格依據。  [NSNAM](#)

問題的核心不只是「提高 throughput」。如果只最佳化 throughput，可能導致 queue 累積、RTT 增加、packet loss 上升，甚至形成 bufferbloat。因此，本專題將 throughput、RTT、loss 與 utility 一起納入評估，並在 reward 設計中明確避免 throughput-only 的錯誤方向。這是基於 Aurora 與 congestion-control

 [Proceedings of Machine Learning Research](#)

benchmark 研究對真實採用、可比較性與多指標權衡的提醒所做的設計推論。 [Learning Research+1](#)

## 2.5 Proposed Direction

本專題的 proposed direction 正式凍結如下：

### 1. 使用 ns-3 建立 single bottleneck link 場景

ns-3 是 Internet systems 的 discrete-event network simulator，主要用於 research and education；其 TCP model 支援多種 congestion control 演算法，適合做 baseline planning。  [NSNAM+1](#)

### 2. 使用 ns3-gym 作為 RL interface

ns3-gym 的定位是整合 OpenAI Gym 與 ns-3，以支援 networking research 中的 RL workflow。

 [GitHub+1](#)

### 3. 使用 Stable-Baselines3 作為訓練框架

Stable-Baselines3 是 PyTorch 上可靠的 RL algorithm implementations；其 DQN 文件明確標示 action space 為 Discrete，適合 MVP 的離散 action 設計。  [GitHub+1](#)

### 4. 使用 DQN 作為 MVP agent

這是基於來源的推論：因為 MVP action 預設為 decrease / keep / increase 這種 discrete action，而 SB3

 [stable-](#)

DQN 對應 Discrete action space，因此 DQN 比 PPO 更適合第一版 MVP。 [baselines3.readthedocs.io+1](#)

### 5. PPO 作為 future extension

PPO 支援更一般的 policy optimization，SB3 文件也說明 PPO 使用 clipping 避免 policy update 過大；但

 [stable-](#)

它不是 change 01 或 MVP 必做項目。 [baselines3.readthedocs.io+1](#)

### 6. 與 NewReno / CUBIC / BBR 比較

這三者構成 baseline 核心，支撐後續 baseline benchmark change。

### 7. 使用 throughput / RTT / loss / utility 評估

不以單一 throughput 作為成功標準。

## 2.6 Success Definition

成功不等於

- DRL 全面贏過所有 baseline
- 完成真實網路部署
- 完成 IPFS 實作

- 完成 QUIC 協定實作
- 寫出 production-level congestion control protocol
- 完成 multi-agent 或 multi-path 系統

## 成功等於

- 題目、scope、non-goals 明確凍結
- baseline plan 明確
- MDP formulation 明確
- metrics 明確
- experiment matrix 明確
- 後續 OpenSpec changes 可執行
- Antigravity 能依規格逐步工作且不偏航
- 最終能支撐 GitHub / PPT / 10 分鐘影片交付物
- 若 DRL 未超越 BBR / CUBIC，也能以 trade-off analysis、limitation、risk control 作為合理 final finding

## 3. Scope / Non-Scope

### 3.1 In Scope

本專題 scope 正式包含：

- single bottleneck link
- ns-3 simulation
- ns3-gym RL interface
- sender-side congestion-control abstraction
- DQN MVP
- NewReno / CUBIC / BBR baseline comparison
- throughput / RTT / loss / utility evaluation
- GitHub / PPT / 10-min video support
- OpenSpec-driven workflow
- baseline-first implementation strategy
- proposal-first / spec-first 專案治理

### 3.2 Out of Scope

本專題 scope 正式排除：



- IPFS implementation
- QUIC implementation
- kernel-level TCP
- real Internet deployment
- multi-agent RL
- multi-path routing
- large-scale network topology
- Pantheon full integration as required dependency
- PPO as MVP requirement
- production-level congestion control protocol
- full Aurora reproduction
- blockchain network simulation
- libp2p / Bitswap / DHT modification

## 3.3 Strict Non-Goals

以下事項不得由 Antigravity 自行擴張：

- **Do not implement IPFS.**
- **Do not implement QUIC.**
- **Do not modify Linux kernel TCP.**
- **Do not replace the project topic.**
- **Do not jump to DRL before baseline benchmark is specified.**
- **Do not optimize for throughput alone.**
- **Do not make Pantheon a required dependency.**
- **Do not introduce multi-agent RL.**
- **Do not introduce multi-path routing.**
- **Do not create large-scale topology.**
- **Do not claim experimental results before experiments exist.**
- **Do not treat PPO as an MVP requirement.**

---

## 4. Research Questions and Hypotheses

### 4.1 Main Research Question

**How can congestion control over a single bottleneck link be formulated as a deep reinforcement learning problem to balance throughput, RTT, and packet loss compared with traditional TCP**

**baselines?**

中文：

如何將單一瓶頸鏈路壅塞控制形式化為深度強化學習問題，並與傳統 TCP baseline 比較其在 throughput、RTT 與 packet loss 之間的權衡能力？

## 4.2 Sub-questions

### 1. State representation

哪些網路觀測值最適合描述壅塞控制 agent 的 state，例如 recent throughput、RTT、loss、queue / congestion indicator、previous action？

### 2. Action design

MVP 階段是否能用離散 action，例如 decrease / keep / increase sending rate 或 cwnd-like control，形成可訓練的第一版 agent？

### 3. Reward design

如何設計 reward，避免 agent 只追求 throughput，而忽略 RTT 與 packet loss？

### 4. Baseline comparison

DRL agent 與 NewReno、CUBIC、BBR 相比，能呈現哪些 throughput-delay-loss trade-off？

### 5. Robustness / generalization

在不同 bandwidth、delay、queue 或 cross traffic 條件下，agent 的策略是否仍能維持穩定表現？

## 4.3 Hypotheses

### H1

將 throughput、RTT、packet loss 同時納入 reward，可能比 throughput-only reward 更能避免高延遲或高丟包策略。

### H2

DQN 搭配離散 action 可以作為第一版 DRL congestion-control prototype 的最低風險路線，但不保證全面超越 CUBIC 或 BBR。

### H3

在變動 bandwidth 或存在 cross traffic 的場景中，DRL agent 可能展現可分析的 adaptive behavior；但此假設需由後續實驗驗證，change 01 不宣稱結果。

## 5. Technical Direction Freeze

### 5.1 Simulator：ns-3

正式凍結：ns-3。

理由：ns-3 是用於 Internet systems 的 discrete-event network simulator，主要面向 research and educational use；其 TCP model 支援多種 congestion control 演算法，適合作為 baseline 實驗規劃基礎。

 NSNAM+1

## 5.2 RL Interface：ns3-gym

正式凍結：ns3-gym。

理由：ns3-gym 的目的就是將 OpenAI Gym 與 ns-3 整合，以支援 reinforcement learning in networking research。這是本題從 ns-3 simulation 接到 Python RL agent 的主要橋接層。  [GitHub+1](#)

## 5.3 Training Framework：Stable-Baselines3

正式凍結：Stable-Baselines3。

理由：SB3 是 PyTorch 上可靠的 RL algorithm implementations，並提供 DQN、PPO 等演算法文件與一致化 API，適合作為後續 DRL MVP 的訓練框架。  [GitHub](#)

## 5.4 MVP Algorithm：DQN

正式凍結：DQN。

理由：MVP action 設計為 discrete action，例如 decrease / keep / increase。SB3 DQN 文件明確將 action space 標示為 Discrete，因此 DQN 是第一版最小可行 DRL agent 的合理選擇。  [stable-baselines3.readthedocs.io](#)

## 5.5 Future Algorithm：PPO

正式凍結：PPO 為 future extension，不是 MVP 必做。

PPO 支援更一般的 policy optimization，SB3 文件說明 PPO 透過 clipping 避免 policy update 過大；但它應在 DQN MVP、baseline logging、ns3-gym smoke test 成功後再考慮。  [stable-baselines3.readthedocs.io](#)

## 5.6 Baselines：NewReno / CUBIC / BBR

Baseline	角色
NewReno	經典 loss-based TCP baseline
CUBIC	現代常見 TCP congestion control baseline；ns-3 文件列為支援演算法且指出 CUBIC 為 default
BBR	以 bottleneck bandwidth / RTT 思維形成不同比較視角

ns-3 官方 TCP 文件列出 NewReno、CUBIC、BBR 等支援演算法，因此三者適合作為本專題 baseline freeze。

 NSNAM

## 5.7 Metrics

Metric	用途
throughput	衡量成功資料傳輸效率
RTT	衡量延遲與 queue 累積情況
packet loss	衡量壅塞嚴重程度
utility	綜合 throughput、RTT、loss 的整體分數
reward curve	觀察 DRL 訓練過程
convergence behavior	評估 agent 是否學到穩定策略

## 6. MVP Definition

### 6.1 MVP Must Have

MVP 必須包含：

- 明確定義 single bottleneck link 場景
- 明確定義 baseline plan：NewReno、CUBIC；BBR 強烈建議
- 明確定義 ns3-gym environment 的 MDP 需求
- 明確定義 DQN MVP 的 state / action / reward
- 明確定義 throughput / RTT / loss / utility 評估方式
- 明確定義 experiment matrix 的最低版本
- 明確產生 GitHub / PPT / 10-min video 可用的研究敘事
- 明確產出後續 change-02、change-03、change-04 的銜接規格

### 6.2 MVP Should Have

MVP 強烈建議包含：

- BBR baseline
- utility score formula
- reward normalization 設計說明
- variable bandwidth scenario
- cross traffic scenario
- risk fallback plan
- baseline vs DRL 圖表規格

## 6.3 MVP Could Have

MVP 加分項目：

- PPO comparison
- queue occupancy
- Jain's fairness index
- multi-flow shared bottleneck
- reward ablation
- generalization across bandwidth / delay settings

## 6.4 MVP Must Not Have

MVP 禁止包含：

- IPFS implementation
- QUIC implementation
- Linux kernel TCP modification
- production deployment
- multi-agent RL
- multi-path routing
- large-scale topology
- Pantheon full integration as required dependency
- PPO as required algorithm
- direct claim that DRL outperforms all baselines

## 6.5 MVP Acceptance Criteria

MVP charter 層級驗收條件：

- ☐ 題目與 repo name 已凍結。
- ☐ In scope / out of scope / strict non-goals 已明確。
- ☐ Baselines 已凍結為 NewReno、CUBIC，BBR 強烈建議。
- ☐ Metrics 已凍結為 throughput、RTT、loss、utility、reward curve、convergence behavior。
- ☐ MVP algorithm 已凍結為 DQN。
- ☐ PPO 已明確標示為 future extension。
- ☐ 成功標準不以 DRL 全面勝過 baseline 為唯一標準。
- ☐ 後續 change-02 / 03 / 04 可從本 charter 接續。
- ☐ 本 change 未包含任何程式碼、命令或實作內容。

## 7. Downstream Change Map

### 7.1 change-02-baseline-benchmark

#### 目的

建立 ns-3 single bottleneck link baseline benchmark 規格。

#### 輸入

- change-01 的 project scope
- baseline list : NewReno、CUBIC、BBR
- metrics : throughput、RTT、loss、utility
- non-goals : 不做 DRL、不做 IPFS、不做 QUIC

#### 輸出

- baseline benchmark specification
- topology specification
- logging requirements
- metric definition
- baseline acceptance criteria

#### 成功標準

- baseline 場景清楚
- 至少 NewReno / CUBIC 可作為第一批 benchmark
- BBR 作為強烈建議或加分
- 指標輸出需求明確
- 不進入 DRL 訓練

#### 依賴 change-01 的內容

- 題目
- scope
- non-goals
- baseline freeze
- metrics freeze

### 7.2 change-03-opengym-env

## 目的

定義 ns3-gym environment 的 MDP interface 規格。

## 輸入

- change-01 的 MDP 方向
- state / action / reward freeze
- ns-3 topology boundary
- DQN discrete action requirement

## 輸出

- observation space specification
- action space specification
- reward function specification
- episode / reset / step conceptual design
- random agent smoke test acceptance criteria

## 成功標準

- state / action / reward 可由 Antigravity 實作
- environment 不擴題到 IPFS / QUIC / multi-agent
- reward 不只最佳化 throughput
- action 保持 discrete MVP

## 依賴 change-01 的內容

- technical direction freeze
- DQN MVP decision
- reward philosophy
- scope / strict non-goals

## 7.3 change-04-drl-mvp

### 目的

定義 DQN MVP agent 的訓練與評估規格。

### 輸入

- change-03 的 environment spec
- change-02 的 baseline metrics

- DQN as MVP algorithm
- evaluation philosophy

## 輸出

- DQN MVP specification
- training / evaluation artifact requirements
- comparison table requirements
- reward curve / metric chart requirements

## 成功標準

- DQN agent 能被明確驗收
- 比較對象與 metrics 一致
- 不宣稱必然超越 baseline
- 產出可支撐 PPT / README / video 的圖表規格

## 依賴 change-01 的內容

- MVP definition
- baseline freeze
- metric freeze
- success definition
- risk register

# 8. Risk Register

Risk ID	Description	Probability	Impact	Prevention	Fallback	Owner	Trigger condition
R1	ns3-gym 安裝 / 版本問題	High	High	change-03 前固定 OS、ns-3、Python、ns3-gym 版本	改用 simplified Python Gym bottleneck environment 作 fallback	Antigravity	ns3-gym example 無法完成 smoke test
R2	ns-3 baseline logging 不完整	Medium	High	change-02 先定義 throughput / RTT / loss logging requirements	先保留 throughput + RTT, loss 作 should-have	Antigravity	baseline 圖表缺少核心 metric
R3	BBR baseline 版本相依性	Medium	Medium	BBR 列為 strongly recommended, 不是 MVP hard requirement	MVP 先 NewReno + CUBIC, BBR 延後	Antigravity	BBR 在指定 ns-3 版本中不可用或不穩



Risk ID	Description	Probability	Impact	Prevention	Fallback	Owner	Trigger condition
R4	DQN 不收斂	Medium	High	使用 minimal state、discrete action、清楚 reward	改成 DQN behavior analysis，不宣稱超越	Antigravity	reward curve 無穩定趨勢
R5	Reward 導致高 throughput 但高 RTT / loss	High	High	reward 必須含 RTT / loss penalty	做 throughput-only vs trade-off reward ablation	Spec owner + Antigravity	throughput 上升但 RTT / loss 異常
R6	Antigravity 自行擴題	Medium	High	OpenSpec change-by-change ; strict non-goals 寫入 charter	停止實作，回到 change review	Spec owner	出現 IPFS / QUIC / multi-agent / multi-path 實作
R7	題目被誤解為 IPFS 實作	Medium	Medium	摘要與 charter 明確寫 IPFS only as motivation	PPT / README 移除主標題中的 IPFS	Spec owner	評審或文件將主題理解成 IPFS
R8	結果沒有超越 baseline	High	Medium	成功標準改為 trade-off analysis 與 reproducible workflow	final report 強調 limitation 與 scenario-level finding	Spec owner	DQN 指標落後於 CUBIC / BBR
R9	實驗圖表不足以支撐簡報	Medium	Medium	change-05 提前定義 required figures	保留 baseline-only + DRL prototype architecture 作報告	Antigravity	PPT 缺 throughput / RTT / loss 圖
R10	OpenSpec 文件與實作脫節	Medium	High	每個 change 先驗收 artifacts 再實作	暫停實作，更新 spec 後再繼續	Spec owner	實作產物不符合 approved change

## 9. OpenSpec Artifact Blueprint

建議 change-01 artifact 結構：

```

openspec/
 changes/
 01-project-charter/
 proposal.md
 design.md
 tasks.md
 specs/
 project-charter.md
 scope.md
 risk-register.md

```

## proposal.md

項目	內容
purpose	說明為什麼需要建立 project charter
required content	Why, What Changes, What Does Not Change, Impact, Dependencies, Acceptance Criteria
reviewer should check	是否明確凍結題目、scope、non-goals、baseline、metrics
acceptance criteria	不含 code、不含實作命令、可作為後續 change 上游依據

## design.md

項目	內容
purpose	定義專案治理與設計決策
required content	Design Goal, Scope Boundary, Technical Direction, Governance Rules, Downstream Dependency, Decision Records
reviewer should check	是否防止 Antigravity 偏航、是否維持 Phase 0 / Phase 1 定案
acceptance criteria	所有 downstream changes 都能從 design.md 找到依據

## tasks.md

項目	內容
purpose	定義 change-01 的文件任務
required content	checkbox tasks
reviewer should check	tasks 是否全為文件任務，未進入實作
acceptance criteria	所有文件任務可逐項驗收

## specs/project-charter.md

項目	內容
purpose	存放正式 project charter
required content	mission、background、problem statement、proposed direction、success definition
reviewer should check	是否足以成為 source of truth
acceptance criteria	能支撐 README / proposal / downstream changes

## specs/scope.md

項目	內容
purpose	凍結 scope / non-scope
required content	in scope、out of scope、strict non-goals、expansion rules
reviewer should check	是否清楚禁止 IPFS / QUIC / multi-agent 等擴題
acceptance criteria	Antigravity 無法模糊解讀專案邊界

## specs/risk-register.md

項目	內容
purpose	管理專案風險與降階策略
required content	risk table、fallback principles、trigger conditions
reviewer should check	是否包含 ns3-gym、DQN、reward、baseline、scope drift 等核心風險
acceptance criteria	每個高風險都有 fallback

## 10. proposal.md 草案內容

### Markdown

#### # Change 01: Project Charter

##### ## Why

This project requires a source-of-truth document before any implementation begins. The final project has been scoped as a Deep Reinforcement Learning study for congestion control and throughput optimization over a single bottleneck link.

Without a project charter, the implementation agent may prematurely expand the project into IPFS, QUIC, kernel-level TCP, multi-agent reinforcement learning, or large-scale network simulation. This change establishes the project title, mission, research scope, non-goals, baseline plan, metrics, MVP definition, and downstream OpenSpec change structure.

##### ## What Changes

This change introduces the project charter for:

- Project title and identity
- Research mission
- Problem statement
- Proposed technical direction
- In-scope items
- Out-of-scope items
- Strict non-goals
- Research questions and hypotheses
- Baseline and metric freeze
- MVP success definition
- Downstream change dependency map
- Risk register

##### ## What Does Not Change

This change does not implement any code.

This change does not:

- Create ns-3 experiments

- Create ns3-gym environments
- Train DQN or PPO agents
- Implement IPFS
- Implement QUIC
- Modify kernel-level TCP
- Add multi-agent reinforcement learning
- Add multi-path routing
- Run experiments
- Generate experimental results

## ## Impact

This charter becomes the upstream governance document for:

- `02-ns3-baseline-benchmark`
- `03-opengym-env`
- `04-drl-mvp`

All downstream changes must follow the scope, non-goals, MVP definition, and success criteria established here.

## ## Dependencies

This change depends on the Phase 0 and Phase 1 decisions:

- Final topic: Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link
- Toolchain direction: ns-3, ns3-gym, Stable-Baselines3
- MVP algorithm: DQN
- Future extension: PPO
- Baselines: NewReno, CUBIC, BBR
- Metrics: throughput, RTT, packet loss, utility, reward curve, convergence behavior

## ## Acceptance Criteria

- [ ] Project title is frozen.
- [ ] Repository name is defined.
- [ ] Project mission is defined.
- [ ] In-scope and out-of-scope items are clearly separated.
- [ ] Strict non-goals are explicit and enforceable.
- [ ] Baselines are frozen.
- [ ] Metrics are frozen.
- [ ] MVP success definition is clear.
- [ ] Downstream changes are mapped.
- [ ] Risk register is included.
- [ ] No implementation code is included.
- [ ] No shell commands are included.
- [ ] No experimental results are claimed.

# 11. design.md 草案内容

## Markdown

### # Design: Project Charter Governance

#### ## Design Goal

The goal of this design is to establish a governance-level project charter for a DRL final project on congestion control over a single bottleneck link.

The charter must ensure that the spec owner, Antigravity, OpenSpec artifacts, GitHub repository, presentation slides, and final video all follow the same project direction.

#### ## Project Scope Boundary

The project is scoped as a networking systems and deep reinforcement learning project.

The project focuses on:

- Single bottleneck link congestion control
- ns-3 simulation
- ns3-gym reinforcement learning interface

- Stable-Baselines3 DQN MVP
- Baseline comparison with NewReno, CUBIC, and preferably BBR
- Evaluation using throughput, RTT, packet loss, and utility

The project does not include:

- IPFS implementation
- QUIC implementation
- Kernel-level TCP modification
- Real Internet deployment
- Multi-agent reinforcement learning
- Multi-path routing
- Large-scale topology
- Production-grade congestion control protocol

## ## Technical Direction

The technical direction is frozen as follows:

- Simulator: ns-3
- RL interface: ns3-gym
- Training framework: Stable-Baselines3
- MVP algorithm: DQN
- Future algorithm: PPO
- Baselines: NewReno, CUBIC, BBR
- Metrics: throughput, RTT, packet loss, utility, reward curve, convergence behavior

## ## Governance Rules

1. No implementation may begin before this project charter is approved.
2. Antigravity must not write code during this change.
3. Antigravity must not create ns-3 experiments during this change.
4. Antigravity must not create DQN or PPO scripts during this change.
5. Antigravity must not expand the project into IPFS, QUIC, multi-agent RL, or multi-path routing.
6. Every downstream change must reference this charter.
7. If a downstream task conflicts with this charter, the task must stop and return to the spec owner.

## ## Downstream Change Dependency

### ### 02-ns3-baseline-benchmark

Depends on:

- Baseline list
- Metrics
- Single bottleneck scope
- Non-goals

### ### 03-opengym-env

Depends on:

- MDP direction
- DQN discrete action requirement
- State / action / reward boundary
- ns3-gym decision

### ### 04-drl-mvp

Depends on:

- DQN as MVP algorithm
- Baseline comparison plan
- Metrics
- Evaluation philosophy

## ## Decision Records

### ### DR-01: Use single bottleneck link as the first scenario

Reason: It minimizes topology complexity while preserving the core congestion-control problem.

### ### DR-02: Use ns-3 as simulator

Reason: ns-3 supports TCP congestion-control baseline experiments and is suitable for research simulation.

### DR-03: Use ns3-gym as RL interface

Reason: It bridges ns-3 simulation and reinforcement learning workflows.

### DR-04: Use DQN for MVP

Reason: The MVP action space is discrete, and DQN is suitable for discrete action tasks.

### DR-05: Keep PPO as future extension

Reason: PPO is more flexible but should be introduced only after the DQN MVP and baseline benchmark are stable.

### DR-06: Do not implement IPFS in this final project

Reason: IPFS is used as long-term motivation and knowledge continuity, not as the implementation target for this course project.

## 12. tasks.md 草案内容

### Markdown

# Tasks: Change 01 Project Charter

## Documentation Tasks

- [ ] Confirm final Chinese project title.
- [ ] Confirm final English project title.
- [ ] Confirm GitHub repository name.
- [ ] Confirm project mission.
- [ ] Confirm project background.
- [ ] Confirm problem statement.
- [ ] Confirm proposed direction.
- [ ] Confirm success definition.
- [ ] Confirm in-scope items.
- [ ] Confirm out-of-scope items.
- [ ] Confirm strict non-goals.
- [ ] Confirm main research question.
- [ ] Confirm sub-questions.
- [ ] Confirm hypotheses.
- [ ] Confirm simulator decision: ns-3.
- [ ] Confirm RL interface decision: ns3-gym.
- [ ] Confirm training framework decision: Stable-Baselines3.
- [ ] Confirm MVP algorithm decision: DQN.
- [ ] Confirm future algorithm decision: PPO.
- [ ] Confirm baseline list: NewReno, CUBIC, BBR.
- [ ] Confirm metrics: throughput, RTT, packet loss, utility, reward curve, convergence behavior.
- [ ] Confirm MVP must-have items.
- [ ] Confirm MVP should-have items.
- [ ] Confirm MVP could-have items.
- [ ] Confirm MVP must-not-have items.
- [ ] Confirm MVP acceptance criteria.
- [ ] Confirm downstream change map.
- [ ] Confirm risk register.
- [ ] Confirm OpenSpec artifact blueprint.
- [ ] Review all documents for accidental implementation details.
- [ ] Review all documents for scope drift.
- [ ] Submit change 01 to spec owner for review.

## 13. specs 草案摘要

### 13.1 specs/project-charter.md

## Purpose

建立本專題的上游 source of truth，確保所有 proposal、GitHub、PPT、影片與 Antigravity implementation 都對齊同一題目與範圍。

## Project mission

將單一瓶頸鏈路壅塞控制建模為 DRL sequential decision-making problem，規劃可重現的 ns-3 / ns3-gym / SB3 workflow，並與傳統 TCP baseline 比較 throughput、RTT、loss 與 utility。

## Research target

- single bottleneck congestion control
- sender-side control abstraction
- DQN MVP
- baseline comparison with NewReno / CUBIC / BBR

## Success definition

成功是建立可執行規格與後續 implementation roadmap，不是保證 DRL 贏過所有 baseline。

## OpenSpec governance role

OpenSpec 是所有 change 的規格中樞。後續實作必須由已驗收 change 驅動，不得直接由聊天內容或 Antigravity 自由發揮。

## 13.2 specs/scope.md

### In scope

- single bottleneck link
- ns-3 simulation
- ns3-gym RL interface
- DQN MVP
- NewReno / CUBIC / BBR baseline
- throughput / RTT / loss / utility evaluation
- OpenSpec-driven workflow

### Out of scope

- IPFS implementation
- QUIC implementation
- kernel-level TCP
- real Internet deployment

- multi-agent RL
- multi-path routing
- large-scale topology
- production-level protocol

## Strict non-goals

- 不得跳過 baseline 直接做 DRL。
- 不得只最佳化 throughput。
- 不得把 IPFS 寫成主體。
- 不得把 PPO 當 MVP 必做。
- 不得把 Pantheon 當必裝依賴。

## Expansion rules

任何擴展只能在 MVP 完成後，由 spec owner 另開 change 審核。

## 13.3 specs/risk-register.md

### Risk categories

- Toolchain risk
- Baseline risk
- RL training risk
- Reward design risk
- Scope drift risk
- Reporting risk
- Governance risk

### Risk table summary

至少包含 R1–R10，包括 ns3-gym 安裝、baseline logging、BBR 版本、DQN 不收斂、reward 學歪、Antigravity 擴題、題目被誤解成 IPFS、結果未超越 baseline、圖表不足、OpenSpec 與實作脫節。

### Fallback principles

- baseline 先行
- DQN 失敗時保留 trade-off analysis
- BBR 失敗時降階為 NewReno + CUBIC
- ns3-gym 失敗時啟動 simplified Gym fallback
- scope drift 發生時停止實作並回到 spec review



## 14. Review Checklist

### change-01 驗收檢查表

- ☐ 題目是否凍結？
- ☐ 英文題目是否凍結？
- ☐ repo name 是否凍結？
- ☐ project mission 是否清楚？
- ☐ problem statement 是否清楚？
- ☐ proposed direction 是否清楚？
- ☐ success definition 是否不過度承諾？
- ☐ scope 是否清楚？
- ☐ non-scope 是否清楚？
- ☐ strict non-goals 是否足夠強硬？
- ☐ baseline 是否凍結？
- ☐ metrics 是否凍結？
- ☐ MVP 是否凍結？
- ☐ DQN 是否被定義為 MVP？
- ☐ PPO 是否被定義為 future extension？
- ☐ IPFS 是否只作為背景動機，而不是實作主體？
- ☐ QUIC 是否被排除？
- ☐ kernel-level TCP 是否被排除？
- ☐ multi-agent / multi-path 是否被排除？
- ☐ change-02 是否能接上？
- ☐ change-03 是否能接上？
- ☐ change-04 是否能接上？
- ☐ risk register 是否包含必要風險？
- ☐ 每個高風險是否都有 fallback？
- ☐ 是否有任何內容偷偷進入實作？
- ☐ 是否有任何 shell command？
- ☐ 是否有任何 code？
- ☐ 是否有任何內容偏離 Phase 0 / Phase 1 定案？
- ☐ 是否可作為 Antigravity 的上游指令依據？

## 15. 給 Antigravity 的 change-01 指令草案

You are working on Phase 2 / OpenSpec change 01: project-charter.

Current task:

Create only the OpenSpec project-charter artifacts for the DRL final project.

Project:

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

Repository:

drl-bottleneck-congestion-control

You must only produce documentation artifacts for:

- openspec/changes/01-project-charter/proposal.md
- openspec/changes/01-project-charter/design.md
- openspec/changes/01-project-charter/tasks.md
- openspec/changes/01-project-charter/specs/project-charter.md
- openspec/changes/01-project-charter/specs/scope.md
- openspec/changes/01-project-charter/specs/risk-register.md

Strict rules:

- Do not write implementation code.
- Do not create ns-3 experiments.
- Do not create ns3-gym environment code.
- Do not create DQN or PPO training scripts.
- Do not run baseline benchmarks.
- Do not implement IPFS.
- Do not implement QUIC.
- Do not modify Linux kernel TCP.
- Do not introduce multi-agent RL.
- Do not introduce multi-path routing.
- Do not change the project topic.
- Do not optimize for throughput alone.
- Do not claim experimental results.

The purpose of this change is to create the source-of-truth project charter. The charter must freeze:

- project title
- mission
- scope
- non-goals
- strict non-goals
- technical direction
- MVP definition
- baselines
- metrics
- downstream change map
- risk register

After completing the documentation artifacts, stop and wait for spec owner review.

## change-01 結論

**OpenSpec change 01 : project-charter** 應該被視為本專題真正的第一份治理文件，而不是一般摘要。它的驗收重點不是「有沒有開始做實驗」，而是「是否已經把後續所有實作可走與不可走的路全部鎖清楚」。本 change 完成並驗收後，才能進入 **change-02-baseline-benchmark**；在那之前，不應啟動 ns-3 baseline、不應串 ns3-gym、不應訓練 DQN。

OpenSpec change 01 : project-charter 階段驗收沒問題

請用一樣的生成標準（最高）產出「change-02-baseline-benchmark」的最佳 prompt

收到。以下是 **Phase 2 / OpenSpec change 02 : baseline-benchmark** 的最高品質最佳 prompt。

這一階段的重點不是跑 ns-3，也不是開始產生 baseline 圖，而是先把 **baseline benchmark** 的規格真相來源寫清楚，讓後續 Phase 3「Baseline 先行」可以照規格執行。ns-3 官方 TCP model 支援多種 TCP congestion control 演算法，包含 NewReno、CUBIC、BBR 等；Flow Monitor 官方文件也說明它可用於量測網路協定效能；ns3-gym 則是把 ns-3 與 RL 研究介面整合的框架。這些會被要求寫入 change-02 的 benchmark 規格基礎。<sup>NSNAM+2</sup>

同時，OpenSpec 的工作方式是以 change folder 管理 proposal、specs、design、tasks，並在 implementation 前先完成規格對齊；所以這份 prompt 會強制產出 **baseline benchmark** 的 **proposal.md / design.md / tasks.md / specs**，但不允許執行任何程式。<sup>GitHub+1</sup>

請下一回合直接貼上以下整段：

你現在要執行的是我 DRL Final Project 的 **\*\*Phase 2：用 OpenSpec 建立專案真相來源\*\*** 的第二階段：

# OpenSpec change 02:baseline-benchmark

## 最高任務目標

你要基於已驗收完成的：

- change-01-project-charter
- Phase 0 定案
- Phase 1 proposal 主體

替我產出一份可以作為後續 **\*\*Phase 3:Baseline 先行\*\*** 的 OpenSpec-style baseline benchmark 規格文件。

這不是實作任務，不是 ns-3 coding，不是跑 baseline，不是產圖。

這是 baseline benchmark 的 **\*\*source of truth\*\***。

本 change 的唯一目標是把後續 baseline 實作要做什麼、怎麼量測、怎麼驗收、哪些是必做、哪些是加分、哪些不能碰，全部寫成可交給 Antigravity 執行的規格。

---

# 一、已驗收前置內容

## 1. 已完成 change

`01-project-charter` 已驗收通過。

該 change 已凍結：

- 題目
- scope
- non-goals
- baseline list
- metrics
- MVP definition
- risk register
- downstream change map

本 change 必須完全繼承，不得自行改題、擴題或跳到 DRL。

---

## 2. 正式題目

### 中文題目

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

### 英文題目  
Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

### GitHub repo name  
drl-bottleneck-congestion-control

---

## 3. 本 change 的定位

本 change 是：

- baseline benchmark 規格階段
- ns-3 baseline 實驗設計階段
- metrics / logging / scenario / acceptance criteria 凍結階段
- change-03-opengym-env 與 change-04-drl-mvp 的前置依據

本 change 不是：

- ns-3 實作階段
- Python 訓練階段
- ns3-gym environment 建立階段
- DQN 訓練階段
- PPO 擴充階段
- IPFS / QUIC / kernel TCP 實作階段

---

# 二、change-02 的核心目的

change-02-baseline-benchmark 的任務是建立 \*\*傳統 TCP baseline benchmark 的規格\*\*。

它要回答：

1. baseline benchmark 要測什麼？
2. 為什麼 baseline 必須先於 DRL？
3. single bottleneck link 場景要如何定義？
4. NewReno / CUBIC / BBR 各自扮演什麼 baseline 角色？
5. throughput / RTT / loss / utility 要怎麼定義？
6. baseline 實驗至少要有哪些 scenario？
7. baseline 結果要輸出哪些 artifacts？
8. 後續 change-03 / change-04 如何使用這些 baseline？
9. 若 BBR 或 logging 有風險，怎麼降階？
10. Antigravity 後續 Phase 3 執行時，如何避免偏離 benchmark 規格？

---

# 三、本階段嚴格禁止事項

本 change 嚴格禁止：

- 不寫任何 C++ code
- 不寫任何 Python code
- 不寫 shell command
- 不建立 ns-3 topology 實作
- 不執行 ns-3
- 不產生實驗數據
- 不產生圖表
- 不啟動 ns3-gym
- 不定義 RL agent implementation
- 不訓練 DQN
- 不接 PPO

- 不接 IPFS
- 不接 QUIC
- 不修改 Linux kernel TCP
- 不做 multi-agent
- 不做 multi-path routing
- 不做大型 topology
- 不把 Pantheon 設為必裝依賴
- 不宣稱已有 baseline 實驗結果

你只能產出 OpenSpec 文件草案與規格內容。

---

#### # 四、你必須先搜尋並引用的來源

請在正式輸出前先做最新網路搜尋，不可只靠記憶。

##### ## 優先使用來源

請優先搜尋並引用：

1. OpenSpec 官方網站 / GitHub README / docs
2. ns-3 官方 TCP model 文件
3. ns-3 Flow Monitor 官方文件
4. ns-3 tracing / statistics / measurement 相關官方文件，如有需要
5. ns3-gym 官方 GitHub / ns-3 app 頁面
6. Aurora 原始論文
7. Pantheon paper / 官方 benchmark 說明
8. 必要時補充正式論文或 RFC

##### ## 你必須用來源確認的判斷

請確認並支撐以下判斷：

- OpenSpec 適合用來建立 change-based benchmark 規格
- ns-3 適合做 TCP / congestion control baseline
- ns-3 支援 NewReno、CUBIC、BBR 等 TCP congestion control
- Flow Monitor 或 ns-3 內建工具可支援 throughput / delay / loss 類指標量測
- baseline-first 是合理順序，因為後續 DRL 必須有比較基準
- Pantheon 適合作為 benchmark / reproducibility 思想來源，但不是 MVP 必裝依賴
- throughput 不能是唯一 baseline metric，必須同時規格化 RTT / loss / utility
- baseline benchmark 的結果將作為 change-04-drl-mvp 的比較基準

---

#### # 五、change-02 必須產出的內容

請嚴格按照以下章節輸出。

---

##### ## # OpenSpec change 02:baseline-benchmark 總結

請用 150–250 字說明：

- change 02 的目的
- 它如何承接 change 01
- 它為什麼必須在 DRL 前完成
- 它如何約束 change 03 / change 04
- 它如何避免 Antigravity 直接跳到 DQN

---

##### ## # 1. Change Metadata

請輸出：

- Change ID
- Change title
- Change type
- Status
- Owner
- Implementer
- Related phase
- Depends on
- Related downstream changes
- Required before implementation: Yes / No
- Code implementation allowed in this change: Yes / No

請明確設定：

- Change ID: 02-baseline-benchmark
- Change type: benchmark specification / experiment planning / metrics definition
- Status: draft for review
- Owner: user / spec owner
- Implementer: Antigravity, after approval
- Depends on: 01-project-charter
- Related downstream changes: 03-opengym-env, 04-drl-mvp
- Code implementation allowed: No

---

## ## # 2. Baseline Benchmark Charter

請產出正式 baseline benchmark charter。

### ### 2.1 Benchmark Mission

請說明：

- baseline benchmark 的使命
- 為什麼 baseline 必須先行
- 為什麼不能直接跳到 DRL
- baseline 對後續 DQN 評估的作用

### ### 2.2 Benchmark Scope

請列出 baseline benchmark 要涵蓋：

- single bottleneck link
- sender / receiver
- TCP traffic flow
- NewReno
- CUBIC
- BBR
- throughput / RTT / loss / utility
- planned scenarios
- logs / tables / figures 的規格

### ### 2.3 Benchmark Non-Scope

請列出 baseline benchmark 不做：

- 不做 DRL
- 不做 ns3-gym
- 不做 DQN / PPO
- 不做 IPFS
- 不做 QUIC
- 不做 kernel TCP
- 不做 multi-agent
- 不做 multi-path

- 不做 large-scale topology
- 不做 Pantheon full integration

### ### 2.4 Benchmark Success Definition

成功不等於：

- baseline 已經跑出最好結果
- baseline 圖已經產生
- DRL 已經比較完成

成功等於：

- baseline methods 明確
- topology requirements 明確
- logging requirements 明確
- metrics definitions 明確
- scenario matrix 明確
- fallback 明確
- downstream changes 可依此使用 benchmark 規格

---

## ## # 3. Baseline Methods Specification

請正式定義 baseline methods。

### ### 3.1 NewReno

請說明：

- 為什麼選它
- 它在本專題中的 baseline 角色
- 它和 DQN 的比較意義

### ### 3.2 CUBIC

請說明：

- 為什麼選它
- 它在本專題中的 baseline 角色
- 若 ns-3 預設為 CUBIC，這對 benchmark 有什麼意義

### ### 3.3 BBR

請說明：

- 為什麼選它
- 它在本專題中的 baseline 角色
- 為什麼它是 strongly recommended 而不是 hard MVP requirement
- 如果 BBR 有版本相依性或跑不起來，如何降階

### ### 3.4 Optional Baselines

請列出加分但不進 MVP hard requirement 的 baseline：

- Vegas
- Westwood
- DCTCP
- LEDBAT

請說明為什麼不把它們列入必做。

---

## ## # 4. Topology Specification

請定義 single bottleneck link 的 benchmark topology。

### ### 4.1 Topology Concept

請用文字描述：

- sender
- receiver
- bottleneck link
- access links
- queue / buffer
- optional cross traffic

### ### 4.2 Minimal Topology

請定義最小拓撲：

- one sender
- one receiver
- one bottleneck link
- one long-lived TCP flow

### ### 4.3 Enhanced Topology

請定義加分拓撲：

- cross traffic
- variable bandwidth
- high-delay path
- multiple flows, 但只能作為 future extension, 不是 MVP

### ### 4.4 Topology Diagram Description

請用文字描述一張後續應畫出的圖：

Sender → Access Link → Bottleneck Queue / Link → Access Link → Receiver

請不要畫圖，只描述圖的構成與標籤。

---

## ## # 5. Metrics and Logging Specification

這是本 change 最重要的部分之一。

請定義所有 baseline benchmark metrics。

### ### 5.1 Average Throughput

請定義：

- metric meaning
- why it matters
- expected unit
- data needed
- possible calculation concept
- logging requirement

### ### 5.2 Average RTT / Delay

請定義：

- metric meaning
- why it matters
- expected unit
- data needed
- logging requirement
- 和 bufferbloat / queueing delay 的關係

### ### 5.3 Packet Loss Rate



請定義：

- metric meaning
- why it matters
- expected unit
- data needed
- logging requirement

### ### 5.4 Utility Score

請定義：

- 為什麼需要 utility
- 為什麼不能只看 throughput
- 建議的概念形式

例如：

$utility = normalized\_throughput - penalty\_delay - penalty\_loss$

請提供：

- 符號注釋
- 白話說明
- 注意此階段不固定最終權重，只固定 utility philosophy

### ### 5.5 Output Artifacts

請定義後續 Phase 3 至少要輸出的 artifacts：

- raw logs
- summary CSV
- per-scenario summary table
- baseline comparison table
- throughput plot
- RTT plot
- packet loss plot
- utility comparison plot
- experiment metadata

注意：

這裡只定義 artifacts，不產生 artifacts。

---

## ## # 6. Scenario Matrix Specification

請設計 baseline benchmark scenario matrix。

至少包含 4 組情境：

1. Stable low-delay bottleneck
2. Stable high-delay bottleneck
3. Variable bandwidth bottleneck
4. Bottleneck with cross traffic

每組情境請提供：

- Scenario ID
- Purpose
- Bandwidth profile
- Delay profile
- Queue / buffer setting
- Traffic pattern
- Required baselines
- Required metrics
- Expected observation
- Risk level
- Whether MVP-required

請注意：

- 不要填過度細的實作參數
- 可以用 reasonable placeholder
- 但要足夠讓 Antigravity 後續實作時不偏航

---

## ## # 7. Benchmark Evaluation Philosophy

請明確定義本專題 baseline benchmark 的評估哲學。

必須包含：

- baseline 不是敵人，而是比較基準
- 不以單一 throughput 最大化為唯一成功標準
- delay / RTT / loss 是同等重要的分析維度
- DRL 後續若輸給 BBR / CUBIC 也不是失敗，只要 trade-off analysis 完整
- baseline 的作用是提供後續 DQN 的 reference point
- baseline 實驗應可重現、可比較、可視覺化
- Pantheon 是 benchmark 思想來源，但不是本 change 的必裝工具

---

## ## # 8. Downstream Dependency

請定義 change-02 如何餵給後續 change。

### ### 8.1 對 change-03-opengym-env 的輸入

請說明 baseline benchmark 如何幫助定義：

- observation candidates
- metric logging
- reward components
- scenario reuse
- episode / evaluation setup

### ### 8.2 對 change-04-drl-mvp 的輸入

請說明 baseline benchmark 如何幫助定義：

- DQN comparison target
- evaluation metrics
- reward curve interpretation
- success / failure analysis
- final project figures

---

## ## # 9. Risk Register for Baseline Benchmark

請建立 change-02 專屬風險清單。

至少包含：

1. ns-3 TCP algorithm name / version mismatch
2. BBR availability issue
3. FlowMonitor / logging 不足以取得 RTT 或 loss
4. throughput 計算口徑不一致
5. scenario 設計太複雜
6. cross traffic 讓 MVP 失控
7. baseline 結果異常但無法解釋
8. Antigravity 直接跳到 DRL
9. baseline 圖表不足以支撐 PPT
10. Pantheon 被誤當必裝工具

表格欄位請包含：

- Risk ID
- Description
- Probability
- Impact
- Prevention
- Fallback
- Owner
- Trigger condition

---

## ## # 10. OpenSpec Artifact Blueprint

請設計 change-02-baseline-benchmark 應產出的 artifact。

請不要真的建立檔案，但要提供建議結構與內容摘要。

至少包含：

```
```text
openspec/
  changes/
    02-baseline-benchmark/
      proposal.md
      design.md
      tasks.md
      specs/
        baseline-methods.md
        topology.md
        metrics-logging.md
        scenario-matrix.md
        benchmark-risk-register.md
```

對每個檔案說明：

- purpose
- required content
- reviewer should check
- acceptance criteria

11. proposal.md 草案內容

請直接產出一份可以放入：

```
openspec/changes/02-baseline-benchmark/proposal.md
```

的 markdown 草案。

內容至少包含：

- Why
- What Changes
- What Does Not Change
- Impact

- Dependencies
- Acceptance Criteria

注意：

這是 OpenSpec change proposal，不是論文 proposal。

12. design.md 草案內容

請直接產出一份可以放入：

```
openspec/changes/02-baseline-benchmark/design.md
```

的 markdown 草案。

內容至少包含：

- Design Goal
- Benchmark Boundary
- Baseline Methods
- Topology Direction
- Metrics and Logging Direction
- Scenario Direction
- Downstream Usage
- Decision Records

注意：

design.md 只處理 benchmark 設計，不處理實作程式。

13. tasks.md 草案內容

請直接產出一份可以放入：

```
openspec/changes/02-baseline-benchmark/tasks.md
```

的 markdown 草案。

tasks 必須是規格任務，不是實作任務。

例如：

- Confirm baseline methods
- Confirm topology spec
- Confirm metrics definitions
- Confirm scenario matrix

- Confirm output artifacts
- Review no implementation included

每個 task 請用 checkbox 格式。

14. specs 草案摘要

請產出 specs 下五個檔案的摘要內容，不必寫成完整檔案，但要夠後續轉成完整 spec。

14.1 specs/baseline-methods.md

請提供：

- Purpose
- Required baselines
- Strongly recommended baselines
- Optional baselines
- Exclusion rules

14.2 specs/topology.md

請提供：

- Purpose
- Minimal topology
- Enhanced topology
- Topology assumptions
- Non-goals

14.3 specs/metrics-logging.md

請提供：

- Purpose
- Metric definitions
- Required logs
- Required output artifacts
- Ambiguity rules

14.4 specs/scenario-matrix.md

請提供：

- Purpose

- Required scenarios
- Scenario table summary
- MVP-required vs optional

14.5 specs/benchmark-risk-register.md

請提供：

- Purpose
- Risk categories
- Risk summary
- Fallback principles

15. Review Checklist

請建立 change-02 驗收檢查表。

檢查表必須讓我能回答：

- baseline methods 是否明確？
- NewReno / CUBIC / BBR 角色是否清楚？
- BBR 是否有 fallback？
- topology 是否清楚？
- metrics 是否清楚？
- logging artifacts 是否清楚？
- scenario matrix 是否清楚？
- 是否避免直接進入 DRL？
- 是否避免實作？
- 是否能餵給 change-03 / change-04？
- 是否符合 change-01 的 scope / non-goals？

請用 checklist 格式輸出。

16. 給 Antigravity 的 change-02 指令草案

請幫我產出一段可以交給 Antigravity 的 change-02 指令。

這段指令必須說清楚：

- 目前只做 baseline benchmark specification
- 不准寫 code

- 不准跑 ns-3
- 不准啟動 ns3-gym
- 不准開始 DQN
- 不准接 PPO
- 不准加入 IPFS / QUIC / multi-agent / multi-path
- 必須只產出 OpenSpec artifacts
- 必須遵守 change-01 project-charter
- 完成後等待 spec owner 驗收

六、輸出品質要求

1. 語言

- 全文使用繁體中文
- 必要英文專有名詞保留英文
- 檔名、change ID、artifact 名稱保持英文

2. 深度

不要只列大綱。

要寫到我可以直接複製部分內容去建立 OpenSpec 文件。

3. 嚴謹度

所有重要判斷要有可信來源支撐。

若是推論，請明確寫「這是基於來源的推論」。

4. 禁止事項

本階段禁止：

- 寫任何程式碼
- 寫 shell command
- 產生 ns-3 實作
- 產生 Python 訓練腳本
- 執行 baseline benchmark
- 產出實驗圖表
- 進入 opengym-env

- 進入 drl-mvp
- 更改題目
- 擴張到 IPFS / QUIC / multi-agent / multi-path
- 宣稱已有 baseline 實驗結果

七、現在開始執行

請先搜尋官方文件與正式來源，再輸出。

請直接從：

OpenSpec change 02：baseline-benchmark 總結

開始。

收到。以下是 Phase 2 / OpenSpec change 02：baseline-benchmark 的最高品質最佳 prompt。

這一階段的重點不是跑 ns-3，也不是開始產生 baseline 圖，而是先把 baseline benchmark 的規格真相來源寫清楚，讓後續 Phase 3「Baseline 先行」可以照規格執行。ns-3 官方 TCP model 支援多種 TCP congestion control 演算法，包含 NewReno、CUBIC、BBR 等；Flow Monitor 官方文件也說明它可用於量測網路協定效能；ns3-gym 則是把 ns-3 與 RL 研究介面整合的框架。這些會被要求寫入 change-02 的 benchmark 規格基礎。

同時，OpenSpec 的工作方式是以 change folder 管理 proposal、specs、design、tasks，並在 implementation 前先完成規格對齊；所以這份 prompt 會強制產出 baseline benchmark 的 proposal.md / design.md / tasks.md / specs，但不允許執行任何程式。

請下一回合直接貼上以下整段：

你現在要執行的是我 DRL Final Project 的 **Phase 2：用 OpenSpec 建立專案真相來源** 的第二階段：

OpenSpec change 02：baseline-benchmark

最高任務目標

你要基於已驗收完成的：

- change-01-project-charter
- Phase 0 定案
- Phase 1 proposal 主體

替我產出一份可以作為後續 **Phase 3：Baseline 先行** 的 OpenSpec-style baseline benchmark 規格文件。

這不是實作任務，不是 ns-3 coding，不是跑 baseline，不是產圖。
這是 baseline benchmark 的 ****source of truth****。

本 change 的唯一目標是把後續 baseline 實作要做什麼、怎麼量測、怎麼驗收、哪些是必做、哪些是加分、哪些不能碰，全部寫成可交給 Antigravity 執行的規格。

一、已驗收前置內容

1. 已完成 change

01-project-charter 已驗收通過。

該 change 已凍結：

- 題目
- scope
- non-goals
- baseline list
- metrics
- MVP definition
- risk register
- downstream change map

本 change 必須完全繼承，不得自行改題、擴題或跳到 DRL。

2. 正式題目

中文題目

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

英文題目

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

GitHub repo name

drl-bottleneck-congestion-control

3. 本 change 的定位

本 change 是：

- baseline benchmark 規格階段
- ns-3 baseline 實驗設計階段

- metrics / logging / scenario / acceptance criteria 凍結階段
- change-03-opengym-env 與 change-04-drl-mvp 的前置依據

本 change 不是：

- ns-3 實作階段
- Python 訓練階段
- ns3-gym environment 建立階段
- DQN 訓練階段
- PPO 擴充階段
- IPFS / QUIC / kernel TCP 實作階段

二、change-02 的核心目的

change-02-baseline-benchmark 的任務是建立 **傳統 TCP baseline benchmark 的規格**。

它要回答：

1. baseline benchmark 要測什麼？
2. 為什麼 baseline 必須先於 DRL？
3. single bottleneck link 場景要如何定義？
4. NewReno / CUBIC / BBR 各自扮演什麼 baseline 角色？
5. throughput / RTT / loss / utility 要怎麼定義？
6. baseline 實驗至少要有哪些 scenario？
7. baseline 結果要輸出哪些 artifacts？
8. 後續 change-03 / change-04 如何使用這些 baseline？
9. 若 BBR 或 logging 有風險，怎麼降階？
10. Antigravity 後續 Phase 3 執行時，如何避免偏離 benchmark 規格？

三、本階段嚴格禁止事項

本 change 嚴格禁止：

- 不寫任何 C++ code
- 不寫任何 Python code
- 不寫 shell command
- 不建立 ns-3 topology 實作
- 不執行 ns-3
- 不產生實驗數據
- 不產生圖表
- 不啟動 ns3-gym
- 不定義 RL agent implementation
- 不訓練 DQN
- 不接 PPO

- 不接 IPFS
- 不接 QUIC
- 不修改 Linux kernel TCP
- 不做 multi-agent
- 不做 multi-path routing
- 不做大型 topology
- 不把 Pantheon 設為必裝依賴
- 不宣稱已有 baseline 實驗結果

你只能產出 OpenSpec 文件草案與規格內容。

四、你必須先搜尋並引用的來源

請在正式輸出前先做最新網路搜尋，不可只靠記憶。

優先使用來源

請優先搜尋並引用：

1. OpenSpec 官方網站 / GitHub README / docs
2. ns-3 官方 TCP model 文件
3. ns-3 Flow Monitor 官方文件
4. ns-3 tracing / statistics / measurement 相關官方文件，如有需要
5. ns3-gym 官方 GitHub / ns-3 app 頁面
6. Aurora 原始論文
7. Pantheon paper / 官方 benchmark 說明
8. 必要時補充正式論文或 RFC

你必須用來源確認的判斷

請確認並支撐以下判斷：

- OpenSpec 適合用來建立 change-based benchmark 規格
- ns-3 適合做 TCP / congestion control baseline
- ns-3 支援 NewReno、CUBIC、BBR 等 TCP congestion control
- Flow Monitor 或 ns-3 內建工具可支援 throughput / delay / loss 類指標量測
- baseline-first 是合理順序，因為後續 DRL 必須有比較基準
- Pantheon 適合作為 benchmark / reproducibility 思想來源，但不是 MVP 必裝依賴
- throughput 不能是唯一 baseline metric，必須同時規格化 RTT / loss / utility
- baseline benchmark 的結果將作為 change-04-drl-mvp 的比較基準

五、change-02 必須產出的內容

請嚴格按照以下章節輸出。

OpenSpec change 02 : baseline-benchmark 總結

請用 150-250 字說明：

- change 02 的目的
- 它如何承接 change 01
- 它為什麼必須在 DRL 前完成
- 它如何約束 change 03 / change 04
- 它如何避免 Antigravity 直接跳到 DQN

1. Change Metadata

請輸出：

- Change ID
- Change title
- Change type
- Status
- Owner
- Implementer
- Related phase
- Depends on
- Related downstream changes
- Required before implementation : Yes / No
- Code implementation allowed in this change : Yes / No

請明確設定：

- Change ID : 02-baseline-benchmark
- Change type : benchmark specification / experiment planning / metrics definition
- Status : draft for review
- Owner : user / spec owner
- Implementer : Antigravity, after approval
- Depends on : 01-project-charter
- Related downstream changes : 03-opengym-env, 04-drl-mvp
- Code implementation allowed : No

2. Baseline Benchmark Charter

請產出正式 baseline benchmark charter。

2.1 Benchmark Mission

請說明：

- baseline benchmark 的使命
- 為什麼 baseline 必須先行
- 為什麼不能直接跳到 DRL
- baseline 對後續 DQN 評估的作用

2.2 Benchmark Scope

請列出 baseline benchmark 要涵蓋：

- single bottleneck link
- sender / receiver
- TCP traffic flow
- NewReno
- CUBIC
- BBR
- throughput / RTT / loss / utility
- planned scenarios
- logs / tables / figures 的規格

2.3 Benchmark Non-Scope

請列出 baseline benchmark 不做：

- 不做 DRL
- 不做 ns3-gym
- 不做 DQN / PPO
- 不做 IPFS
- 不做 QUIC
- 不做 kernel TCP
- 不做 multi-agent
- 不做 multi-path
- 不做 large-scale topology
- 不做 Pantheon full integration

2.4 Benchmark Success Definition

成功不等於：

- baseline 已經跑出最好結果
- baseline 圖已經產生
- DRL 已經比較完成

成功等於：

- baseline methods 明確

- topology requirements 明確
- logging requirements 明確
- metrics definitions 明確
- scenario matrix 明確
- fallback 明確
- downstream changes 可依此使用 benchmark 規格

3. Baseline Methods Specification

請正式定義 baseline methods。

3.1 NewReno

請說明：

- 為什麼選它
- 它在本專題中的 baseline 角色
- 它和 DQN 的比較意義

3.2 CUBIC

請說明：

- 為什麼選它
- 它在本專題中的 baseline 角色
- 若 ns-3 預設為 CUBIC，這對 benchmark 有什麼意義

3.3 BBR

請說明：

- 為什麼選它
- 它在本專題中的 baseline 角色
- 為什麼它是 strongly recommended 而不是 hard MVP requirement
- 如果 BBR 有版本相依性或跑不起來，如何降階

3.4 Optional Baselines

請列出加分但不進 MVP hard requirement 的 baseline：

- Vegas
- Westwood
- DCTCP
- LEDBAT

請說明為什麼不把它們列入必做。

4. Topology Specification

請定義 single bottleneck link 的 benchmark topology。

4.1 Topology Concept

請用文字描述：

- sender
- receiver
- bottleneck link
- access links
- queue / buffer
- optional cross traffic

4.2 Minimal Topology

請定義最小拓樸：

- one sender
- one receiver
- one bottleneck link
- one long-lived TCP flow

4.3 Enhanced Topology

請定義加分拓樸：

- cross traffic
- variable bandwidth
- high-delay path
- multiple flows, 但只能作為 future extension, 不是 MVP

4.4 Topology Diagram Description

請用文字描述一張後續應畫出的圖：

Sender → Access Link → Bottleneck Queue / Link → Access Link → Receiver

請不要畫圖，只描述圖的構成與標籤。

5. Metrics and Logging Specification

這是本 change 最重要的部分之一。

請定義所有 baseline benchmark metrics。

5.1 Average Throughput

請定義：

- metric meaning
- why it matters
- expected unit
- data needed
- possible calculation concept
- logging requirement

5.2 Average RTT / Delay

請定義：

- metric meaning
- why it matters
- expected unit
- data needed
- logging requirement
- 和 bufferbloat / queueing delay 的關係

5.3 Packet Loss Rate

請定義：

- metric meaning
- why it matters
- expected unit
- data needed
- logging requirement

5.4 Utility Score

請定義：

- 為什麼需要 utility
- 為什麼不能只看 throughput
- 建議的概念形式

例如：

utility = normalized_throughput - penalty_delay - penalty_loss

請提供：

- 符號注釋
- 白話說明
- 注意此階段不固定最終權重，只固定 utility philosophy

5.5 Output Artifacts

請定義後續 Phase 3 至少要輸出的 artifacts：

- raw logs
- summary CSV
- per-scenario summary table
- baseline comparison table
- throughput plot
- RTT plot
- packet loss plot
- utility comparison plot
- experiment metadata

注意：

這裡只定義 artifacts，不產生 artifacts。

6. Scenario Matrix Specification

請設計 baseline benchmark scenario matrix。

至少包含 4 組情境：

1. Stable low-delay bottleneck
2. Stable high-delay bottleneck
3. Variable bandwidth bottleneck
4. Bottleneck with cross traffic

每組情境請提供：

- Scenario ID
- Purpose
- Bandwidth profile
- Delay profile
- Queue / buffer setting
- Traffic pattern
- Required baselines
- Required metrics
- Expected observation
- Risk level
- Whether MVP-required

請注意：

- 不要填過度細的實作參數
- 可以用 reasonable placeholder
- 但要足夠讓 Antigravity 後續實作時不偏航

7. Benchmark Evaluation Philosophy

請明確定義本專題 baseline benchmark 的評估哲學。

必須包含：

- baseline 不是敵人，而是比較基準
- 不以單一 throughput 最大化為唯一成功標準
- delay / RTT / loss 是同等重要的分析維度
- DRL 後續若輸給 BBR / CUBIC 也不是失敗，只要 trade-off analysis 完整
- baseline 的作用是提供後續 DQN 的 reference point
- baseline 實驗應可重現、可比較、可視覺化
- Pantheon 是 benchmark 思想來源，但不是本 change 的必裝工具

8. Downstream Dependency

請定義 change-02 如何餵給後續 change。

8.1 對 change-03-opengym-env 的輸入

請說明 baseline benchmark 如何幫助定義：

- observation candidates
- metric logging
- reward components
- scenario reuse
- episode / evaluation setup

8.2 對 change-04-drl-mvp 的輸入

請說明 baseline benchmark 如何幫助定義：

- DQN comparison target
- evaluation metrics
- reward curve interpretation
- success / failure analysis
- final project figures

9. Risk Register for Baseline Benchmark

請建立 change-02 專屬風險清單。

至少包含：

1. ns-3 TCP algorithm name / version mismatch
2. BBR availability issue
3. FlowMonitor / logging 不足以取得 RTT 或 loss
4. throughput 計算口徑不一致
5. scenario 設計太複雜
6. cross traffic 讓 MVP 失控
7. baseline 結果異常但無法解釋
8. Antigravity 直接跳到 DRL
9. baseline 圖表不足以支撐 PPT
10. Pantheon 被誤當必裝工具

表格欄位請包含：

- Risk ID
- Description
- Probability
- Impact
- Prevention
- Fallback
- Owner
- Trigger condition

10. OpenSpec Artifact Blueprint

請設計 change-02-baseline-benchmark 應產出的 artifact。

請不要真的建立檔案，但要提供建議結構與內容摘要。

至少包含：

```
text
openspec/
  changes/
    02-baseline-benchmark/
      proposal.md
      design.md
      tasks.md
      specs/
        baseline-methods.md
        topology.md
        metrics-logging.md
        scenario-matrix.md
        benchmark-risk-register.md
```

對每個檔案說明：

- purpose
- required content
- reviewer should check
- acceptance criteria

11. proposal.md 草案內容

請直接產出一份可以放入：

`openspec/changes/02-baseline-benchmark/proposal.md`

的 markdown 草案。

內容至少包含：

- Why
- What Changes
- What Does Not Change
- Impact
- Dependencies
- Acceptance Criteria

注意：

這是 OpenSpec change proposal，不是論文 proposal。

12. design.md 草案內容

請直接產出一份可以放入：

`openspec/changes/02-baseline-benchmark/design.md`

的 markdown 草案。

內容至少包含：

- Design Goal
- Benchmark Boundary
- Baseline Methods
- Topology Direction
- Metrics and Logging Direction
- Scenario Direction
- Downstream Usage
- Decision Records

注意：

design.md 只處理 benchmark 設計，不處理實作程式。

13. tasks.md 草案內容

請直接產出一份可以放入：

`openspec/changes/02-baseline-benchmark/tasks.md`

的 markdown 草案。

tasks 必須是規格任務，不是實作任務。

例如：

- Confirm baseline methods
- Confirm topology spec
- Confirm metrics definitions
- Confirm scenario matrix
- Confirm output artifacts
- Review no implementation included

每個 task 請用 checkbox 格式。

14. specs 草案摘要

請產出 specs 下五個檔案的摘要內容，不必寫成完整檔案，但要夠後續轉成完整 spec。

14.1 specs/baseline-methods.md

請提供：

- Purpose
- Required baselines
- Strongly recommended baselines
- Optional baselines
- Exclusion rules

14.2 specs/topology.md

請提供：

- Purpose
- Minimal topology
- Enhanced topology
- Topology assumptions
- Non-goals

14.3 specs/metrics-logging.md

請提供：

- Purpose
- Metric definitions
- Required logs
- Required output artifacts
- Ambiguity rules

14.4 specs/scenario-matrix.md

請提供：

- Purpose
- Required scenarios
- Scenario table summary
- MVP-required vs optional

14.5 specs/benchmark-risk-register.md

請提供：

- Purpose
- Risk categories
- Risk summary
- Fallback principles

15. Review Checklist

請建立 change-02 驗收檢查表。

檢查表必須讓我能回答：

- baseline methods 是否明確？
- NewReno / CUBIC / BBR 角色是否清楚？
- BBR 是否有 fallback？
- topology 是否清楚？
- metrics 是否清楚？
- logging artifacts 是否清楚？
- scenario matrix 是否清楚？
- 是否避免直接進入 DRL？
- 是否避免實作？
- 是否能餵給 change-03 / change-04？
- 是否符合 change-01 的 scope / non-goals？

請用 checklist 格式輸出。

16. 給 Antigravity 的 change-02 指令草案

請幫我產出一段可以交給 Antigravity 的 change-02 指令。

這段指令必須說清楚：

- 目前只做 baseline benchmark specification
- 不准寫 code
- 不准跑 ns-3
- 不准啟動 ns3-gym
- 不准開始 DQN
- 不准接 PPO
- 不准加入 IPFS / QUIC / multi-agent / multi-path
- 必須只產出 OpenSpec artifacts
- 必須遵守 change-01 project-charter
- 完成後等待 spec owner 驗收

六、輸出品質要求

1. 語言

- 全文使用繁體中文
- 必要英文專有名詞保留英文
- 檔名、change ID、artifact 名稱保持英文

2. 深度

不要只列大綱。

要寫到我可以直接複製部分內容去建立 OpenSpec 文件。

3. 嚴謹度

所有重要判斷要有可信來源支撐。

若是推論，請明確寫「這是基於來源的推論」。

4. 禁止事項

本階段禁止：

- 寫任何程式碼
- 寫 shell command
- 產生 ns-3 實作
- 產生 Python 訓練腳本
- 執行 baseline benchmark
- 產出實驗圖表
- 進入 opengym-env
- 進入 drl-mvp
- 更改題目
- 擴張到 IPFS / QUIC / multi-agent / multi-path
- 宣稱已有 baseline 實驗結果

七、現在開始執行

請先搜尋官方文件與正式來源，再輸出。

請直接從：

OpenSpec change 02: baseline-benchmark 總結

開始。

顯示更多顯示較少

OpenSpec change 02 : baseline-benchmark 總結

change-02-baseline-benchmark 的目的，是把傳統 TCP baseline benchmark 的方法、場景、指標、輸出物與驗收規格先定義清楚，作為後續 **Phase 3 : Baseline 先行** 的 source of truth。它承接已驗收的 **01-project-charter**，不改題、不擴題、不跳進 DRL，而是專注建立 NewReno、CUBIC、BBR 等 baseline 的比較規格。OpenSpec 的價值在於讓人與 AI coding assistant 在寫 code 前先對齊規格，且每個 change 可集中管理 proposal、specs、design、tasks，因此本 change 必須先完成文件化規格，才能交給 Antigravity 執行。



本 change 必須在 **03-opengym-env** 與 **04-drl-mvp** 前完成，因為 DRL agent 後續的 observation、reward、evaluation、success / failure analysis 都需要 baseline 作為 reference point。ns-3 官方 TCP model 支援 NewReno、CUBIC、BBR 等多種 congestion control 演算法；Flow Monitor 也可用於量測網路協定效能並收集 flow-level 統計，因此適合作為 baseline benchmark 的規格依據。 NSNAM+1

1. Change Metadata

欄位	內容
Change ID	02-baseline-benchmark
Change title	Baseline Benchmark Specification for TCP Congestion Control
Change type	benchmark specification / experiment planning / metrics definition
Status	draft for review
Owner	user / spec owner
Implementer	Antigravity, after approval
Related phase	Phase 2 : 用 OpenSpec 建立專案真相來源
Depends on	01-project-charter
Related downstream changes	03-opengym-env, 04-drl-mvp
Required before implementation	Yes
Code implementation allowed in this change	No

治理規則：

此 change 僅負責 baseline benchmark 規格。Antigravity 在此階段不得寫 C++、不得寫 Python、不得建立 ns-3 topology、不得執行 baseline、不得啟動 ns3-gym、不得開始 DQN。

2. Baseline Benchmark Charter

2.1 Benchmark Mission

Baseline benchmark 的使命，是為後續 DRL congestion-control agent 建立可信、可重現、可比較的傳統 TCP 參考基準。若沒有 baseline，後續 DQN 的表現就沒有合理比較對象，也無法判斷 agent 的 throughput、

RTT、loss、utility trade-off 是否有研究意義。

baseline 必須先於 DRL，原因有三個。第一，baseline 先行能確認 single bottleneck topology、traffic flow、logging、metrics 都定義正確。第二，baseline 結果會成為 **04-drl-mvp** 的 comparison target。第三，若直接跳到 DRL，Antigravity 很可能會把問題變成「先訓練模型」而忽略網路實驗口徑一致性。

本 change 不產生任何實驗結果，只定義未來 Phase 3 應如何跑 baseline、收集什麼資料、輸出什麼 artifacts、如何供後續 DRL 使用。

2.2 Benchmark Scope

baseline benchmark 規格涵蓋：

- single bottleneck link
- sender / receiver
- one long-lived TCP traffic flow
- TCP NewReno
- TCP CUBIC
- TCP BBR
- average throughput
- average RTT / delay
- packet loss rate
- utility score
- planned scenario matrix
- raw logs / summary CSV / tables / plots 的輸出規格

ns-3 官方 TCP 文件指出，native ns-3 TCP 支援完整雙向 TCP 與多種 congestion control 演算法，且 CUBIC 為 default，NewReno、BBR 等也被支援，因此這三個 baseline 適合作為本專題的核心比較對象。  NSNAM

2.3 Benchmark Non-Scope

baseline benchmark 不做：

- 不做 DRL
- 不做 ns3-gym environment
- 不做 DQN / PPO
- 不做 IPFS
- 不做 QUIC
- 不做 kernel TCP
- 不做 multi-agent
- 不做 multi-path routing

- 不做 large-scale topology
- 不做 Pantheon full integration
- 不產生實驗圖表
- 不宣稱 baseline 結果

Pantheon 可作為 benchmark / reproducibility 思想來源，因為它提供 Internet transport protocol 與 congestion control 的 common benchmark algorithms、shared evaluation platform 與 public archive 思維；但本專題 MVP 不把 Pantheon 作為必裝依賴，以降低工具鏈風險。  Pantheon

2.4 Benchmark Success Definition

成功不等於：

- baseline 已經跑出最好結果
- baseline 圖已經產生
- DRL 已經比較完成
- CUBIC / BBR / NewReno 表現已經被證明
- 已經完成 Phase 3 實驗

成功等於：

- baseline methods 明確
- topology requirements 明確
- logging requirements 明確
- metrics definitions 明確
- scenario matrix 明確
- fallback 明確
- downstream changes 可依此使用 benchmark 規格
- Antigravity 可在 Phase 3 依規格執行，而不自行擴題

3. Baseline Methods Specification

3.1 NewReno

選擇原因：

NewReno 是經典 loss-based TCP congestion control，適合作為傳統 TCP baseline。它代表以 loss / ACK feedback 為核心的較基本壅塞控制思路。

在本專題中的 baseline 角色：

NewReno 是最基礎的比較基準，用來回答：「DRL agent 至少能否在單一瓶頸環境中形成可與經典 TCP 比較的控制行為？」

與 DQN 的比較意義：

若 DQN 連 NewReno 都無法接近，代表 state、action、reward 或 training setup 可能需要檢討。若 DQN 在部分情境接近或超過 NewReno，則可進一步與 CUBIC / BBR 比較。

3.2 CUBIC

選擇原因：

CUBIC 是現代常見 TCP congestion control，且 ns-3 官方文件指出 CUBIC 是 native ns-3 TCP model 的 default congestion control algorithm。 

在本專題中的 baseline 角色：

CUBIC 應作為主要 baseline。相較 NewReno，CUBIC 更能代表現代 TCP 的標準比較對象。

若 ns-3 預設為 CUBIC，這對 benchmark 的意義：

若指定版本的 ns-3 預設為 CUBIC，Antigravity 在 Phase 3 仍必須明確記錄使用的 TCP variant，而不能只寫「default TCP」。baseline benchmark 規格必須要求每個 scenario 明確標示 TCP algorithm，以避免實驗結果不可追溯。

3.3 BBR

選擇原因：

BBR 的設計思路與 loss-based TCP 不同，通常以 bottleneck bandwidth 與 RTT 估計作為核心概念。將 BBR 納入 baseline，可讓後續 DRL 不只和 loss-based method 比，也能和較現代的 bandwidth-delay based control 思路比較。

在本專題中的 baseline 角色：

BBR 是強烈建議 baseline。它能提升 benchmark 的說服力，也能讓後續 DQN 的 trade-off analysis 更完整。

為什麼是 strongly recommended 而不是 hard MVP requirement：

雖然 ns-3 官方文件列出 BBR 為支援演算法，但實際可用性可能受 ns-3 版本、API、logging 或實作細節影響。為了降低 MVP 風險，BBR 不應阻塞 baseline benchmark 的最低完成。若 BBR 跑不起來，MVP 仍可先用 NewReno + CUBIC 完成 baseline-first 流程。這是基於 ns-3 版本相依性與 final project 完成率的風險控管推論。 

降階方案：

- 第一降階：NewReno + CUBIC 作為 MVP baseline
- 第二降階：BBR 留作 **should-have**
- 第三降階：以文獻 / related work 說明 BBR，實作比較延後到 future work

3.4 Optional Baselines

加分但不列入 MVP hard requirement：

- Vegas
- Westwood
- DCTCP

- LEDBAT

這些 baseline 不列入必做，原因是 baseline 數量過多會稀釋 MVP 目標，增加 Phase 3 實驗矩陣與圖表整理負擔。ns-3 官方文件列出 Vegas、Westwood、DCTCP、LEDBAT 等亦為支援演算法，但本專題核心目的不是做 TCP variant survey，而是建立 DRL baseline comparison pipeline。  NSNAM

4. Topology Specification

4.1 Topology Concept

baseline benchmark 採用 single bottleneck topology。概念上包含：

- **sender**：產生 TCP traffic flow 的來源端
- **receiver**：接收 TCP traffic flow 的目的端
- **access links**：sender / receiver 與 bottleneck 之間的連線
- **bottleneck link**：頻寬較低或限制明確的核心鏈路
- **queue / buffer**：位於 bottleneck 附近，用於觀察排隊、delay、loss
- **optional cross traffic**：加分場景，用於測試干擾下的 baseline 表現

4.2 Minimal Topology

MVP 最小拓樸：

- one sender
- one receiver
- one bottleneck link
- one long-lived TCP flow
- fixed bandwidth
- fixed propagation delay
- fixed queue / buffer setting

此拓樸是 baseline benchmark 的最低風險起點。它保留 congestion control 的核心問題，同時避免 multi-flow、multi-path、large topology 帶來的複雜度。

4.3 Enhanced Topology

加分拓樸，只能作為 future extension：

- cross traffic
- variable bandwidth
- high-delay path

- multiple flows
- shared bottleneck fairness scenario

這些拓樸可增加研究深度，但不得阻塞 MVP。尤其 multiple flows 與 fairness 可能會引入 Jain's fairness index 等額外指標，應放在 MVP 完成後再評估。

4.4 Topology Diagram Description

後續應畫出的圖如下：

Sender → Access Link → Bottleneck Queue / Link → Access Link → Receiver

圖中應標示：

- sender
- receiver
- access link bandwidth / delay placeholder
- bottleneck link bandwidth / delay placeholder
- bottleneck queue / buffer
- TCP flow direction
- optional cross traffic arrow，僅在 enhanced scenario 圖中出現

圖的目的不是展示實作細節，而是讓 PPT / README / video 能一眼看懂 benchmark 場景。

5. Metrics and Logging Specification

5.1 Average Throughput

Metric meaning

Average throughput 指接收端在觀察時間內成功接收的資料量除以時間，通常可用 Mbps 表示。

Why it matters

吞吐量是本題的核心效能指標，代表鏈路資料傳輸效率。但它不能單獨作為成功標準，因為過高吞吐可能伴隨 queue buildup、RTT 增加或 packet loss。

Expected unit

- Mbps
- 或 Kbps，視實驗輸出尺度而定

Data needed

- received bytes
- measurement duration
- flow start / end time

Flow Monitor 官方文件指出，它會追蹤 flow-level packets，並收集 `timeFirstTxPacket`、`timeLastRxPacket`、`txBytes`、`rxBytes`、`txPackets`、`rxPackets` 等資料，這可支撐 throughput 類指標的計算。

Possible calculation concept

概念形式：

```
average_throughput = received_data_size / measurement_duration
```

此處只定義概念，不固定實作公式與單位換算。

Logging requirement

Phase 3 應輸出：

- per-flow received bytes
- measurement start time
- measurement end time
- computed throughput
- scenario ID
- baseline method

5.2 Average RTT / Delay

Metric meaning

Average RTT / delay 衡量封包往返或端到端延遲的平均狀態。本 benchmark 可先以 Flow Monitor 能取得的 delay 類統計作為 delay observation，再於實作階段確認是否可取得 RTT-specific trace。

Why it matters

delay 是壅塞控制的核心副作用。只追 throughput 可能造成 queue 累積，導致 queueing delay 上升，這正是 bufferbloat 相關問題的核心現象。

Expected unit

- ms
- seconds，視 ns-3 / Flow Monitor 輸出而定

Data needed

- delay sum

- received packet count
- optional RTT trace, 若後續實作可取得

Flow Monitor 官方文件列出 **delaySum** 與 **rxPackets** 等 flow-level 統計，可支撐平均 delay 類指標；但是否直接得到 TCP RTT，需在 Phase 3 依 ns-3 trace source 進一步確認。 

Logging requirement

Phase 3 應輸出：

- delay-related statistic
- rxPackets
- average delay
- 若可行，輸出 TCP RTT trace 或 RTT estimate
- scenario ID
- baseline method

和 bufferbloat / queueing delay 的關係

若 throughput 增加但 delay 大幅上升，可能代表傳輸策略讓 queue 過度累積。benchmark 應保留這類 trade-off 觀察，而不是只看 throughput 排名。

5.3 Packet Loss Rate

Metric meaning

Packet loss rate 指封包遺失比例，用於衡量壅塞或 queue drop 情況。

Why it matters

loss 是傳統 TCP congestion control 的重要訊號，也是使用者體驗與傳輸效率的重要風險。DRL agent 若以高 loss 換取 throughput，不應被視為成功。

Expected unit

- percentage
- ratio 0-1

Data needed

- transmitted packets
- received packets
- lost packets or drop count

Flow Monitor 官方文件說明 flow probe 會在 packets sent、forwarded、received、dropped 等點分類與追蹤 packets；其 flow statistics 也包含 tx / rx packet 類資料，因此可作為 loss 類指標規格依據。 

Logging requirement

Phase 3 應輸出：

- txPackets
- rxPackets
- lostPackets 或 drop-related statistic
- computed loss rate
- scenario ID
- baseline method

5.4 Utility Score

為什麼需要 utility

utility score 用於把 throughput、delay、loss 放在同一個比較框架中。因為單看 throughput 可能導致錯誤解讀：例如某 baseline throughput 最高，但 RTT 與 loss 也最高，未必代表整體控制品質最好。

為什麼不能只看 throughput

壅塞控制的目標不是單純「塞越多資料越好」，而是在網路容量、延遲與丟包之間取得平衡。Aurora 將 congestion control 視為 deep RL control problem，並指出 learned schemes 真實採用仍需面對 fairness、

 Proceedings of
Machine Learning

safety、generalization 等挑戰；這支撐本專題使用多指標評估，而非 throughput-only。 [Research](#)

建議概念形式

```
utility = normalized_throughput - penalty_delay - penalty_loss
```

符號注釋

- **normalized_throughput**：正規化後的吞吐量，值越高越好。
- **penalty_delay**：延遲懲罰，RTT / delay 越高，扣分越多。
- **penalty_loss**：丟包懲罰，loss rate 越高，扣分越多。

白話說明

utility 的意思是：「傳得快很好，但如果快是靠塞爆 queue 或大量丟包換來的，就要扣分。」

本階段規則

change-02 不固定最終權重，只固定 utility philosophy。最終權重應留給 change-03 / change-04 或 Phase 3 後續調整。

5.5 Output Artifacts

後續 Phase 3 至少要產出的 artifacts：

- raw logs
- summary CSV
- per-scenario summary table
- baseline comparison table
- throughput plot
- RTT / delay plot
- packet loss plot
- utility comparison plot
- experiment metadata

Experiment metadata 至少包含

- scenario ID
- baseline method
- topology version
- bandwidth profile
- delay profile
- queue / buffer setting
- traffic pattern
- measurement duration placeholder
- run ID / seed placeholder

注意：本 change 只定義 artifacts，不產生 artifacts。

6. Scenario Matrix Specification

Scenario ID	Purpose	Bandwidth profile	Delay profile	Queue / buffer setting	Traffic pattern	Required baselines	Required metrics
S1 Stable low-delay bottleneck	建立最基本 baseline，比較各 TCP 在穩定低延	Fixed, reasonable placeholder	Low delay placeholder	Medium queue placeholder	One long-lived TCP flow	NewReno, CUBIC; BBR strongly recommended	throughput delay/RTT, loss, utility

Scenario ID	Purpose	Bandwidth profile	Delay profile	Queue / buffer setting	Traffic pattern	Required baselines	Required metrics
	遲瓶頸下的表現						
S2 Stable high-delay bottleneck	測試高延遲路徑下 baseline 的 throughput-delay trade-off	Fixed, same or comparable to S1	High delay placeholder	Medium queue placeholder	One long-lived TCP flow	NewReno, CUBIC; BBR strongly recommended	throughput delay/RTT, loss, utility
S3 Variable bandwidth bottleneck	測試 baseline 面對容量變動時的穩定性	Variable bandwidth placeholder	Medium delay placeholder	Medium queue placeholder	One long-lived TCP flow	NewReno, CUBIC; BBR optional if stable	throughput delay/RTT, loss, utility
S4 Bottleneck with cross traffic	測試 baseline 在干擾流量下的穩定性	Fixed or variable placeholder	Medium delay placeholder	Medium or small queue placeholder	Main TCP flow + cross traffic	NewReno, CUBIC; BBR optional	throughput delay/RTT, loss, utility

Scenario 設計規則

- S1 / S2 是 MVP baseline 的主軸。
- S3 是 strongly recommended，但不應阻塞 MVP。
- S4 是加分或 future extension，不應讓 Phase 3 失控。
- 不在 change-02 固定過細實作參數，避免後續 Antigravity 被不必要數值綁死。
- 所有 scenario 必須輸出相同 metrics，否則後續無法比較。

7. Benchmark Evaluation Philosophy

本專題 baseline benchmark 的評估哲學如下：

1. baseline 不是敵人，而是比較基準。

NewReno、CUBIC、BBR 不是要被「打敗」的對象，而是用來校準後續 DQN agent 表現的 reference point。

2. 不以單一 throughput 最大化為唯一成功標準。

throughput 重要，但若 RTT 或 loss 過高，整體傳輸品質仍可能變差。

3. delay / RTT / loss 是同等重要的分析維度。

baseline comparison 至少要同時輸出 throughput、delay/RTT、loss、utility。

4. DRL 後續若輸給 BBR / CUBIC 也不是失敗。

若 DQN 在某些情境低於 baseline，但能揭示 reward 設計、state representation 或 training stability 的限制，仍可形成合理 final findings。

5. **baseline** 的作用是提供 DQN 的 **reference point**。

change-04-drl-mvp 必須沿用 change-02 的 metrics 與 scenario 口徑。

6. **baseline** 實驗應可重現、可比較、可視覺化。

Pantheon 作為 congestion-control benchmark 的重要價值在於共同演算法集合、shared evaluation platform 與 public results archive；本專題吸收其 reproducibility 精神，但不要求 MVP 導入 Pantheon 工具鏈。  Pantheon

8. Downstream Dependency

8.1 對 change-03-opengym-env 的輸入

change-02 會提供 change-03 以下輸入：

Observation candidates

baseline logging 中會規格化：

- throughput
- delay / RTT
- loss
- congestion indicator
- queue-related signal if available

這些將成為 ns3-gym environment 的 observation candidates。

Metric logging

change-03 必須沿用 change-02 的 logging philosophy，確保 DRL environment 的 metrics 和 baseline benchmark 可比較。

Reward components

change-02 定義的 throughput、delay、loss 將成為 reward components 的基礎：

```
reward = throughput benefit - delay penalty - loss penalty
```

Scenario reuse

change-03 應優先重用 S1 / S2，以避免 environment 與 baseline 場景不一致。

Episode / evaluation setup

change-03 的 episode 設計應參照 baseline scenario 的 measurement window、traffic pattern 與 metric collection 需求。

8.2 對 change-04-drl-mvp 的輸入

change-02 會提供 change-04 以下輸入：

DQN comparison target

DQN 必須與 NewReno / CUBIC / BBR 或其 MVP subset 比較。

Evaluation metrics

DQN 必須輸出與 baseline 相同的：

- throughput
- delay / RTT
- loss
- utility

Reward curve interpretation

DQN 的 reward curve 不能單獨解讀，必須搭配 baseline metrics 看是否學到合理 trade-off。

Success / failure analysis

若 DQN 低於 CUBIC / BBR，change-04 必須根據 change-02 定義的 evaluation philosophy，做 limitation / trade-off analysis，而非視為專案失敗。

Final project figures

change-04 最終圖表應至少包含：

- baseline comparison
- DQN vs baseline comparison
- reward curve
- utility comparison

9. Risk Register for Baseline Benchmark

Risk ID	Description	Probability	Impact	Prevention	Fallback	Owner	Trigger condition
B-R1	ns-3 TCP algorithm name / version mismatch	Medium	High	在 change-02 僅凍結 algorithm role，不寫死 API name	Phase 3 查版本後更新 implementation notes	Antigravity	指定 TCP variant 無法被版本識別
B-R2	BBR availability issue	Medium	Medium	BBR 設為 strongly	MVP 先 NewReno + CUBIC	Antigravity	BBR 在指定 ns-3 版本跑不起來

Risk ID	Description	Probability	Impact	Prevention	Fallback	Owner	Trigger condition
				recommended, 不設為 hard MVP			
B-R3	FlowMonitor / logging 不足以取得 RTT 或 loss	Medium	High	delay / loss 分開規格化；RTT-specific trace 另列 should-have	先用 delaySum / rxPackets / txPackets 等 flow-level 指標	Antigravity	無法取得直接 RTT trace
B-R4	throughput 計算口徑不一致	Medium	High	規格要求所有 baseline 使用相同 measurement window 與 calculation concept	重跑 baseline, 統一 summary CSV schema	Antigravity	不同 baseline 使用不同時間窗
B-R5	scenario 設計太複雜	Medium	Medium	S1 / S2 列為 MVP, S3 / S4 不阻塞 MVP	只保留 stable low-delay / high-delay	Spec owner	Phase 3 scope 超出預期
B-R6	cross traffic 讓 MVP 失控	High	Medium	S4 設為 optional / future extension	移除 cross traffic, 只做單流	Spec owner	baseline 結果波動難解釋
B-R7	baseline 結果異常但無法解釋	Medium	Medium	要求輸出 metadata 與 per-scenario summary	將異常情境標為 limitation, 不作為主要比較	Antigravity	指標和預期差距極大
B-R8	Antigravity 直接跳到 DRL	Medium	High	change-02 明確禁止 ns3-gym / DQN / PPO	停止工作, 回到 spec owner review	Spec owner	出現 DQN、agent、training script
B-R9	baseline 圖表不足以支撐 PPT	Medium	Medium	artifact 規格先要求 throughput / delay / loss / utility plots	PPT 使用 table + topology + metric definition 作補充	Antigravity	缺少至少 3 種核心圖
B-R10	Pantheon 被誤當必裝工具	Medium	Medium	明確寫 Pantheon 是 benchmark philosophy, 不是 dependency	移除 Pantheon 實作任務, 只保留 related work	Spec owner	任務出現 Pantheon integration

10. OpenSpec Artifact Blueprint

建議 change-02 artifact 結構：

```

openspec/
  changes/
    02-baseline-benchmark/
      proposal.md
      design.md
      tasks.md
      specs/
        baseline-methods.md

```

topology.md
 metrics-logging.md
 scenario-matrix.md
 benchmark-risk-register.md

檔案	purpose	required content	reviewer should check	acceptance criteria
proposal.md	說明為什麼需要 baseline benchmark spec	Why, What Changes, What Does Not Change, Impact, Dependencies, Acceptance Criteria	是否承接 change-01；是否禁止實作	不含 code、不含命令、不宣稱結果
design.md	定義 benchmark 設計方向	Benchmark Boundary, Baseline Methods, Topology, Metrics, Scenario, Downstream Usage	是否能餵給 Phase 3 與 change-03 / 04	baseline 設計完整且不跳 DRL
tasks.md	定義 change-02 規格任務	checkbox tasks	是否全為規格任務	沒有實作任務
specs/baseline-methods.md	凍結 baseline 角色	NewReno, CUBIC, BBR, optional baselines	BBR 是否有 fallback	baseline list 清楚
specs/topology.md	凍結 single bottleneck topology	minimal / enhanced topology	MVP 是否不被 enhanced topology 阻塞	topology 清楚且不擴題
specs/metrics-logging.md	凍結 metrics 與 artifacts	throughput, RTT/delay, loss, utility, logs, CSV, plots	指標口徑是否一致	可餵給 Phase 3 logging
specs/scenario-matrix.md	凍結 scenario matrix	S1-S4	MVP-required vs optional 是否明確	S1/S2 可做 MVP
specs/benchmark-risk-register.md	管理 baseline 風險	risk table, fallback	是否包含 BBR、logging、scope drift	每項高風險有 fallback

11. proposal.md 草案內容

Markdown

Change 02: Baseline Benchmark

Why

The project requires a clear TCP baseline benchmark specification before any reinforcement learning work begins.

The final project is focused on Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link. Before building an ns3-gym environment or training a DQN agent, the project must define traditional TCP baselines, topology assumptions, metrics, scenario matrix, and required output artifacts.

Without this baseline specification, future DRL results would lack a valid comparison point, and AntigraVity may prematurely jump into RL implementation without verifying the benchmark foundation.

What Changes

This change defines:

- Baseline benchmark mission
- Benchmark scope and non-scope
- Baseline methods: NewReno, CUBIC, BBR
- Optional baselines: Vegas, Westwood, DCTCP, LEDBAT
- Single bottleneck topology requirements
- Metrics and logging requirements
- Utility score philosophy
- Required output artifacts
- Scenario matrix
- Downstream dependency for opengym-env and drl-mvp
- Benchmark-specific risk register

What Does Not Change

This change does not implement anything.

This change does not:

- Write C++ code
- Write Python code
- Run ns-3
- Create ns-3 topology implementation
- Generate experiment data
- Generate plots
- Start ns3-gym
- Define or train DQN/PPO agents
- Implement IPFS
- Implement QUIC
- Modify kernel TCP
- Add multi-agent RL
- Add multi-path routing
- Require Pantheon integration

Impact

This change becomes the source of truth for Phase 3: Baseline First.

It also provides required inputs for:

- 03-opengym-env
- 04-drl-mvp

All later DRL evaluation must compare against the baseline methods and metrics defined here.

Dependencies

Depends on:

- 01-project-charter

Inputs inherited from change-01:

- Project title
- Scope / non-goals
- Baseline list
- Metrics
- MVP definition
- Risk register
- Downstream change map

Acceptance Criteria

- [] Baseline methods are clearly defined.
- [] NewReno, CUBIC, and BBR roles are clear.
- [] BBR has a fallback plan.
- [] Single bottleneck topology is defined.
- [] Minimal and enhanced topology boundaries are separated.
- [] Metrics are clearly defined.
- [] Logging artifacts are defined.
- [] Scenario matrix is defined.
- [] MVP-required scenarios are separated from optional scenarios.
- [] Downstream dependency to 03-opengym-env is clear.
- [] Downstream dependency to 04-drl-mvp is clear.
- [] Risk register is included.
- [] No implementation code is included.

- [] No shell commands are included.
- [] No experimental results are claimed.

12. design.md 草案内容

Markdown

Design: Baseline Benchmark Specification

Design Goal

The goal of this design is to define the baseline benchmark foundation for the DRL congestion-control final project.

This change specifies which TCP baselines will be evaluated, what topology they will run on, what metrics must be logged, which scenarios are required, and how these baseline results will later serve as reference points for DRL evaluation.

Benchmark Boundary

The benchmark is limited to a single bottleneck link.

In scope:

- one sender
- one receiver
- one bottleneck link
- one long-lived TCP flow
- NewReno
- CUBIC
- BBR if supported cleanly
- throughput, delay/RTT, loss, utility

Out of scope:

- DRL
- ns3-gym
- DQN
- PPO
- IPFS
- QUIC
- kernel TCP
- multi-agent RL
- multi-path routing
- large-scale topology
- Pantheon full integration

Baseline Methods

Required:

- NewReno
- CUBIC

Strongly recommended:

- BBR

Optional:

- Vegas
- Westwood
- DCTCP
- LEDBAT

BBR must not block the MVP. If BBR is unavailable or unstable in the selected ns-3 version, the MVP proceeds with NewReno and CUBIC.

Topology Direction

Minimal topology:

Sender -> Access Link -> Bottleneck Queue / Link -> Access Link -> Receiver

The minimal topology must use one long-lived TCP flow and one bottleneck link.

Enhanced topology may include:

- variable bandwidth
- high-delay path
- cross traffic
- multiple flows

Enhanced topology is optional and must not block the MVP.

Metrics and Logging Direction

Required metrics:

- average throughput
- average delay or RTT
- packet loss rate
- utility score

Required artifacts:

- raw logs
- summary CSV
- per-scenario summary table
- baseline comparison table
- throughput plot
- delay/RTT plot
- packet loss plot
- utility comparison plot
- experiment metadata

Scenario Direction

Required MVP scenarios:

- S1: Stable low-delay bottleneck
- S2: Stable high-delay bottleneck

Strongly recommended:

- S3: Variable bandwidth bottleneck

Optional:

- S4: Bottleneck with cross traffic

Downstream Usage

03-opengym-env must reuse the metrics and scenario definitions from this benchmark.

04-drl-mvp must compare DQN results against the baseline methods and metrics defined here.

Decision Records

DR-02-01: Baseline before DRL

Reason: DRL evaluation requires a valid comparison reference.

DR-02-02: NewReno and CUBIC are MVP-required

Reason: They provide a basic and modern TCP baseline pair.

DR-02-03: BBR is strongly recommended but not hard-required

Reason: It improves benchmark quality but may introduce version-specific risk.

DR-02-04: Throughput is not the only metric

Reason: Congestion control must consider throughput-delay-loss trade-offs.

DR-02-05: Pantheon is a benchmark philosophy reference, not a dependency

Reason: The MVP must reduce tooling complexity while preserving reproducibility thinking.

13. tasks.md 草案內容

Markdown

Tasks: Change 02 Baseline Benchmark

Specification Tasks

- [] Confirm that change-02 depends on 01-project-charter.
- [] Confirm that no implementation is included in this change.
- [] Confirm baseline benchmark mission.
- [] Confirm benchmark scope.
- [] Confirm benchmark non-scope.
- [] Confirm NewReno baseline role.
- [] Confirm CUBIC baseline role.
- [] Confirm BBR baseline role.
- [] Confirm BBR fallback plan.
- [] Confirm optional baselines.
- [] Confirm minimal single bottleneck topology.
- [] Confirm enhanced topology boundaries.
- [] Confirm topology diagram description.
- [] Confirm average throughput definition.
- [] Confirm delay / RTT definition.
- [] Confirm packet loss rate definition.
- [] Confirm utility score philosophy.
- [] Confirm required output artifacts.
- [] Confirm scenario S1.
- [] Confirm scenario S2.
- [] Confirm scenario S3.
- [] Confirm scenario S4.
- [] Confirm evaluation philosophy.
- [] Confirm downstream dependency to 03-opengym-env.
- [] Confirm downstream dependency to 04-drl-mvp.
- [] Confirm benchmark risk register.
- [] Review that no C++ code is included.
- [] Review that no Python code is included.
- [] Review that no shell command is included.
- [] Review that no experiment result is claimed.
- [] Submit change-02 to spec owner for review.

14. specs 草案摘要

14.1 specs/baseline-methods.md

Purpose

定義本專題 baseline benchmark 中使用的 TCP congestion control methods, 並明確區分 MVP-required、strongly recommended、optional。

Required baselines

- NewReno
- CUBIC

Strongly recommended baselines

- BBR

Optional baselines

- Vegas
- Westwood
- DCTCP
- LEDBAT

Exclusion rules

- 不加入非 TCP baseline 作為 MVP
- 不加入 QUIC baseline
- 不加入 Pantheon full integration
- 不加入 production transport protocols

14.2 specs/topology.md

Purpose

定義 baseline benchmark 的 single bottleneck link topology。

Minimal topology

- one sender
- one receiver
- one bottleneck link
- one long-lived TCP flow
- fixed bandwidth
- fixed delay
- fixed queue / buffer placeholder

Enhanced topology

- variable bandwidth
- high delay
- cross traffic
- multiple flows

Topology assumptions

- MVP 使用 single flow
- enhanced topology 不阻塞 MVP
- cross traffic 僅作 optional scenario

Non-goals

- no large-scale topology
- no multi-path
- no multi-agent
- no IPFS / QUIC overlay

14.3 specs/metrics-logging.md

Purpose

定義 baseline benchmark 的 metrics、logging requirements 與 output artifacts。

Metric definitions

- average throughput
- average delay / RTT
- packet loss rate
- utility score

Required logs

- raw flow logs
- per-flow statistics
- received bytes
- tx / rx packets
- delay-related statistics
- loss / drop-related statistics if available

Required output artifacts

- raw logs
- summary CSV
- per-scenario table
- baseline comparison table
- throughput plot
- delay / RTT plot
- loss plot
- utility plot
- experiment metadata

Ambiguity rules

- 若 RTT trace 不可得，先使用 Flow Monitor delay 類統計作替代並明確標註。
- 若 loss 無法直接取得，使用 tx / rx / drop 類資訊推導並標註計算口徑。
- 所有 baseline 必須使用相同 measurement window。

14.4 specs/scenario-matrix.md

Purpose

定義 baseline benchmark 的 scenario matrix。

Required scenarios

- S1 Stable low-delay bottleneck
- S2 Stable high-delay bottleneck

Scenario table summary

- S1 / S2 : MVP-required
- S3 variable bandwidth : strongly recommended
- S4 cross traffic : optional / future extension

MVP-required vs optional

MVP 只要求 S1 / S2 先完成。S3 / S4 不得阻塞 MVP。

14.5 specs/benchmark-risk-register.md

Purpose

管理 baseline benchmark 階段風險。

Risk categories

- toolchain version risk
- logging ambiguity risk
- scenario complexity risk
- baseline availability risk
- scope drift risk
- reporting artifact risk

Risk summary

核心風險包含 BBR availability、FlowMonitor / RTT ambiguity、throughput calculation inconsistency、Antigravity jumping to DRL、Pantheon mistaken as dependency。

Fallback principles

- baseline-first
- NewReno + CUBIC can satisfy MVP
- BBR does not block MVP
- S1 / S2 can satisfy MVP
- S3 / S4 are optional if unstable
- no DRL until baseline spec is approved

15. Review Checklist

change-02 驗收檢查表

- ☐ baseline methods 是否明確？
- ☐ NewReno 的角色是否清楚？
- ☐ CUBIC 的角色是否清楚？
- ☐ BBR 的角色是否清楚？
- ☐ BBR 是否有 fallback？
- ☐ optional baselines 是否未被列入 MVP hard requirement？
- ☐ topology 是否清楚？
- ☐ minimal topology 是否足以支撐 MVP？
- ☐ enhanced topology 是否被標成 optional / future extension？
- ☐ metrics 是否清楚？
- ☐ throughput 定義是否清楚？
- ☐ RTT / delay 定義是否清楚？
- ☐ loss 定義是否清楚？
- ☐ utility philosophy 是否清楚？
- ☐ logging artifacts 是否清楚？
- ☐ scenario matrix 是否清楚？
- ☐ S1 / S2 是否為 MVP-required？
- ☐ S3 / S4 是否未阻塞 MVP？
- ☐ 是否避免直接進入 DRL？
- ☐ 是否避免實作？
- ☐ 是否未寫 C++ code？

- ☐ 是否未寫 Python code ?
- ☐ 是否未寫 shell command ?
- ☐ 是否未宣稱 baseline 結果 ?
- ☐ 是否能餵給 change-03 ?
- ☐ 是否能餵給 change-04 ?
- ☐ 是否符合 change-01 的 scope ?
- ☐ 是否符合 change-01 的 non-goals ?
- ☐ 是否禁止 IPFS / QUIC / multi-agent / multi-path 擴張 ?
- ☐ 是否把 Pantheon 定位為 benchmark philosophy 而非 dependency ?

16. 給 Antigravity 的 change-02 指令草案

You are working on Phase 2 / OpenSpec change 02: baseline-benchmark.

Current task:

Create only the OpenSpec baseline benchmark specification artifacts for the DRL final project.

Project:

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

Repository:

drl-bottleneck-congestion-control

You must follow the approved 01-project-charter.

You must only produce documentation artifacts for:

- openspec/changes/02-baseline-benchmark/proposal.md
- openspec/changes/02-baseline-benchmark/design.md
- openspec/changes/02-baseline-benchmark/tasks.md
- openspec/changes/02-baseline-benchmark/specs/baseline-methods.md
- openspec/changes/02-baseline-benchmark/specs/topology.md
- openspec/changes/02-baseline-benchmark/specs/metrics-logging.md
- openspec/changes/02-baseline-benchmark/specs/scenario-matrix.md
- openspec/changes/02-baseline-benchmark/specs/benchmark-risk-register.md

Strict rules:

- Do not write C++ code.
- Do not write Python code.
- Do not write shell commands.
- Do not run ns-3.
- Do not create ns-3 topology implementation.
- Do not generate experiment data.
- Do not generate plots.
- Do not start ns3-gym.
- Do not create DQN or PPO training logic.
- Do not implement IPFS.
- Do not implement QUIC.
- Do not modify Linux kernel TCP.
- Do not introduce multi-agent RL.
- Do not introduce multi-path routing.
- Do not make Pantheon a required dependency.
- Do not claim any baseline result.

The purpose of this change is to define:

- baseline methods
- single bottleneck topology
- metrics and logging requirements
- output artifacts

- scenario matrix
- benchmark evaluation philosophy
- downstream dependency to opengym-env and drl-mvp
- risk register

After completing the documentation artifacts, stop and wait for spec owner review.

change-02 結論

OpenSpec change 02 : baseline-benchmark 的驗收重點，不是有沒有開始跑 ns-3，而是 baseline benchmark 的方法、拓樸、指標、輸出物與風險是否已足夠清楚。這份規格通過後，才適合進入後續 **change-03-opengym-env**；而真正執行 baseline，仍應等到 Phase 3「Baseline 先行」再做。

Phase 5：做 DQN MVP

截圖是 Phase 4：DRL MVP Implementation 的 commit

Antigravity 做完「Phase 4：DRL MVP Implementation」**最佳 Prompt** 指令了

並已推上 github Phase 4：DRL MVP Implementation 的異動 請幫我驗收 git diff

是否全部 **Phase 4：DRL MVP Implementation** 要求都有通過

是否能推進到「**Phase 5：做 DQN MVP**」了呢

顯示更多顯示較少

我懂你的焦慮，但我會很直接說：**DQN 表現不好，反而更像一個合格的 DRL 期末專題。**

因為 DRL 課不是在教你「丟一個 agent 進去就一定贏」，而是在教你：

如何把問題建成 MDP、定義 state/action/reward、建立 environment、設 baseline、訓練 agent、評估 policy，最後分析為什麼成功或失敗。

你現在有做到這些，所以你不是「不好意思說在上 DRL 課」。你真正要避免的是把 DQN 包裝成「很強」。你應該把它包裝成：

DQN MVP 成功完成，但結果暴露了 Discrete(3) action space 在 congestion control 上的限制，因此 Phase 6 進一步提出 PPO / continuous action extension。

這樣才是研究型 DRL。

你的 DQN 真的很難看嗎？

不要用「有沒有贏 baseline」當唯一標準。這是錯的。

你目前 DQN 的價值是：

1. 它跑得起來：ns-3 / ns3-gym / SB3 DQN pipeline 已完成。
2. 它有 **policy** 行為：S1 學到 100% increase，雖然是 degenerate policy，但不是 random。
3. 它揭露 **action space** 限制：S2 high loss 說明三段式動作太粗。
4. 它有和 **NewReno / CUBIC / BBR** 比較：這比單純 reward curve 強很多。
5. 它支撐下一步 **PPO** 的動機：不是憑空說 PPO，而是由 DQN failure case 推出 PPO。

這在 DRL 課裡是合理成果。

為什麼 DQN 沒贏 TCP baseline 一點都不丟臉？

因為你挑的對手不是弱 baseline。NewReno、CUBIC、BBR 不是隨便的 heuristic，它們是長期工程演化出來的 congestion-control family。TCP congestion control 本來就是 Internet 的核心機制，牽涉 cwnd、slow start、loss、delay、fairness、stability 等多個權衡；傳統 TCP 不是簡單任務。^W [維基百科](#)

而且 DRL congestion control 本來就是困難題。Aurora 這類 DRL congestion-control work 雖然展示了 RL 可以應用在 Internet congestion control，但原論文也明確指出，真實採用會遇到 fairness、safety、generalization 等挑戰，這些不是一般 RL formulation 能輕鬆解掉的。^X [arXiv](#)

所以你現在的結果不是「DRL 很爛」，而是：

在 single bottleneck + coarse discrete action + DQN 的 MVP 設定下，agent 能學到 throughput-seeking behavior，但無法穩定處理 delay/loss trade-off。

這句話非常像研究報告，不丟臉。

你應該怎麼講才有面子？

不要說：

DQN 表現很差。

要說：

DQN MVP revealed a key control limitation: with only three coarse actions, the agent tends to over-aggressively increase sending rate. This explains the S1 degenerate policy and S2 high loss, and motivates PPO with continuous action as Phase 6 extension.

中文可以講：

DQN MVP 的價值不在於全面擊敗 TCP baseline，而在於證明 DRL pipeline 可行，並揭露粗粒度離散動作在壅塞控制上的限制。S1 的 100% increase policy 與 S2 的高丟包，正好說明後續需要 PPO / continuous action 這種更細緻的控制方式。

這樣老師會覺得你知道自己在幹嘛。

DRL 課的「漂亮成果」不一定是贏

在研究上，「模型沒贏」不等於失敗。機器學習研究本來就有很多負結果，學界也一直在強調 negative results 的價值，因為它們能避免大家重複走錯路、揭露方法限制、改善研究透明度。2024 年一篇關於 machine learning negative results 的 position paper 就明確主張，不能只用 predictive performance 評價研究價值，negative results 對改善學術產出也有價值。 [arXiv](#)

在 systems / networking 題目中更是如此。你不是在玩 CartPole；你是在把 DRL 套進 congestion control。這類題目如果結果「過度漂亮」，反而可能讓人懷疑：

- baseline 是否太弱？
- metric 是否偏袒 DRL？
- reward 是否 hacking？
- 是否只挑一個 seed？
- 是否只報成功 scenario？

Stable-Baselines3 官方 tips 也提醒，RL 結果受 random seed 影響，evaluation 要用 separate test environment，不應只看 training reward。這也支持你現在的保守報告方式。 [維基百科](#)

你現在真正該做的是 Phase 6 PPO，不是遮掩 DQN

你問「怎麼好意思說在上 DRL 課」，我的答案是：

正因為 DQN 不夠好，你才更需要 Phase 6 PPO。

但 Phase 6 要這樣定位：

DQN MVP → reveals coarse-action limitation
PPO extension → tests whether policy-gradient / continuous-action control can improve rate adaptation

也就是：

不是：DQN 爛，所以補救
而是：DQN 建立 baseline，PPO 驗證下一個 DRL hypothesis

這樣就漂亮了。

報告主線應該改成這樣

你現在最好的報告故事是：

1. **Phase 3**：先建立 TCP baseline，不亂吹 DRL。
2. **Phase 4**：實作 DQN MVP，證明 DRL pipeline 可行。
3. **Phase 5 / 7**：完成 final package 與 honest evaluation。

4. **Phase 6**：因 DQN 顯示 Discrete(3) action 不足，加入 PPO / continuous action extension。
5. **最後結論**：本專題不是宣稱 DQN 全勝，而是完成「baseline → DQN → limitation → PPO extension」的完整 DRL 研究流程。

這樣就不是「DQN 很難看」。

這是一條很完整的 DRL 實驗路線。

最後我給你一句可以放心放進報告的核心句

The DQN agent did not universally outperform traditional TCP baselines. However, this negative result is informative: it reveals that coarse discrete rate-control actions are insufficient for robust congestion-control behavior, especially under high-delay conditions. This motivates Phase 6, where PPO and continuous-action control are introduced as a more suitable DRL extension.

中文：

DQN 並沒有全面勝過傳統 TCP baseline，但這不是本專題的失敗，而是重要發現：粗粒度離散動作不足以穩定處理高延遲壅塞控制，因此本專題進一步引出 Phase 6，以 PPO 與連續動作空間作為更合適的 DRL 擴展方向。

這樣講，你不但好意思說你在上 DRL 課，而且會比只報一張 reward curve 的同學更像研究型專題。

請幫我解釋 Delay is a proxy 這句話是什麼意思

以及介紹這幾個演算法各是什麼 還有 為什麼他們這麼厲害

BBR

DQN (ours)

CUBIC

NewReno

顯示更多顯示較少

可以，我用你這份專題的語境來解釋。

1. Delay is a proxy 是什麼意思？

proxy 在研究裡的意思是：我沒有直接量到真正想量的東西，所以用一個高度相關、可取得的替代量來代表它。

所以：

Delay is a proxy

意思是：這份專題裡的 delay 不是直接量到 TCP 真正的 RTT，而是用 ns-3 FlowMonitor 提供的 flow-level delay 統計，作為「延遲狀況」的替代指標。

你的專題用的是類似：

```
avg_delay = delaySum / rxPackets
```

這代表「收到的封包平均經歷多少 delay」。ns-3 FlowMonitor 官方文件確實會記錄每個 flow 的 `delaySum`、`rxPackets`、`lostPackets`、`txBytes`、`rxBytes` 等 flow-level 統計；但這個 delay 是封包層 / flow-level 的 delay 統計，不等於 TCP sender 實際用來做 congestion control 判斷的 direct TCP RTT。🟡 [Nsnam](#)

所以你報告裡不能說：

```
DQN has lower RTT
```

要說：

```
DQN has lower / higher delay proxy
```

或中文：

```
DQN 在 FlowMonitor delay proxy 上較低 / 較高
```

為什麼這很重要？

因為 **RTT 是 TCP 壅塞控制裡非常核心的訊號**。真正的 RTT 通常是 sender 看到「封包送出 → ACK 回來」的時間；但你目前報告裡的 delay proxy 是 FlowMonitor 從模擬中統計 flow packet delay 的替代值。

兩者高度相關，但不是同一個東西。

你可以這樣講：

本專題使用 FlowMonitor 的 average delay 作為 delay proxy，而非 direct TCP RTT。這個 proxy 足以支撐不同方法在模擬環境中的相對比較，但不能宣稱等同於真實 TCP RTT measurement。

這句很安全，也很學術。

2. BBR 是什麼？為什麼很厲害？

BBR 全名是：

Bottleneck Bandwidth and Round-trip propagation time

中文可以叫：

瓶頸頻寬與往返傳播時間壅塞控制

BBR 是 Google 提出的 congestion control。它和 NewReno、CUBIC 最大差異是：**BBR 不是主要靠丟包判斷壅塞，而是建立網路模型。**

BBR 會估計兩個東西：

1. **Bottleneck Bandwidth**：瓶頸鏈路可承受的最大傳輸率
2. **RTprop**：最低 round-trip propagation time，近似路徑本身的基本延遲

然後 BBR 會根據這兩個量決定 pacing rate 和 sending behavior。ACM Queue 的 BBR 原始文章說，BBR 不是根據 packet loss 或 buffer delay 反應，而是用最近 delivery rate 和 round-trip time 形成網路 path model。 queue.acm.org

BBR 為什麼厲害？

因為傳統 loss-based TCP 常常要等「丟包」才知道自己送太多。可是等到丟包時，queue 可能已經塞爆，delay 已經變高。

BBR 的想法比較像：

我先估計這條路最多能送多少、最低延遲是多少，然後盡量貼近這個最佳 operating point。

所以 BBR 很適合：

- 高頻寬鏈路
- bufferbloat 問題
- 長距離高 BDP 網路
- 需要高 throughput 又想控制 queue delay 的場景

但 BBR 不是無敵

BBR 的問題是 fairness、和其他 TCP 共存、不同 RTT flow 競爭時可能不公平。很多後續研究都在修 BBR fairness。

所以你報告裡可以這樣講：

BBR 是非常強的 model-based baseline。DQN 沒有贏 BBR 不代表 DRL 失敗，而是代表 BBR 本身已經是高度工程化、經過大量實戰優化的 congestion-control algorithm。

3. DQN (ours) 是什麼？為什麼它有意義？

DQN 全名是：

Deep Q-Network

中文可以叫：

深度 Q 網路

DQN 是把 Q-learning 和 deep neural network 結合。它學的是：

在某個 state 下，做某個 action，長期預期 reward 會是多少？

也就是學一個 Q function：

$Q(s, a)$

你的專題裡，DQN 的 state 大概是：

[throughput, delay proxy, loss, congestion signal, previous action]

action 是：

decrease / keep / increase

reward 是：

throughput reward - delay penalty - loss penalty

SB3 官方文件也說 DQN 使用 replay buffer、target network、gradient clipping 等設計來穩定 neural-network Q-learning，而且 DQN 支援 discrete action space；這正好對應你目前的 **Discrete(3)** action

 Stable Baselines3

design。 [Docs](#)

DQN 為什麼有意義？

DQN 的強項不是「一開始就比 TCP 強」，而是：

它讓 congestion control 從人工規則，轉成可以從環境 feedback 中學習的 sequential decision problem。

你的 DQN agent 不是照 NewReno / CUBIC 的固定公式走，而是看目前 throughput、delay、loss 狀態後，學著決定要 increase、keep、decrease。

為什麼你的 DQN 表現不好也有價值？

因為它揭露了 DRL control design 的限制：

現象	代表意思
S1 100% increase	low-delay 場景中，DQN 學到一個 near-capacity / degenerate policy
S2 high loss	high-delay 場景中，Discrete(3) 動作太粗，agent 太容易 aggressive
沒有全面贏 TCP	DQN MVP 還不夠細緻，不代表 DRL 方向錯

所以你要講：

DQN MVP 的貢獻是建立完整 DRL congestion-control pipeline，並揭露 coarse discrete action 對 congestion control 的限制。

而不是說：

DQN 很強，打敗 TCP。

這樣才是正確研究敘事。

4. CUBIC 是什麼？為什麼很厲害？

CUBIC 是現代 TCP 最重要的 congestion control 之一，也是 Linux 長期預設的 TCP congestion control。RFC 8312 說 CUBIC 是 TCP standards 的 extension，它把 sender-side congestion window 的增加方式從傳統線性增加，改成 cubic function，以改善 fast long-distance networks 的 scalability 和 stability。🔗 [RFC 編輯器](#)

簡單講：

CUBIC 是一個 loss-based TCP congestion control，但它比 NewReno 更適合高速、長距離網路。

CUBIC 的核心想法

CUBIC 不像 NewReno 那樣很單純地每 RTT 線性增加 congestion window。它用一個三次函數控制 cwnd 成長。

它會記得上次發生 congestion 前的 window size，稱為 W_{max} 。然後它用 cubic curve 慢慢接近、停留、再探測更高的 window。

直覺是：

先快速恢復到上次安全容量附近
接近 W_{max} 時變保守
超過 W_{max} 後再慢慢探索更多頻寬

CUBIC 為什麼厲害？

因為它在工程上很平衡：

- 比 NewReno 更適合高 BDP 網路
- 比單純 aggressive 的演算法穩定
- 比傳統 Reno-style TCP 更能利用高速長距離鏈路
- 有長期 deployment 經驗
- 是非常強的 default baseline

RFC 8312 也強調 CUBIC 的設計目標包含 high utilization、stability、RTT fairness、與 standard TCP friendly。🔗 [RFC 編輯器](#)

所以 CUBIC 很難打敗，因為它不是簡單規則，而是多年網路工程折衷後的結果。

5. NewReno 是什麼？為什麼它也厲害？

NewReno 是 Reno 的改良版，屬於經典 TCP congestion control family。

它主要改良的是 Reno 在 **fast recovery** 期間處理 multiple packet losses 的能力。RFC 6582 說 NewReno 修改 TCP fast recovery algorithm，讓 TCP 在收到 partial acknowledgment 時可以更好地處理一個 window 內多個 segment loss 的情況。🔗 [RFC 編輯器](#)

NewReno 的核心邏輯

NewReno 基本仍是 loss-based：

沒有 loss → 慢慢增加 cwnd
偵測到 loss → 降低 cwnd

它使用 TCP 經典的：

- slow start
- congestion avoidance
- fast retransmit
- fast recovery
- multiplicative decrease

NewReno 為什麼厲害？

不是因為它最新，而是因為它：

- 簡單
- 穩定
- 可解釋
- 是 TCP congestion control 的經典 baseline
- 在 loss-based 場景下很有代表性
- 很適合當「傳統 TCP」對照組

你可以把 NewReno 當成：

經典規則型 TCP 的代表。

在你的專題裡，NewReno 的價值不是要當最強 baseline，而是讓老師看到：

DQN 不是只和自己比 reward curve，而是有跟標準 TCP family 比較。

6. 四個方法的直覺比較

方法	類型	它看什麼訊號	控制方式	為什麼厲害
NewReno	loss-based classic TCP	packet loss / duplicate ACK	AIMD + fast recovery	簡單、穩定、經典 baseline
CUBIC	loss-based modern TCP	packet loss + cubic cwnd growth	用 cubic function 調整 cwnd	高速長距離網路利用率高，Linux 長期主流
BBR	model-based TCP	bottleneck bandwidth + min RTT	估計網路模型與 pacing rate	不等丟包才反應，能高 throughput / 控 delay
DQN (ours)	learning-based DRL	throughput / delay proxy / loss / previous action	學 policy : increase / keep / decrease	可從 feedback 學控制策略，揭露 DRL control design 限制

7. 一句話速記

你可以這樣記：

NewReno：經典老派，看到 loss 就退。
 CUBIC：現代主流，用三次曲線穩定探頻寬。
 BBR：模型派，估瓶頸頻寬和最低 RTT，不只看 loss。
 DQN：學習派，讓 agent 自己從 throughput / delay / loss 中學動作。

8. 你專題裡最漂亮的講法

你可以在報告中這樣講：

傳統 TCP baselines 代表三種不同設計哲學：NewReno 是經典 loss-based baseline，CUBIC 是現代高速長距離網路常用的 cubic-window loss-based algorithm，BBR 則是 model-based congestion control，透過估計 bottleneck bandwidth 與 propagation delay 來控制 sending behavior。相較之下，本專題的 DQN agent 是 learning-based 方法，它不依賴固定 TCP 規則，而是從 throughput、delay proxy 與 loss feedback 中學習動作。實驗結果顯示，DQN MVP 能建立完整 DRL pipeline，但 coarse discrete action 在高延遲場景下仍容易造成 high loss，因此後續需要 PPO / continuous action extension。

這段非常適合放在你的口頭報告或答辯裡。

PPO 又是什麼 為什麼我的期末報告原先要加上他

PPO 全名是 **Proximal Policy Optimization**，中文可以翻成 **近端策略最佳化**。它是 DRL 裡很常用的一種 **policy-gradient / actor-critic** 演算法。

你原先期末報告會想加上 PPO，核心原因不是「PPO 比 DQN 一定強」，而是：

你的 DQN 結果剛好暴露出 Discrete(3) action 太粗的限制；PPO 可以支援 continuous action，因此是很自然的下一步擴展。

1. PPO 是什麼？

先用一句話說：

DQN 是學 Q-value：在這個 state 做哪個 action 分數最高。PPO 是直接學 policy：在這個 state 應該用什麼機率選什麼 action。

DQN 學的是：

$Q(s, a)$

也就是「狀態 s 下，做動作 a 的長期價值」。

PPO 學的是：

$$\pi(a | s)$$

也就是「狀態 s 下，我應該如何選動作」。

更白話：

演算法	它在學什麼	像什麼
DQN	每個 action 的分數	考試時先幫每個選項打分數，再選最高分
PPO	直接學行為策略	直接學一套「遇到什麼情況要怎麼做」的決策習慣

PPO 原始論文說，它會反覆跟環境互動收集資料，再最佳化一個 surrogate objective；它的設計目標是保留 TRPO 的部分好處，但更容易實作、更 general、sample complexity 也更好。 [arXiv](#)

2. PPO 的「Proximal」是什麼意思？

Proximal 可以理解成：

不要讓 policy 一次改太大。

DRL 很容易出現一個問題：agent 今天學到一點東西，下一次更新 policy 時改太猛，結果整個策略崩掉。

PPO 的核心設計就是：

新 policy 可以比舊 policy 好，但不要離舊 policy 太遠。

Stable-Baselines3 文件也明確說，PPO 的主要想法是更新後的新 policy 不應該離舊 policy 太遠，因此 PPO 使用 clipping 避免過大的 update。 [Stable-Baselines3 文檔](#)

所以 PPO 的精神是：

可以進步，但不要暴衝。

這其實很適合 congestion control，因為壅塞控制也怕暴衝。發送速率一下加太多，就可能造成 queue 爆掉、delay 變高、loss 變高。

3. PPO 跟 DQN 最大差別在哪？

差別一：DQN 比較適合離散動作

你的 DQN action 是：

decrease / keep / increase

也就是三個離散動作。

這很適合 DQN，因為 Stable-Baselines3 的 DQN 支援 **Discrete** action space。 [Stable-Baselines3 文檔](#)

但問題是：這三個動作太粗。

在 congestion control 裡，真正想控制的是「發送速率要調多少」：

降低 3%
降低 8%
維持
提高 2%
提高 10%
提高 20%

可是你的 DQN 只有：

降低 / 維持 / 提高

它沒有細緻程度。

這就是為什麼你的 DQN 在 S1 可能變成 100% increase，在 S2 則出現 high loss。不是 DRL 完全沒用，而是 action design 太粗。

差別二：PPO 可以支援連續動作

Stable-Baselines3 的 PPO 文件列出 PPO 支援 **Discrete** action，也支援 **Box** action；**Box** 就是連續動作空間。

 [Stable-Baselines3 文檔](#)

這對你的專題非常重要。

因為如果用 PPO continuous action，你可以把 action 改成：

$a \in [-1, 1]$

再映射成：

$\text{rate_multiplier} = 1.0 + k * a$

例如：

PPO 輸出	對應發送速率調整
-1.0	大幅降低
-0.3	小幅降低
0	維持
0.2	小幅提高
1.0	大幅提高

這比 DQN 的三動作細很多。

所以 PPO 對你的專題的意義是：

它可能讓 agent 學到更細緻的 sending-rate adjustment，而不是只會 decrease / keep / increase。

4. 為什麼原本期末報告會想加 PPO ?

因為你的專題路線其實很自然：

Baseline TCP → DQN MVP → 發現 DQN 限制 → PPO / continuous action 擴展

這是一條很完整的 DRL 研究流程。

你的 DQN 結果不是很好看，但它給了你一個很好的研究洞察：

DQN 現象	暗示的問題	PPO 為什麼合理
S1 100% increase	agent 學到退化策略，不是真正細緻控制	PPO continuous action 可學「提高多少」
S2 high loss	三動作太粗，容易過度 aggressive	PPO 可做小幅調整，可能降低 loss
沒有全面勝 baseline	DQN MVP 不夠細緻	PPO 是下一個合理 DRL extension
rate control 不是 TCP cwnd	目前只是 abstraction	PPO 可作更細 rate-control prototype

所以 PPO 原先出現在報告裡，不是因為「一定要做」，而是因為它是 DQN 之後最合理的下一步。

5. PPO 為什麼有可能改善你的結果？

因為目前你的 DQN 問題主要不是「神經網路太小」或「訓練步數太少」而已，而是：

action granularity 太粗。

DQN 只能做：

-1 / 0 / +1

PPO continuous action 可以做：

-0.73 / -0.12 / +0.04 / +0.31 / +0.88

對 congestion control 來說，這很重要。

TCP / rate control 本質上不是只有三段式動作，而是連續調速問題。你真正想知道的是：

現在該把 **sending rate** 調高多少？
現在該把 **sending rate** 調低多少？

不是只問：

該不該提高？

所以 PPO 可能幫你解決：

- S1 不要永遠 100% increase
- S2 不要衝太快造成 high loss

- delay / loss trade-off 更細緻
- action distribution 更有變化
- utility 可能更穩定

但注意，我說的是**可能**，不是保證。

6. PPO 也不是魔法

你報告裡絕對不能寫：

PPO 可以反轉結果
PPO 一定比 DQN 強
PPO 一定打敗 TCP

要寫：

PPO / continuous action is a reasonable extension because it may address the coarse-action limitation observed in the DQN MVP. However, PPO does not guarantee better performance and still requires empirical validation.

中文：

PPO / 連續動作空間是合理的後續擴展，因為它可能改善 DQN MVP 中觀察到的粗粒度動作限制；但 PPO 不保證一定提升表現，仍需實驗驗證。

這樣最安全。

7. 為什麼「上 DRL 課」更應該提 PPO ？

因為 DRL 課通常不是只看你會不會跑一個 DQN，而是看你是否理解：

- value-based method
- policy-gradient method
- discrete action
- continuous action
- reward design
- exploration / stability
- evaluation
- baseline comparison
- failure analysis

你的專題如果只有 DQN，會像是：

我把 DQN 接上 ns-3，跑出結果。

但如果加上 PPO 作為 Phase 6，它會變成：

我先用 DQN 建立 MVP，發現 Discrete(3) action 限制；
因此進一步提出 PPO / continuous action 作為 extension，
用來處理更細緻的 sending-rate control。

這就更像 DRL 課的專題。

8. 你報告裡可以這樣介紹 PPO

你可以放這段：

PPO (Proximal Policy Optimization) 是一種 policy-gradient / actor-critic 類型的深度強化學習演算法。不同於 DQN 透過估計每個離散動作的 Q-value 來選擇行為，PPO 直接最佳化 policy，並透過 clipping mechanism 限制每次 policy update 的幅度，避免策略更新過大造成訓練不穩。由於 PPO 可支援 continuous action space，因此它適合作為本專題 DQN MVP 之後的自然擴展，用來處理 DQN Discrete(3) action 在 congestion control 中過於粗粒度的限制。

這段很適合放在報告 Future Work 或 Phase 6。

9. 最後用一句話總結

PPO 在你的期末報告中的角色是：

不是救火，不是硬補，不是保證反轉結果；PPO 是 DQN MVP 暴露出粗粒度離散動作限制後，最自然、最 DRL、也最有研究合理性的下一階段。

所以你原先要加它是對的。現在如果要真的做 Phase 6 PPO，也合理；但報告敘事要記住：**PPO 是用來驗證下一個 hypothesis，不是拿來保證贏。**